

Revisiting Keyed-Verification Anonymous Credentials

Michele Orrù
CNRS
m@orru.net

Abstract

Keyed-verification anonymous credentials (KVACs) have demonstrated their practicality through large-scale deployments in Apple, Google, Signal, and Tor. Despite their widespread adoption, the theoretical framework underlying KVACs lacks the flexibility needed to support diverse applications, which in general require different security properties. For instance, rate-limiting credentials only need a weaker unforgeability notion (one-more unforgeability), yet the framework cannot easily accommodate this relaxation. Similarly, digital identity protocols require stronger properties than unforgeability, specifically extractability for security proofs when adversaries can observe other users’ credentials.

We address these limitations by introducing new notions of extractability and one-more unforgeability. We improve two foundational works in the space:

- The scheme by Chase et al. (CCS 2014), commonly referred to as CMZ or PS MAC can be made statistically anonymous, and issuance cost reduced from $O(n)$ to $O(1)$.
- The scheme by Barki et al. (SAC 2016), known as BBDT or BBS MAC can be issued more efficiently (one less group element).

We provide a rigorous security proof for both schemes in the algebraic group model (CRYPTO 2018) and describe how these “core credential schemes” can be extended to construct more complex anonymous credential systems such as time-based policies, pseudonyms, and rate-limiting extensions.

Finally we note that, for KVACs, designated-verifier proofs suffice since the verifier is known in advance. We introduce designated-verifier polynomial commitment schemes and instantiate a variant of the popular KZG commitment scheme without pairings. We introduce a new IOP compiler for designated-verifier proofs, and use it to build designated-verifier fully-succinct zk-SNARKs without pairings for algebraic groups.

Keywords: Algebraic MACs, Keyed-Verification Anonymous Credentials, Anonymous Tokens

Contents

1	Introduction	3
1.1	Our contributions	3
1.2	Related work	5
2	Technical Overview	9
2.1	Review: keyed-verification credentials	9
2.2	New notions for keyed-verification credentials	13
2.3	μ CMZ credentials	14
2.4	μ BBS credentials	17
2.5	Building on KVACs: credential extensions	19
2.6	Straight-line extraction for Sigma protocols	21
3	Preliminaries	24
3.1	Cryptographic assumptions	24
3.2	Algebraic message authentication codes	24
3.3	Zero-knowledge arguments	25
3.4	Anonymous Tokens	26
4	Keyed-verification credential systems	28
4.1	Syntax	28
4.2	Correctness	29
4.3	Anonymity	30
4.4	Extractability	31
5	Chase–Meiklejohn–Zaverucha credentials	33
5.1	Protocol description	33
5.2	Theorems	35
5.3	Algebraic MAC	36
5.4	Anonymity	40
5.5	Extractability	42
5.6	One-more unforgeability	43
6	Boneh–Boyen–Shacham credentials	48
6.1	Protocol description	48
6.2	Theorems	50
6.3	Algebraic MAC	51
6.4	Anonymity	53
6.5	Extractability	55
6.6	One-more unforgeability analysis	56
7	Designated-verifier fully-succinct SNARKs without pairings	58
7.1	Designated-verifier Kate–Zaverucha–Goldberg commitments	59
7.2	IOP compiler for designated-verifier polynomial commitments	61
8	Building on keyed-verification credential systems	65
8.1	Time-based policies	65
8.2	Rate-limiting	65
8.3	Pseudonyms	71
9	Straight-line extraction from Σ-protocols	77
10	Acknowledgements	80
	References	80

1 Introduction

Anonymous credential systems, introduced by David Chaum [Cha85], allow a user to obtain certified credentials from an organization and later prove their possession, disclosing as little information as possible about the user’s identity. Chaum envisioned a system where different organizations would know a user by different pseudonyms. Pseudonyms are unlinkable, and colluding organizations cannot track a user. After Chaum, a practical anonymous credential system was proposed in Stefan Brands’s PhD thesis [Bra95; Bra00], leading to what is known today as the U-Prove technology from Microsoft Research [PZ13]. Under the hood, Brands credentials tweak blind Schnorr signatures [Sch01] to support digital identity claims to be efficiently tied to the use of tamper-resistant devices such as smart cards.

Signatures with efficient protocols. Camenisch and Lysyanskaya [CL01; CL03] approached the problem of anonymous credentials by constructing a signature scheme for which statements are easy to state using zero-knowledge proofs. Their idea is to have the issuer sign a commitment to the user’s attributes, and have the user later prove (in zk) knowledge of a signature on them. The proof might partially reveal some properties of the predicates, but nothing more than what can be directly inferred by the access policy itself. If the proof is valid, then via the knowledge extractor the reduction can recover a signature and a message, reducing the security of the credential to the one of the signature and the commitment scheme. If the message space of the signature scheme is “compatible” (*algebraic*) with the commitment space, proofs can be very efficient. This line of work led to the Idemix technology from IBM Research [CV02].

Algebraic MACs. Chase, Meiklejohn, and Zaverucha [CMZ14] remarked that, if the issuer and the verifier are the same entity, it is sufficient to rely on message authentication codes with similar algebraic properties (*algebraic MACs*), as opposed to signatures. They show that such schemes can be instantiated over prime-order groups without a bilinear pairing map. Such credentials are called *keyed-verification anonymous credentials*. The two core notions that they must satisfy are unforgeability (it’s not possible to present a credential for a statement that is false for all issued credentials), and anonymity (presentations are unlinkable across executions).

Theory gap. Anonymous authentication protocols often require more complex access policies than simple selective disclosure. In the keyed-verification setting, however, there is a lack of modular approaches for constructing such protocols. For many of these policies, the standard security notions for keyed-verification credentials are insufficient.

For example, on the web, servers frequently need to establish trust in clients and enforce rate limits on requests. In such cases where third-party verification is unnecessary keyed-verification credentials could be a compelling alternative to cookies. However, unforgeability alone is not the right notion of security here: what we need is one-more unforgeability rather than plain unforgeability. Additionally, real-world adversaries can observe credential presentations made by other users!

Similarly, the privacy notion of anonymity is fragmented into three distinct definitions: key consistency, blind issuance, and anonymity itself. As a result, it becomes difficult to understand exactly what privacy guarantees keyed-verification anonymous credentials provide.

1.1 Our contributions

This paper introduces new security notions for keyed-verification credentials. First, we propose a strengthened notion of anonymity that requires the existence of a simulator covering both issuance

and redemption. Then, we define two notions of unforgeability:

- *Extractability*, which requires the existence of an extractor capable of recovering the attributes from a credential;
- *One-more unforgeability*, a weaker notion, closer to that of anonymous tokens, which can lead to simpler and more efficient issuance protocols.

Extractability is stronger and more versatile than unforgeability of [CMZ14], and allows for arbitrary extensions to be proven secure; one-more unforgeability is weaker, but allows for simpler issuance protocols as, in general, it will not require the user to compute zero-knowledge proofs. We show that the notion of extractability already applies to (more efficient) variants of popular KVAC schemes, and that the notion of one-more unforgeability can be used to make these schemes simpler.

μCMZ . MAC_{GGM} (Chase, Meiklejohn and Zaverucha [CMZ14]) was proven secure in the generic group model. We show that it can be slightly modified to reduce issuance cost to 1 group element from $2n + 1$ (where n is the number of private attributes).

Theorem 1. *μCMZ (Figure 1) is a keyed-verification anonymous credential system with extractability advantage:*

$$2\text{Adv}_{\text{GrGen}}^{3\text{-dl}}(\lambda) + \frac{3}{p} + \text{Adv}_{\text{ZKP}}^{\text{zk}}(\lambda) + 2\text{Adv}_{\text{ZKP}}^{\text{simex}}(\lambda) ,$$

in the algebraic group model, and anonymity advantage:

$$2\text{Adv}_{\text{ZKP}}^{\text{zk}}(\lambda) + \text{Adv}_{\text{ZKP}}^{\text{simex}}(\lambda) .$$

The variant $\mu\text{CMZ}_{\text{AT}}$ is one-more unforgeable with advantage

$$O(q) \cdot \text{Adv}_{\text{GrGen}}^{\text{dl}}(\lambda) + 3\text{Adv}_{\text{GrGen}}^{2\text{-dl}}(\lambda) .$$

In particular, for $\mu\text{CMZ}_{\text{AT}}$, we show that the user stays secure even without the user sending the zero-knowledge proof at issuance time. When instantiated with sigma protocols, μCMZ is statistically anonymous.

μBBS . We alter BBDT (Barkai et al. [BBDT17]) to make it compatible with the ongoing IETF standardization effort [KB], improving issuance and presentation cost by sending one less group element for each. We show how, removing the zero-knowledge proof sent at issuance time, the scheme still satisfies a weaker notion of unforgeability (one-more-unforgeability), and prove its security in the algebraic group model, while previous analyses were in the generic group model or loose in the q -strong DH assumption.

Theorem 2. *μBBS (Figure 1) is a keyed-verification anonymous credential system with extractability advantage:*

$$\text{Adv}_{\text{GrGen}}^{(q+2)\text{-dl}}(\lambda) + \text{Adv}_{\text{GrGen}}^{\text{dl}}(\lambda) + 2\text{Adv}_{\text{ZKP}}^{\text{simex}}(\lambda) + \delta(\lambda, q) + \frac{3}{p} ,$$

in the algebraic group model, and anonymity advantage:

$$2\text{Adv}_{\text{ZKP}}^{\text{zk}}(\lambda) + \text{Adv}_{\text{ZKP}}^{\text{simex}}(\lambda) + \text{Adv}_{\text{GrGen}}^{\text{dl}}(\lambda) + \frac{1}{p} .$$

The variant $\mu\text{BBS}_{\text{AT}}$ is one-more unforgeable.

Credential extensions. In practice, the issuance and presentation proofs are often used in conjunction with more complex predicates to support the access policies of the system where they are being used. For this reason, on top of μCMZ and μBBS **credential extensions** can be built. Using our notions, we provide efficient modular extensions for:

- *credential expiry*, where the issuer can set an expiration date on the credential,
- *public metadata*, where the issuer can add public information to the credential,
- *pseudonyms*, where the user can present the same credentials using different pseudonyms,
- *rate-limiting*, where the issuer can limit the number of times a credential can be presented.

The relation family supported by the proof system used determines which extensions can be supported, and their efficiency.

Efficient designated-verifier SNARKs. We build a compiler that uses an interactive oracle proof (IOP) and a designated-verifier polynomial commitment to produce a designated-verifier zero-knowledge non-interactive argument. We build a designated-verifier polynomial commitment (denoted dvKZG) that does not require pairings, inspired by KZG.

Keyed-verification credentials, often, rely on zero-knowledge proofs. Since the verifier is known in advance and it's the issuer, malicious designated-verifier zero-knowledge proofs suffice. As an example application, we instantiate an efficient constant-size range proof that does not require pairings. While it is common to think of designated-verifier proofs as “more expensive than normal proofs” (1 NIZK + 1 OR proof), we show that this is not always the case.

Theorem 3. *Let IOP be a polynomial interactive oracle proofs. Let PCS be a designated-verifier polynomial commitment scheme. Then, the compiler $\text{ARK}[\text{IOP}, \text{PCS}]$ in Section 7.2 is a designated-verifier zero-knowledge non-interactive argument system.*

In particular, when an IOP is compiled with dvKZG , the resulting argument system is a fully-succinct designated-verifier zero-knowledge non-interactive argument system.

As an example, we provide a constant-sized range proof in the keyed-verification setting that is competitive with state-of-the-art range proofs.

1.2 Related work

The literature that builds modular credential systems is vast. Camenisch et al. [CKLMNP16] develop a modular framework for constructing anonymous credentials called *privacy-enhancing attribute-based credential systems*. Using a commitment scheme, a signature scheme (more precisely, a privacy-enhancing attribute-based signature scheme), a revocation scheme and a pseudonym scheme, they show how to compose them into a full-fledged credential system. Their framework can be instantiated with Camenisch–Lysyanskaya [CL01; CL03] and Brands signatures [Bra95; Bra00], and has been foundational for extensions on credential systems, such as *issuer-hiding attribute-based credentials* [BEKRS21], *encrypted attribute-based credentials* [KLSS17], and more. Chase and Lysyanskaya [CL06] introduce *signatures of knowledge*, signatures that can be issued on behalf of any NP statement. Belenkiy et al. [BCKL08] introduce the notion of *P-signatures*, a cryptographic primitive that consists of a signature scheme, a commitment scheme, a non-interactive protocol for obtaining signatures on committed values, and a non-interactive proof system for proving that a pair of commitments are commitments to the same value. Abe, Fuchsbauer, Groth et al. [AFGHO16] introduced the notion of *structure-preserving signatures*, signatures that lie in a bilinear group for which the verification equation consists of pairing products [GS08]. Fuchsbauer,

Hanser, and Slamanig [FHS19] introduce *structure-preserving signatures for equivalence classes*, and inspired *mercurial signatures* [CL19], centered around the idea that signatures that can be re-randomized instead of the commit and prove approach at credentials. Compared to all above works, instead of showing how to build full-fledged credentials from smaller cryptographic primitives, we define a (slightly different) core credential system primitive that supports issuance predicates, and then build on top extensions that can be added at the discretion of the credential designer.

Campanelli, Fiore, and Querol introduce LegoSNARK [CFQ19], a framework to build modular SNARKs that can be composed to enable different efficiency trade-offs. Their work is inscribed within the commit-and-prove paradigm and, more specifically, investigates SNARKs about commitments made ahead of time, without knowing the statement they are going to be proven on. Our work is complementary, and is meant to be used in tandem with commit-and-prove systems.

Deployment status. Real-world uses of credentials can be grouped into:

- (a) *Based on Camenisch–Lysyanskaya* [CL01; CL03]. Used in the Open Wallet Foundation’s Bifold [Ope] and Hyperledger’s anoncreds implementation [Hypa]. These credentials are specifically in the public verification setting, where the issuer and the verifier are different entities.
- (b) *Based on Chase–Meiklejohn–Zaverucha* (CMZ) [CMZ14; PS16]. These credentials are by far the most widely used in practice. Deployments of this family of credential systems are mostly in the keyed-verification setting (where the issuer and the redeemer are the same entity), despite a public-verification variant exists [PS16].

The most prominent use-case of CMZ is Signal, where credentials are used for “private group systems” [Sig; CPZ20]. In Signal, group membership is not stored by the server and, to verify group membership and role (e.g. admin), users present a CMZ credential to the server, which verifies it and then allows the user to perform actions such as adding new members or changing the profile picture.

The NYM Network [Nym] uses threshold issuance of CMZ credentials (called “Zk-nyms”) with selective disclosure of attributes to provide anonymous access to access or paying for services on their network.

The Tor Project deployed a system called Lox [TG23] which internally relies on CMZ credentials (used as spend-once credentials) for the distribution of bridges. (Bridges are entry nodes of the network which are not publicly advertised. They are used to counter censorship events that involve blocking of entry nodes.)

In all of the above, our contribution can offer faster efficiency and post-quantum anonymity guarantees.

More recently, Apple published an Internet Draft within the IETF [YW25] that uses the rate-limiting compiler and the PRF described in Section 8.

- (c) *Based on BBS signatures* [BBS04; TZ23]. Initiatives for this family of credentials are mostly in the public-verification setting, where the issuer and the redeemer are the same entity. The W3C hosts multiple efforts in this direction, more specifically on decentralized identity [SLSRSA] and verifiable credentials [SB]. The IETF currently has an ongoing proposal for BBS credentials [LKWL]. This is also what is adopted by Idemix [Hypb] for their DLOG credentials, which are based on the work of Au et al. [ASM06]. (Idemix is also implemented by IRMA [DS].) BBS are also at the core of Dock [Lab], a blockchain network that provides reusable digital identification and verifiable credentials.
- (d) *Tokens Based on Blind Signatures*. A number of show-once credentials have been created

in the past years. RFC9578 [CDVW24] internally relies either on the verifiable oblivious random function (VOPRF) of Jareki et al. [JKK14] or the Blind RSA signature scheme of Chaum [Cha82]. Google’s BoringSSL implements anonymous tokens [KLOR20] (which are lightweight spend-once keyed verification anonymous credentials). Blind RSA signatures are also used by Apple Cloud Relay [App] and Google One’s VPN service [Goo].

- (e) *Based on SNARKs*. This broad class of credentials generally relies on SNARKs and recursive SNARKs [Chi10], and generally boils down to black-box approaches of proving knowledge of a signature key, or membership in Merkle tree. This class generally lacks a public provable-security formalisation. Google [Fs24] recently released an anonymous credential system (“longfellow”) that uses Liger [AHIV17] to prove knowledge of a valid signature for legacy identity standards such as the ISO MDOC standard, the JWT standard, and W3 Verifiable Credentials. Semaphore [Eth] and Anon Aadhaar protocol [Ano] from Privacy Scaling Explorations (PSE) are a collection of tools used for building applications that leverage anonymous signaling on the Ethereum blockchain, relying on general-purpose zero-knowledge succinct arguments (zk-SNARKs). Zupass [0xP] is an authentication system based on the proof-carrying data paradigm.

Table 1: Concrete communication and space costs for some keyed-verification credential systems with n hidden attributes (public attributes at issuance/presentation are for free). The bit size of $\mathbb{G}$ is denoted g , the scalar field bit size s , and the security parameter λ . In [square brackets] we indicate the size for empty predicates or optional elements, using Schnorr proofs. ATR is checked if $n = \text{poly}(\lambda)$ attributes are supported, PMB denotes private metadata bit feature, NYM refers to pseudonyms, RTL to rate-limiting, THRLD to thresholding, and IBR to efficient issuer-hiding presentation [BEKRS21]. Empty cells represent open problems, not impossibility.

Scheme	Key material		Issuance		Presentation	Supported features						Security	
	$ pp $	$ \sigma $	$ I.Usr $	$ I.Srv $	$ P.Usr $	ATR	PMB	NYM	RTL	THRLD	IHP	Unforgeability	Anonymity
μCMZ , Figure 9	$(n+2)g$	$g + [g]$	$g + [(n+2)s]$	$2g + [3s]$	$(n+2)g + [(2n+2)s]$	✓	✓	✓	✓	✓		AGM + 3-DL	statistical
μBBS , Figure 10	g	$g + [2\lambda]$	$g + [(n+2)s]$	$g + 2\lambda + [2s]$	$2g + [(n+4)s]$	✓		✓	✓	✓		AGM + q -DL	statistical*
[BBDT17]	g	$g + [2\lambda]$	$g + [(n+2)s]$	$g + 2\lambda + s + [2s]$	$3g + [(n+7)s]$	✓		✓	✓	✓		q -SDH	statistical
[CMZ14]	$(n+1)g$	$g + [g]$	$(2n+1)g + [(2n+2)s]$	$3g + [(2n+4)s]$	$(n+2)g + [(2n+2)s]$	✓	✓	✓	✓	✓		GGM	DDH
[CPZ20]	$2g$	$2g + s$	$(3n+2)g + [4s]$	$3g + s + [(n+6)s]$	$(n+3)g + [4s]$	✓		✓	✓			GGM	DDH
[DGSTV18]	g	$2\lambda + [g]$	g	$g + [2s]$	4λ	✗	✗	✗	naïve	✓	✓	RO + gap-OMCDH	statistical

*A technicality in the simulator makes it difficult to simulate issuance when the user is invoked on a message $\tilde{m}$ such that $\sum_i m_i G_i = -G_p$. This can be circumvented setting the message space to exclude this bad case or relying on the DL assumption during credential issuance.

Table 2: Concrete sizes over 256-bit curves. Relevant examples are secp256k1 [Qu99], Ristretto [Ber06; Ham15], BN254 curves [BN06], Pallas, Vesta, and JubJub curves. Note that the comparison does not take into account that, to have comparable levels of security, μBBS and [BBDT17] should be instantiated over larger (≈ 300 bit) curves.

Scheme	Attributes $n = 1$					Attributes $n = 5$				
	Key material		Issuance		Presentation	Key material		Issuance		Presentation
	$ pp $	$ \sigma $	$ I.Usr $	$ I.Srv $	$ P.Usr $	$ pp $	$ \sigma $	$ I.Usr $	$ I.Srv $	$ P.Usr $
μCMZ , Figure 9	96B	64B	128B	160B	224B	224B	64B	256B	160B	608B
μBBS , Figure 10	32B	64B	128B	128B	224B	32B	64B	256B	128B	352B
[BBDT17]	32B	64B	128B	160B	352B	32B	64B	256B	160B	480B
[CMZ14]	64B	64B	224B	288B	224B	192B	64B	736B	544B	608B
[CPZ20]	64B	96B	288B	352B	256B	64B	96B	672B	480B	384B
[DGSTV18]	32B	64B	32B	96B	64B	32B	64B	32B	96B	64B

Table 3: Concrete sizes over ≈ 400 -bit curves. Relevant examples are BLS12-381 and BLS12-377 [BLS04].

Scheme	Attributes $n = 1$					Attributes $n = 5$				
	Key material		Issuance		Presentation	Key material		Issuance		Presentation
	$ pp $	$ \sigma $	$ I.Usr $	$ I.Srv $	$ P.Usr $	$ pp $	$ \sigma $	$ I.Usr $	$ I.Srv $	$ P.Usr $
μCMZ , Figure 9	144B	96B	144B	192B	272B	336B	96B	272B	192B	720B
μBBS , Figure 10	48B	80B	144B	144B	256B	48B	80B	272B	144B	384B
[BBDT17]	48B	80B	144B	176B	400B	48B	80B	272B	176B	528B
[CMZ14]	96B	96B	272B	336B	272B	288B	96B	912B	592B	720B
[CPZ20]	96B	128B	368B	400B	320B	96B	128B	944B	528B	512B
[DGSTV18]	48B	80B	48B	112B	64B	48B	80B	48B	112B	64B

2 Technical Overview

An anonymous credential is a signature or a MAC over some *attributes*. Instead of just revealing them, the user proves (in zk) that the attributes satisfy some properties. We call this to *present* or *show* a credential. The entity issuing a credential is called *issuer*; the entity to which a credential is issued *user*. The entity that verifies the credential is called *redeemer*.

Public- and keyed-verification. It is possible to distinguish two lines of literature in the credential space: (i) keyed-verification credentials, where the issuer and the redeemer are the same entity, and both hold the same signing key sk ; (ii) public-verification credentials, where the issuer and the redeemer are different entities. In this case, the issuer holds a signing key sk and the redeemer holds the respective verification key vk . The verification key vk is insufficient to produce new credentials. It is a common practice to build public-verification credentials from keyed-verification credentials (and vice-versa) assuming (respectively, removing) a pairing map in a bilinear group, but this is not true for all credential systems.¹ Two popular choices of credentials are Chase–Meiklejohn–Zaverucha [CMZ14] MACs, with their publicly verifiable variant Pointcheval–Sanders [PS16]; and Boneh–Boyen–Shacham [BBS04], with their keyed-verification variant Barki et al. [BBDT17]. This work focuses on keyed-verification credentials (KVAC).

Single-show and multi-show credentials. There is another axis over which credentials can be seen: (i) *Single-show credentials*, where a user will present a particular credential and that credential cannot then be used again. They can be very efficient, and can even be as simple as a blind signature scheme. Single use keyed-verification credentials are sometimes called “anonymous tokens” [KLOR20]. (ii) *Multi-show credentials*, where the user might want to present the same credential multiple times. Of course one can imagine an application that gives 1000 one-time tokens, and burns one of them at a time in place of multi-show credentials. However, this would put in the application layer the burden of tuning the number of credentials to be issued and when to refresh the pool of tokens to be spent. Schemes that are native specifically for the multi-show setting can be more efficient.

2.1 Review: keyed-verification credentials

Canonical keyed-verification credentials enjoy two core security properties: (a) *Anonymity*, which asks the existence of a simulator that, given as input the signing key, can simulate any credential presentation message (without knowing the attributes); (b) *Unforgeability*, which states that an adversary cannot successfully present a credential for a predicate that none of the previously-issued attributes satisfy.

Limitations of anonymity. In practice, the issuer can try to break anonymity by issuing maliciously-crafted credentials, for instance by maliciously choosing two signing keys associated to the same set of public parameters. These attack vectors are commonly included in the notion of “unlinkability” in the literature. For keyed-verification credentials, instead, handling such attacks is (in part) deferred to two separate properties:

1. *Key consistency*, which asks that the issuer cannot generate two signing keys for the same public parameters.

¹For instance, it seems unlikely to produce a “keyed-verification variant” of Groth’s structure-preserving signatures [Gro15] without a pairing map.

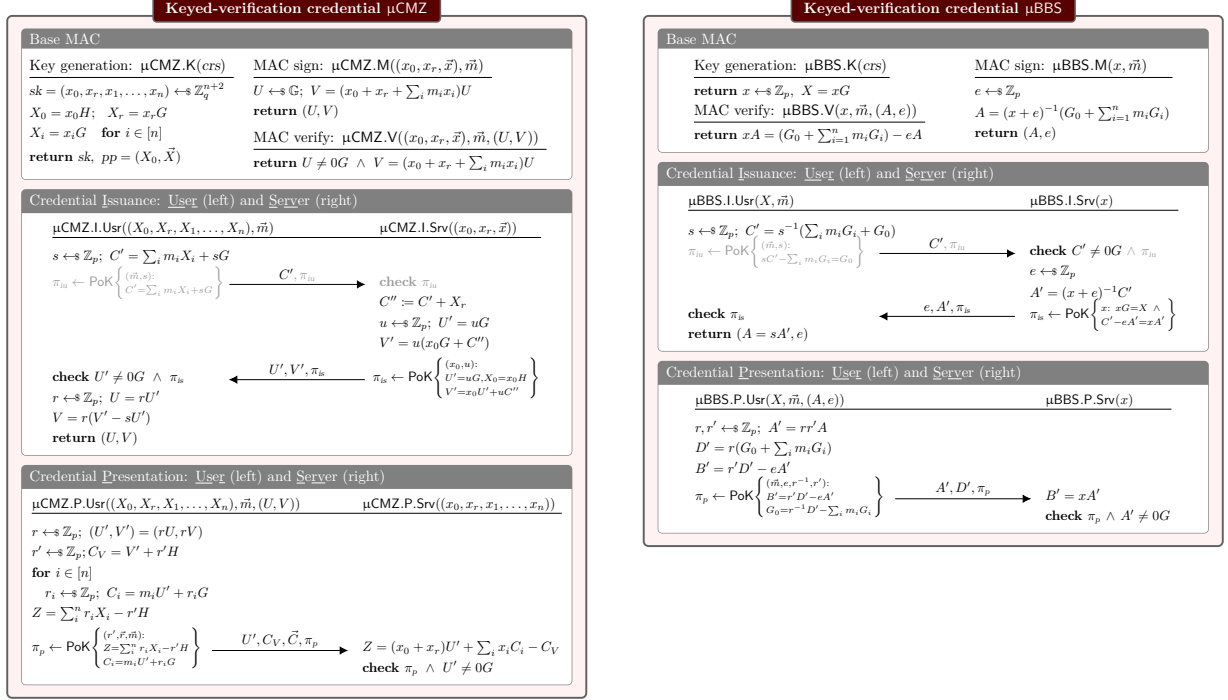


Figure 1: Overview of our novel keyed-verification credential systems Left to right: μCMZ , and μBBS . A more detailed description can be found in the full version Figures 9 and 10.

2. *Blind Issuance*, which asks that the issuance protocol is a multi-party computation protocol for the signing procedure, where the signing key is input by the issuer and the message by the user.

In recent times, it has emerged as a desirable property for anonymity to hold even against computationally unbounded adversaries. However, the compound anonymity guarantees of these definitions are not known.

Limitations of unforgeability. One possible attack surface not covered by unforgeability, but generally considered in practice, is security against eavesdroppers. This attack vector considers adversaries that can also observe (and manipulate!) credentials being issued to other users. For example, if an honest user receives a credential for some private key later used to authorize payments, a realistic threat model should include adversaries who can observe successful transactions and subsequently attempt to forge new ones.

One-more unforgeability vs unforgeability. Spend-once credentials are popular type of “lightweight” anonymous credential that have gained traction in the real-world. They rely on a weaker notion of unforgeability, called *one-more unforgeability*. In this setting, it only matters that the user can’t present more than issued. Exploiting the weaker security requirements, it is possible to build more efficient constructions in this setting. In particular, this notion will be sufficient for rate-limiting and pseudonyms (cf. Section 8).

2.1.1 Chase–Meiklejohn–Zaverucha credentials

Let $\mathbb{G}$ be an additive group of order p where discrete logarithm is hard, with generators G and H . Given a secret key $(x_0, x_r, x_1, \dots, x_n)$, which is a $(n+2)$ -tuple of integers uniformly distributed modulo a large prime p , a CMZ credential for attributes $m_1, \dots, m_n \in \mathbb{Z}_p$ is a pair

$$(U \leftarrow \$ \mathbb{G}^\times, \quad V = (x_0 + \sum_i x_i m_i) U) \quad . \quad (1)$$

Unforgeability of CMZ has been shown in the generic group model [CMZ14]. In a pairing group, when given a “verification key” in the “second group”, i.e., given $vk = (\bar{X}_0 = x_0 \bar{G}, \bar{X}_1 = x_1 \bar{G}, \dots) \in \bar{\mathbb{G}}^{n+1}$, the verification equation can be checked in the “target group” checking $e(V, \bar{X}_0) = e(U, \bar{G}) + \sum_i e(m_i U, \bar{X}_i)$, where $e: \mathbb{G} \times \bar{\mathbb{G}} \mapsto \mathbb{G}_T$ denotes the bilinear pairing map. The resulting scheme is known as the Pointcheval–Sanders signature scheme [PS16; PS18], proven secure in the generic group model.

Blind issuance. Sometimes, we need to issue a MAC over some hidden (“blind”) attributes, signed by the server, without fully knowing them. A classic use-case is when the credential is signing also a user’s secret key. In these cases, the approach of Chase et al. [CMZ14] is to use the homomorphic properties of ElGamal: given ciphertexts $(E_{i,1} = r_i G, E_{i,2} = r_i Y + m_i G)$ of each attribute m_i (encrypted by the user under their public key Y), one can blindly issue a credential by sampling $u \leftarrow \$ \mathbb{Z}_p$, computing

$$\begin{aligned} \begin{bmatrix} D_1 \\ D_2 \end{bmatrix} &= \begin{bmatrix} 0 \\ ux_0 + G \end{bmatrix} + u \sum_i x_i \begin{bmatrix} E_{i,1} \\ E_{i,2} \end{bmatrix} \\ &= \begin{bmatrix} \sum_i x_i r_i U \\ (x_0 + \sum_i x_i m_i) U + u \sum_i x_i r_i Y \end{bmatrix} \quad , \end{aligned}$$

and responding a credential ciphertext $(U = uG, D_1, D_2)$ to the user. The pair (D_1, D_2) decrypts (under y) to a credential (U, V) valid for $m_1, \dots, m_n$. For anonymity, the server must also prove (in zero-knowledge) correct computation of the above, under some public parameters:

$$pp = (X_0 = x_0 H + x_r G, \quad X_1 = x_1 G, \quad \dots, \quad X_n = x_n G) \quad .$$

(Nota bene: X_0 is committed as a Pedersen with blinding factor x_r to prevent some trivial forgeries.)

The above approach relies on semantic security of ElGamal encryption and therefore the user’s attributes are at best computationally hidden.

Presentation. Thanks to the algebraic structure of the MAC, it is possible to efficiently prove knowledge of a MAC without revealing it. A common approach here is to prove that a committed signature satisfies the verification equation for a given set of committed messages. Given a MAC (U, V) valid for attributes $(m_1, \dots, m_n)$, let:

$$(U', C_V) = (rU, rV + r'H) \text{ where } r, r' \leftarrow \$ \mathbb{Z}_p$$

be a commitment to the MAC, and

$$C_i = m_i U' + r_i G \text{ where } r_i \leftarrow \$ \mathbb{Z}_p$$

be commitment to the attributes. If (U, V) is a valid MAC for $m_1, \dots, m_n$, then it satisfies:

$$x_0 U + \sum_i x_i \cdot m_i U - V = 0G \quad ,$$

which in turn means that:

$$x_0 U' + \sum_i x_i C_i - C_V = \sum_i r_i X_i - r' H . \quad (2)$$

The above equation can be checked in zero-knowledge using Σ -protocols for linear relations. The user sends commitments to the attributes $C_1, \dots, C_n$ and to the MAC (U', C_V) as defined above, and then computes $Z = \sum_i r_i X_i - r' H$. Sends a zero-knowledge proof of knowledge of representation for T in bases $X_1, \dots, X_n, H$ using $r', r_1, \dots, r_n$. This is the right-hand side of Equation (2).

The issuer, upon receiving commitments to the attributes $C_1, \dots, C_n$, a MAC (U', C_V) , and the representation proof (denoted π_p), uses the key $(x_0, x_r, x_1, \dots, x_n)$ to re-compute T as $Z := x_0 U' + \sum_i x_i C_i - C_V$ (this corresponds to the left-hand side of Equation (2)) and checks the zero-knowledge proof.

Limitations in practice. CMZ credentials, as defined in [CMZ14], are often attributed the following limitations.

- *Linear issuance and presentation costs:* excluding the cost of zero-knowledge proofs (which can be sub-linear), issuing a credential costs $2n + 1$ group elements (two for each ElGamal encryption, plus a public key), and presenting a credential costs $n + 1$ group elements. This limitation is highlighted especially when comparing them to [BBDT17], which has constant issuance cost, even when instantiating the zero-knowledge proof using Σ -protocols.
- *Weak anonymity guarantees:* while presentation is perfectly zero-knowledge, issuance relies on ElGamal encryption, which computationally protects the user's attributes, and as a consequence the user's anonymity relies at best on the hardness of the decisional Diffie–Hellman problem. Additionally, the server's public key contains a Pedersen commitment, which unfortunately is computationally binding. An adversary able to break DL may be able to issue two credentials valid under two different secret keys, and as a consequence de-anonymise users.

2.1.2 Barki et al.'s credentials

A BBS+ anonymous credential [BBS04; ASM06] for attributes $m_1, \dots, m_n \in \mathbb{Z}_p^n$ is a triple

$$\left(\begin{array}{l} A = \frac{1}{x + e} (G_0 + \sum_i m_i G_i + sG), \\ e \leftarrow \mathbb{Z}_p, \\ s \leftarrow \mathbb{Z}_p \end{array} \right) \quad (3)$$

where x is the signing key, e, s are random in $\mathbb{Z}_p$, and $(G_0, \dots, G_n)$ are random independent generators of $\mathbb{G}$. The verifier may check in a pairing group that $e(A, \bar{X} + e\bar{G}) = e(G_0 + \sum_i m_i G_i + sG, \bar{G})$ where $\bar{X} = x\bar{G}$ is the verification key. Barki et al. [BBDT17] propose a keyed-verification variant where verification is performed using x , without relying on a pairing map.

Blind issuance. To issue a BBS MAC without knowing the message being authenticated, BBDT relies on the “ s ” term: the user computes the commitment $C = sG + \sum_i m_i G_i$ and sends it to the issuer, together with a zero-knowledge proof of representation in $G_1, \dots, G_n, G$. The issuer computes the MAC over the commitment as $A = (x + e)^{-1} (C + G_0 + s'G)$ for random $e, s' \leftarrow \mathbb{Z}_p$. The final MAC is $(A, e, s + s')$. This approach is also taken in the current blind BBS RFC draft [KB], where the “ s ” term component is called *secret blind*.

Presentation. To present a credential, the user sends a commitment to the MAC and to the attributes, and proves in zero-knowledge that the MAC is valid for the attributes.² Let

$$A' = rr'A, \quad \text{and} \quad D' = r(G_0 + \sum_i m_i G_i + sG) ,$$

for $r, r' \leftarrow \mathbb{Z}_p$ blinding factors. If (A, e, s) is a valid MAC for $m_1, \dots, m_n$, it satisfies:

$$xA' = rr' \cdot xA = rr'(G_0 + \sum_i m_i G_i + sG - eA) = r'D' - eA' .$$

The above can be proven in zero-knowledge using Σ -protocols for linear relations. The user sends the commitments to the MAC A', D' as defined above, and then computes $B' = r'D' - eA'$. Sends a zero-knowledge proof of knowledge of representation for B' in bases D', A' using the scalars r', e and a representation for G_0 in bases $D', G_1, \dots, G_n, G$ using $r^{-1}, -m_1, \dots, -m_n, -s$. That is, prove the following equalities:

$$\begin{aligned} B' &= r'D' - eA' \\ G_0 &= \frac{1}{r}D' - \sum_i m_i G_i - sG . \end{aligned}$$

Limitations in practice. In Barki et al.'s blind issuance protocol, the server signs a commitment of the form $G_0 + m_1 G_1 + \dots + m_n G_n + sG$, where the blinding factor s *cannot* be later "unblinded" by the user. This blinding factor serves no purpose for the security of the MAC itself (as suggested by the analysis in [TZ23]), yet it must be carried with the signature since it's essential for hiding the attributes during issuance. Therefore: when proving knowledge of attributes, the user must always include s while treating it differently from other attributes, as revealing s would immediately compromise anonymity. Additionally, this structure is incompatible with the RFC draft specification [LKWL], creating an interoperability gap between blind and plain signatures in the BBS+ ecosystem.

2.2 New notions for keyed-verification credentials

We introduce new notions of security for keyed-verification credentials: anonymity and extractability.

Anonymity. We study a strong notion of anonymity, which implies the one in [CMZ14]. A game-based definition is available in Theorem 4.4. It consists of two adversaries, A, D , which are asked to distinguish between:

- A real interaction, where A is given as input crs , and can interact with the user. The adversary can try to issue them a (potentially malicious) credential using arbitrary credential attributes and issuance predicates (valid for the given credential attributes). If a credential is successfully produced at the end of this interaction, the distinguisher D can interact with the user an arbitrary number of times for arbitrary predicates.
- A simulated interaction, where A and D interact with a simulator. The simulator is given as input only the public parameters, the issuance and presentation predicates.

²We describe here the approach of Tessaro and Zhu [TZ23] in place of the original proof statement of [BBDT17] as it is more efficient and applies in a straightforward way to the keyed-verification setting.

When considering long-term user anonymity, two flavors of this notion are desirable:

- **statistical anonymity**, which demands that computationally-unbounded adversaries A and D cannot de-anonymise users. The threat model here is
- **everlasting forward anonymity**, where A is computationally bounded and D is computationally unbounded.

In both notions, the adversary A plays the role of a malicious issuer. While the former is stronger, the second is more still effectively captures evesdroppers that store credentials and try to de-anonymise users later.

Extractability. We study a strong notion of unforgeability for credentials, which implies the one in [CMZ14]. A game-based definition is available in Theorem 4.5. In our notion, the adversary can ask for credentials for themselves, or for honest users. (In particular, this captures also man-in-the-middle attacks that can observe issuance and presentation of credentials to honest users.) We allow for arbitrary **issuance predicates** to be proven upon issuance (not present in the original syntax of keyed-verification credentials, but recently explored in the context of blind signatures by Fuchsbauer and Wolf [FW24]) and **presentation predicates** to be proven upon redemption. Two efficient extractors (one during issuance, and one during redemption) are asked to recover the messages issued to (and presented by) the adversary. The adversary wins if any of the presented credentials extracts to a message that was not previously issued to them, or if the statement is not verified by an extracted message.

2.3 μ CMZ credentials

2.3.1 Outline

In Figure 1 we present μ CMZ. The changes with respect to MAC_{GGM} are summarized below. We first modify the underlying MAC, and replace its public parameters with $X_0 = x_0H, X_r = x_rG, \dots, X_n = x_nG$ and the verification equation with:

$$V = (x_0 + x_r + \sum_i x_i m_i)U . \quad (4)$$

This modification is required in order to make the public parameters perfectly binding to the secret key and, looking ahead, this modification will be useful to prove statistical anonymity.

We then modify issuance, and replace the user’s issuance message with a Pedersen commitment to the attributes. More specifically, the user sends

$$C' = \sum_i m_i X_i + sG \quad (5)$$

where $X_1 = x_1G, \dots, X_n = x_nG$ are part of the public parameters. Then, the signer computes (U', V') using the homomorphic properties of Pedersen commitments. (We focus on the user algorithm for now.) The user can unblind the credential subtracting sU' from C' . This variant of the issuance protocol is more efficient (as it is independent of the number of attributes) and perfectly hides the attributes.

Remark 2.1. *The two edits shown in Equations (4) and (5) can be seen as independent modifications. The issuance technique shown in Equation (5) has been previously introduced in [PS16], but not*

formally proven. The main differences, beyond our formal treatment in a different model, lie in the underlying MAC (which has a different structure). In [PS16], the issuer must add new elements to the list of public parameters, while in the keyed-verification credential setting we don't, but the server must additionally prove correctness in zero-knowledge (via the proof π_{is} in the issuance protocol). Note that π_{is} provided by the server is shorter than the original proof of MAC_{GGM} .

2.3.2 Analysis

We outline the salient points and proof strategy of Theorem 1 (formally stated in Theorem 5.2).

MAC unforgeability. First, we prove that μCMZ is unforgeable in the algebraic group model [FKL18] assuming it is hard to find x given $(G, xG, x^2G, x^3G) \in \mathbb{G}$. (This assumption is known as 3-DL.) The full proof is available in Theorems 5.4 and 5.5. The proof is done in two steps. First, we prove for the special case $n = 1$ (where the bounds are lower). Then, we argue the case of $n > 1$ by reducing it to the base case.

One of the main takeaways that our security analysis highlights is that CMZ requires only a constant number $q = O(1)$ in the q -DL reduction. As a consequence, μCMZ can rely on 256-bit curves for 128-bit security. This is a major difference from BBS credentials, which are often thought as more efficient. See Table 1 for a comparison of the security assumptions of the different credentials.

Extractability. When proving extractability (Section 5.5), one seeks a reduction from the credential scheme to the underlying MAC. At a high level, an adversary for μCMZ has three available strategies to break extractability:

1. At issuance time, send a message (C', π_{in}) where π_{in} verifies, but for which extraction fails. This happens if it extracts an $m_1, \dots, m_n$ for which the predicate ϕ is not satisfied.
2. At presentation time, send a message $(U', C_V, C_1, \dots, C_n, \pi_p)$ where π_p verifies, but for which extraction fails. This happens if the extractor returns $m_1, \dots, m_n$ for which $\phi(m_1, \dots, m_n) = 0$.
3. The adversary returned a presentation message $(U', C_V, C_1, \dots, C_n, \pi_p)$ for which the extractor of π_p recovered a list of attributes $m_1^*, \dots, m_n^*$ that satisfies the predicate, but $m_1^*, \dots, m_n^*$ has not been previously queried.

The first two items reduce trivially to soundness of the proof system, while the third one should intuitively reduce to MAC unforgeability. The reduction **B** to MAC unforgeability must internally use the adversary **A**, which takes as input some “challenge” public parameters $(X_0, X_r, X_1, \dots, X_n)$, and interact with it in order to produce a forgery for the MAC scheme. **B** can issue credentials to **A** by:

- (a) extracting the attributes $m_1, \dots, m_n$ from the user proof π_{in} ,
- (b) use one query the signing oracle to obtain a MAC (U, V) for the attributes $m_1, \dots, m_n$,
- (c) use the zero-knowledge simulator to produce a proof π_{is} for the issuer.

However, it is hard for **B** to check a credential being presented: in order to extract (or even verify) the proof material π_p , the reduction **B** has to compute the element Z (cf. Figure 1), for which the secret key is required.

Fixing the proof of Chase et al. This problem is also present in standard unforgeability of MAC_{GGM} and the analysis of Chase et al. [CMZ14] is inaccurate. The reduction **B** must be able to produce Z without knowing the secret key, and this requires an additional assumption. We tackle this problem in the following way:

- First, we prove a *stronger* MAC unforgeability experiment, where the adversary is given an additional oracle in addition to the one for MAC verification (cf. Section 5.2). The auxiliary oracle, given as input $A_0, A_1, \dots, A_n, Z$, returns 1 if $(x_0 + x_r)A_0 + \sum_i x_i A_i = Z$, and 0 otherwise. This stronger notion sets the ground for later checking that a certain candidate Z is correctly computed.
- Then, we ask that from a proof it is possible to recover a candidate instance for which it is proven. This allows us to recover a candidate “ Z ” from the proof π_p without knowing the secret key. For an extracted instance, we have no guarantee that it is correct (even if the proof is valid!). This is a new, ad-hoc assumption for the zero-knowledge proof system.

The above approach allows us carry out a reduction to MAC unforgeability, and conclude the extractability proof.

One-more unforgeability. It is possible to see the MAC procedure as computing an authentication code $(U = uG, x_0U + uC)$ for an arbitrary group element $C \in \mathbb{G}$. No assumption on the representation of C is required, and in particular the proof π_{iu} need not be sent.

In this setting, while unforgeability under chosen message attack cannot be proven (because we don’t know the message being signed), it is possible to prove one-more unforgeability in the algebraic group model.

The main takeaway of our analysis is that π_{iu} may be removed, but the analysis requires different techniques from the unforgeability proof. The reduction has non-tight security bounds to the discrete logarithm problem.

Statistical anonymity. The Pedersen commitment sent during issuance in $\mu\text{CMZ.I.Usr}$ perfectly hides the user attributes (since Pedersen commitments are in fact perfectly hiding). Compared to MAC_{GGM} , where the user attributes are encrypted using ElGamal, this represents a step forward in achieving statistical anonymity.

However, one of the main limitations in statistical anonymity for MAC_{GGM} is that the public parameters are only computationally binding the server to a set of secret keys: a malicious server still has a chance at de-anonymising users by using two different x_0, x'_0 associated to the same public key. An adversary can find x_0, x'_0, x_r, x'_r such that $X_0 = x_0H + x_rG = x'_0H + x'_rG$, and this is as hard as solving the DL of H base G . A valid attempt for achieving perfect anonymity is to set $x_r = 0$, leading to $X_0 = x_0H$, which is perfectly binding and computationally hiding x_0 . This however leads to a secure credential system for which forgeries on the zero message are easy: in fact, all signatures of the form $(U = \alpha H, V = \alpha X_0 = \alpha x_0 H)$, for any $\alpha \in \mathbb{Z}_p$, are valid.³ We solve this issue by making the server publish $X_0 = x_0H$ and $X_r = x_rG$, essentially fixing a “fictitious message” to 1. We define a μCMZ credential for attributes $m_1, \dots, m_n \in \mathbb{Z}_p^n$ is a pair

$$(U \leftarrow_{\$} \mathbb{G}^\times, \quad V = (x_0 + x_r + \sum_i x_i m_i) U) \quad (6)$$

where U is sampled uniformly at random from $\mathbb{G}$. Since now the server signs using $x_0 + x_r$, but the two “shares” of the message-independent term are published in different bases H and G , an adversary can’t just re-use the public parameters to forge a credential. The rest of the protocol in Figure 1 is adapted to the above change, at no additional communication cost for the user or the issuer.

Remark 2.2. *Providing statistical anonymity required us to make two changes compared to CMZ_{GGM} :*

³The author thanks Jacques Traoré for spotting this mistake in a previous version of this paper.

1. Replacing the encrypted user attributes at issuance time with Pedersen commitment to them;
 2. Replacing the core MAC so that the public parameters are perfectly binding to the secret key.
 The threat model we consider here is the following: what if someone (e.g., the issuer) is able to solve the discrete logarithm (DL) problem at issuance time, while the user is not? Can the user still remain anonymous?

The (weaker) notion of everlasting forward anonymity (cf. Theorem 4.4), where $\mathbf{A}$ is computationally bounded and $\mathbf{D}$ is not, instead considers issuers who are unable to solve DL at issuance time, but may attempt to de-anonymise users later with the help of a computationally unbounded adversary. In this case, one does not require the server to change its public parameters, and only Item 1—i.e., the issuance procedure—needs to be modified.

2.4 μ BBS credentials

2.4.1 Outline

An informal presentation of μ BBS is in Figure 1. The changes with respect to the original scheme are summarized below.

The main change consists in fixing $s = 0$ and hereby using the shorter MAC (A, e) , as described Tessaro and Zhu [TZ23], and the recent BBS draft [LKWL]. The challenge in using this variant lies in enabling blind issuance with selective disclosure in the credential system, and to “unblind” the s term once signed, without carrying it along as part of the final MAC. We address this challenge by relying on *multiplicative* (instead of additive) blinding⁴: $C = s^{-1}(\sum_i m_i G_i + G_0)$ for a uniformly random $s \neq 0$.

We show how a representation proof can still be carried out at no additional cost, proving knowledge of $m_1, \dots, m_n, s$ such that

$$sC - \sum_i m_i G_i = G_0 .$$

(This is part of the issuance proof π_{in} sent by the user, which is needed to reduce blind issuance of a credential to unforgeability of the underlying MAC scheme.) The issuer computes the MAC as before, that is $(A = (x + e)^{-1}(C + G_0), e)$ for a random $e \leftarrow \mathbb{Z}_p$. The issuer’s computation must be shown to have been done using $x = \log_G X$, which is generally done with the help of a zero-knowledge proof. The user can unblind the MAC by multiplying A by s .

Remark 2.3. *A major drawback of multiplicative blinding is that now user commitments are not homomorphic, and therefore it becomes hard for the server to explicitly set metadata attributes for the user. It is however still possible to re-randomize user attributes. We discuss this in more detail in Figure 14.*

2.4.2 Analysis

We outline the salient points and proof strategy of Theorem 2 (formally stated in Theorem 6.6).

MAC Unforgeability. We prove μ BBS is unforgeable in the algebraic group model assuming it is hard to find x given $(G, xG, \dots, x^q G) \in \mathbb{G}$. (This assumption is known as q -DL.) The full proof

⁴In parallel, Durak et al. [DMTV24] used a similar technique in the context of anonymous tokens. The security requirements here (and consequently the proof) are different.

is available in Theorems 6.8 and 6.9. The proof is done in two steps, by first proving security for the special case $n = 1$, and then argue the case $n > 1$ by reducing it to the base case.

Remark 2.4 (Comparison with [TZ23]). *The BBS signature analysis Tessaro and Zhu [TZ23] relies on a pairing map for checking MAC validity. In our setting (keyed-verification, without a pairing map) we instead rely on a higher degree for the discrete logarithm challenge in the $q + 2$ -DL assumption. We provide a more clean distinction for the reduction when $n = 1$.*

Fixing the proof of Barki et al. Extractability can't be proven solely relying on MAC unforgeability. One of the points overlooked in the analysis of [BBDT17] is that the reduction B must be able to produce B' without knowing the secret key, in order to check (and extract!) from the user presentation message.

To circumvent this problem, in Section 6.3 we must prove a stronger unforgeability game where the adversary is also given a DDH oracle $\text{DDH}(A, Z)$ that returns 1 if $Z = xA$. Such oracle is straightforward to implement in the algebraic group model and does not affect the security bounds.

We assume that, from the zero-knowledge proof ZKP, it is possible to retrieve a candidate instance, as in the previous section. The candidate instance can then be checked against the DDH oracle, and the reduction B can check the validity of the user presentation message without knowing the secret key.

One-more unforgeability. It is possible to view the MAC procedure as computing an authentication code $(A = (x + e)^{-1}C)$ for a group element $C \in \mathbb{G}$, where no assumption about the algebraic representation of C is made.

Also here, one-more unforgeability can be proven without reliance on the user's issuance proof π_{in} . However, *caveat emptor!* While μBBS and μBBS_{AT} both rely on q -DL, for μBBS best attacks have complexity $\leq O(\sqrt{q})$, while for μBBS_{AT} they have complexity $O(\sqrt{d} + \sqrt{(p \pm 1)/d})$ where $d \mid p \pm 1$ is the number of issuance sessions required (internally, it relies on Cheon [Che06]). Realistically, d can be upper-bounded at 2^{40} , and thus around 20 bits of security are lost. The attack is shown in Theorem 6.12.

The only way in which we know to break μBBS (with π_{in}) and BBS signatures is the reduction by Jao and Yoshida [JY09], which requires all signatures to be on the same message, an important restriction of real-world cases. In the one-more unforgeability setting, this restriction is lifted (cf. Theorem 6.12).

Statistical Anonymity. It is fairly straightforward to build a simulator that produces issuance and presentation messages that are indistinguishable from real user interactions. However, a minor technical subtlety hinders statistical anonymity in μBBS . The commitment C is statistically close to uniformly distributed and perfectly hides $m_1, \dots, m_n$, except in the case where

$$\sum_i m_i G_i = -G_0 \quad , \quad (7)$$

which would yield a solution to the discrete logarithm problem. Simulating such an interaction is therefore challenging for a simulator that does not know $m_1, \dots, m_n$. This issue appears to be purely theoretical: a real-world implementation can always exclude messages satisfying Equation (7) from the message space. We discuss this more in detail in Section 6.4.

2.5 Building on KVACs: credential extensions

At credential issuance and presentation time, the zero-knowledge proofs π_{iu} and π_p prove knowledge of some attributes $m_1, \dots, m_n$ (in the case of credential presentation, also knowledge of a valid MAC). In conjunction with the “core” statement being proven (which proves in zero-knowledge the MAC verification equation) it is possible to *extend* the proof statement and prove (both at issuance and presentation) other properties of the user attributes. We use this feature to build more complex access policies, which we call **credential extensions**.

The relation family supported by the proof system determines which predicates can be proven about authenticated attributes. For instance, if the proof system supports arithmetic relations, one can prove that an authenticated age attribute is greater than 18, while if it only supports equality relations, such range proofs would not be possible. We denote such predicates as ϕ . (In the formal descriptions for μCMZ and μBBS in Figures 9 and 10, they are part of the input for issuance and presentation.)

Some examples of extensions relevant in practice are described below: rate-limiting credentials and pseudonyms (e.g. see the recent Internet Drafts [YW25; KB25]).

Rate-limiting. In rate-limiting anonymous tokens (Section 8.2), a server wants to budget the number of requests users can make, without tracking them.

A common approach here is to issue many blind signatures (or verifiable oblivious random functions) upon authentication [RWM; HIPVW24]: the issuer holds a list of “spent tokens”, and issues batches of credentials to users, each for a different hidden attribute (a *token*), that is revealed upon presentation. However, this approach is very expensive, and different techniques have been introduced to scale this solution in large services doing rate-limiting across different *scopes* (e.g., a URL) [SS22; AYY25]. We follow a different approach, initially described in the context of k -times anonymous credentials [TFS04; CHKLM06]: use a PRF evaluation over the pair (scp, i) , where $0 \leq i < \ell$ is a counter kept by the user for each access in scp .

A game-based formal definition of rate-limiting credentials is provided in Section 8.2. Informally, such a system must satisfy:

- *correctness*, that is, all tokens issued to honest users verify;
- *unforgeability*, that is, users cannot present more tokens than allowed by the system. This is formalized with a game where an adversary, upon receiving as input the public parameters, can issue credentials for themselves or for honest users. The adversary can also observe credential presentations from other users. At the end, the adversary outputs $\ell + 1$ valid credentials for the same scope, and wins if they are all valid.
- *unlinkability*, that is, the server cannot link two different credentials issued to the same user.

To achieve this, we rely on a PRF that is **compatible** with the keyed-verification credential system, that is: the PRF key is in the message space of the credential, and the zero-knowledge proof system can express the relation that the PRF is evaluated correctly. In Section 8.2.2, we introduce

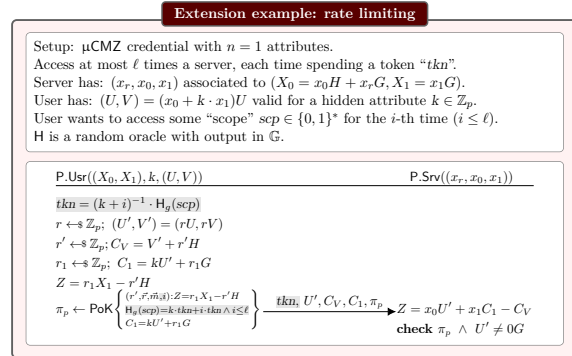


Figure 2: Example: rate-limiting extension on μCMZ .

a compiler that, given as input a keyed-verification credential system and a PRF compatible with it, produces a rate-limiting anonymous token.

Informally, the user holds a (secret) PRF key k and uses (blind) credential issuance to receive a credential for it. From it, the user can generate ℓ different tokens for a given scope $scp \in \{0, 1\}^*$. To do so, it sends $\text{PRF}(k, (i, scp))$, proving that the PRF is evaluated correctly using a k for which a credential was given, and that $i \leq \ell$. By pseudorandomness of the PRF, the values $\text{PRF}(k, (i, scp))$ are all random and independent. By one-more unforgeability of the credential system, the user cannot present more than ℓ credentials for the same scope.

We propose a PRF inspired from Dodis–Yampolskiy [DY05], which given as input a secret key $k \in \mathbb{Z}_p$ and a “message” $(i, scp) \in \text{poly}(\lambda) \times \{0, 1\}^*$, outputs a pseudorandom value in $\mathbb{G}$:

$$\text{HashDY}(k, (i, scp)) = \frac{1}{k + i} H(scp) \ .$$

The use of the Dodis–Yampolskiy construction, as described here, was introduced in [ASM06] but without a proof of security, which we provide in Theorem 8.7 with precise security bounds.

While the original reduction from Dodis–Yampolskiy to the inverse Diffie–Hellman problem requires the message space to be polynomially bounded, our setting allows the protocol metadata to come from an arbitrarily large space. This is because, when messages are hashed into the group, the size of the original message space no longer affects the reduction, thus avoiding this limitation.

Pseudonyms. Another popular use of credentials is for pseudonym credential systems. One such example is the “Per-verifier Linkability” internet draft [KB25] where the user can use the credential to create an “authenticated” pseudonym for a specific resource scp . Pseudonym systems can be used for blocking and logging in users in a specific service without making requests to third-party services or requiring another registration process. The user’s identity is otherwise unlinkable.

Similarly to rate-limiting credentials, we follow Camenisch [CHKLM06] and rely on a PRF that is **compatible** with the keyed-verification credential system. That is, the PRF key is in the message space of the credential, and the zero-knowledge proof system can express the relation that the PRF is evaluated correctly. In Section 8.2.2, we introduce a compiler that, given as input a keyed-verification credential system and a PRF compatible with it, produces a rate-limiting anonymous token. More precisely, a pseudonym system must be:

- *correct*, that is, all pseudonyms issued to honest users verify;
- *unlinkable*, that is, the server cannot link two different pseudonyms issued to the same user.
- *unforgeable*, that is, it is not possible to steal pseudonyms from honest users or to produce two pseudonyms for the same scope. This is formalized by providing the adversary with oracle that create new honest users, and present credentials for arbitrary scopes. At the end, the adversary outputs set of presentation messages for the same scope. The adversary wins if those messages were not previously presented and:
 - (i) are valid for a honest user, or
 - (ii) are more than then number of credentials issued to the adversary.

Roughly speaking, the approach here is to rely on a PRF that will generate context-based identities. Upon presenting the credential for accessing a *scope* (e.g. a website URL) identified by a string $scp \in \{0, 1\}^*$, the user presents $nym = \text{PRF}(k, scp)$, by proving that the valuation was done correctly for a k that was issued with a credential. By pseudorandomness of the PRF, the values nym are all random and independent, By one-more unforgeability of the credential system, the user cannot present more than one pseudonym for the same scope. Crucially, in this setting the key k

cannot be chosen solely by the user, to prevent collisions across nyms. To achieve this, we provide the issuer with the ability to re-randomize credentials upon issuance.

As an example, we provide an illustration of how presentation can be done for CMZ credentials highlighting the additional presentation material.

The intuition behind the security of this system is that, since the PRF is a function, one cannot produce more than one identity for the same scope; since the PRF is pseudorandom, the ephemeral identities look random to any adversary that does not possess the key k . Some simple PRF choices, such as Naor–Pinkas–Reingold $\text{NPR}(k, \text{scp}) = kH(\text{scp})$ are particularly efficient [NPR99] when using Σ -protocols.

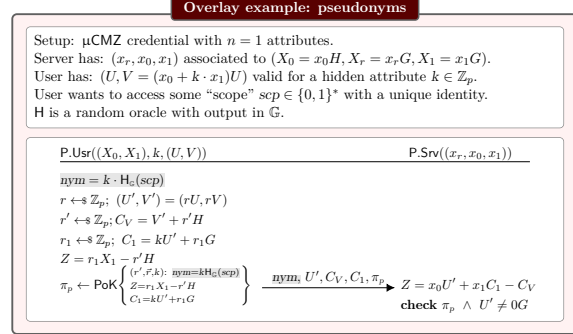


Figure 3: Example: pseudonym extension on μCMZ .

Remark 2.5 (Differences with rate-limiting).

Pseudonym systems are often seen as a special case of rate-limiting credential systems, where the “user counter” is set to 1. However, the two systems are not equivalent: in a pseudonym system, attackers may attempt to create two different credentials that yield the same pseudonym, effectively causing collisions across users. This issue does not arise in rate-limiting credentials, as any such behavior would, at worst, result in the adversary limiting their own credentials.

In practice, the security parameter for the pseudorandomness of the PRF must be large enough to make pseudonym collisions negligible.

2.6 Straight-line extraction for Sigma protocols

For all above proofs, Σ -protocols are the go-to choice given the simplicity of the relations. However, formally the security of the resulting system is tedious to argue. Σ -protocols have been historically studied using rewinding, whereas credentials often times need to rely on straight-line extraction techniques. Theoretically, this problem has been circumvented with the Fischlin transform [Fis05] but, due to the lack of concrete attacks over the strong Fiat–Shamir heuristic [FS86; BPW12] its adoption has been underwhelming.

We show straight-line extractability in the algebraic group model for linear relations:

$$\mathbf{R}_{\mathbf{F}} = \left\{ (\vec{x}, \vec{X}) \in \mathbb{Z}_p^n \times \mathbb{G}^m : \sum_k x_k F_{1,k} = X_1 \wedge \cdots \wedge \sum_k x_k F_{m,k} = X_m \right\},$$

where:

- each linear constraint $\vec{F}_j$ is hard: it is computationally hard to find a non-zero vector $\vec{w} \in \mathbb{Z}_p^n$ such that $\sum_k w_k F_{j,k} = 0G$;
- all non-trivial group elements appearing in the matrix are independent. In other words, for any p.p.t. adversary A , it is computationally hard to find a non-zero vector $\vec{w}$ such that $\sum_k w_k G_k = 0$, where $(G_k)_k$ are the distinct non-zero elements appearing in $\mathbf{F}$.

Informally, the above conditions are necessary to avoid the adversary “malleating” a simulated proof. For instance, given a proof $(\vec{R}, \vec{s})$ consisting of commitment and response of the Σ -protocol, with $\vec{w} \in \ker(\mathbf{F})$ it is possible to produce a new valid proof $(\vec{R}, \vec{s} + \vec{w})$.

Compatibility with modern SNARKs. For keyed-verification credentials, since the verifier is known in advance, designated-verifier zero-knowledge proofs are sufficient for issuance and presentation. We present a slightly modified version of Kate–Zaverucha–Goldberg’s commitment [KZG10] that is designated verifier and does not rely on pairing-friendly groups. Consider the KZG commitment scheme, where the commitment key is:

$$pk = (G, \tau G, \tau^2 G, \dots, \tau^{n-1} G) \quad (8)$$

and the trapdoor τ is set to be the verification key. A KZG commitment to a polynomial $f \in \mathbb{Z}_p[x]$ is a group element $C = f(\tau)G$ computed via add-and-multiply operations over the commitment key. An opening proof for $z \in \mathbb{Z}_p$, that is, a proof that $f(z) = y$ is

$$Q = q(\tau)G$$

where $q(x)$ is the quotient of the Euclidean division of $f(x) - y$ by $(x - z)$. The proof can be trivially checked in the group using the trapdoor, and soundness follows the same argument given in [KZG10]. Kohrita and Towa [KT24] provide an efficient way for hiding KZG commitments. The user now “blinds” f committing to it as $f(\tau)G + rH$ for $r \leftarrow \mathbb{Z}_p$, and proceeds similarly for the quotient polynomial. An evaluation proof now has an additional term to “balance out” the blinding factors of the verification equation.

In the designated-verifier case, the (malicious) verifier can build the trapdoor information itself, preserving extractability of the commitment. But zero-knowledge doesn’t hold! In fact, we have no guarantees that the proving key doesn’t come from a bad distribution that does not preserve hiding (a trivial example is when $H = 0G$). We show that it is possible to prove zero-knowledge (see Theorem 7.3) for any proving key that is “well-formed”, independently of its distribution.

One can prove that the commitment key is well-formed (and argue security against malicious verifiers) by equipping pk with a proof that the powers of τ have been correctly encoded. Using batching techniques, this boils down to 1 DLEQ proof (2 group elements). Upon receiving pk , a user would check that the proof is valid, and that no trivial (identity) elements are present in pk .

The most common ingredients for building non-interactive SNARKs today are interactive oracle proofs (IOPs) [BCS16] and commitment schemes. In an IOP, the prover may send field elements as messages, as well as special “oracle messages”, that the verifier can query. Roughly speaking, the “IOP oracles” are replaced by commitments to instantiate the protocol and build an argument. Among the most common combinations of IOP+commitment we have polynomial IOPs (PIOPs), where the prover may send polynomials as oracle messages and the verifier may ask for polynomial evaluations to those oracles, and polynomial commitment scheme (PCS). We show that, in the designated-verifier case, one can build a designated-verifier argument via a variant of the IOP compiler given in [CHMMVW20], that uses a polynomial IOP and a designated-verifier commitment scheme. See Theorem 7.5.

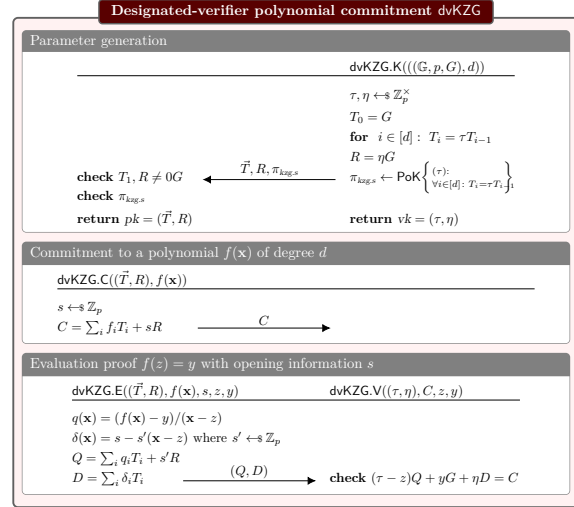


Figure 4: Overview of our designated-verifier polynomial commitment scheme dvKZG.

As an example application, we provide a constant-size range proof protocol that is both asymptotically and concretely more efficient than generic algebraic approaches for keyed-verification credentials. The LegoSNARK framework [CFQ19] is a helpful tool to bridge presentation proofs (which are generally done with Σ -protocols over generic linear relations) with the arguments resulting from our compiler, via the commit-then-prove paradigm.

3 Preliminaries

Notation. The security parameter is denoted λ . We consider the group generation procedure $\text{GrGen}(1^\lambda)$ which takes as input the security parameter in unary form 1^λ and outputs a group description $\Gamma = (\mathbb{G}, p, G)$ consisting of a group $\mathbb{G}$ of odd prime order p generated by $G \in \mathbb{G}$ where discrete logarithm (DL) is hard. Throughout this work we will use two random oracles: $\mathbf{H}_p$, with image over the field $\mathbb{Z}_p$, and $\mathbf{H}_{\mathbb{G}}$, with image over the group $\mathbb{G}$. The domain of both oracles is the set of strings of arbitrary length.

Let $[n]$ denote the range $\{1, \dots, n\}$, and $[n, m]$ denote the range $\{n, n+1, \dots, m\}$ if $n \leq m$, and $\emptyset$ otherwise. Assignment is denoted “ $:=$ ”, which we distinguish from assignment of the output of a randomized algorithm, where we use the symbol “ $\leftarrow$ ” to underscore the sampling of random coins before assignment. The range of a p.p.t. algorithm A (that is, the set of possible outputs that happen with non-zero probability) is denoted $[A(x)]$; the length of the random coins as $A.\text{rl}(\lambda)$. The notation $z \leftarrow (A(x) \stackrel{?}{=} B(y))$ denotes the interaction of the interactive Turing machine A on the input x with the (interactive) Turing machine B on input y . The output of A (the machine on the left-hand side) is assigned to z . When both machines return some value, respectively z_0 for A and z_1 for B , we separate them with the semicolon, i.e. $(z_0; z_1) \leftarrow (A(x) \stackrel{?}{=} B(y))$.

3.1 Cryptographic assumptions

The discrete logarithm (DL) assumption holds for GrGen if it is computationally hard, given $X \leftarrow \$ \mathbb{G}$ uniformly distributed, to compute $x \in \mathbb{Z}_p$ such that $xG = X$, where $(\mathbb{G}, p, G)$ is output by GrGen . The decisional Diffie–Hellman (DDH) assumption holds for GrGen if it is hard to distinguish the tuple (P, aP, bP, abP) from (P, aP, bP, cP) with $a, b, c \leftarrow \$ \mathbb{Z}_p$ and $P \in \mathbb{G}$. The advantage for an adversary A will be denoted respectively as $\text{Adv}_{\text{GrGen}, A}^{\text{dl}}(\lambda)$, $\text{Adv}_{\text{GrGen}, A}^{\text{ddh}}(\lambda)$. The gap discrete logarithm assumption is hard if DL is hard even in the presence of a DDH oracle for the DL challenge.

q -type Assumptions. The q -discrete logarithm (q -DL) assumption holds for the group generator GrGen if it is hard for any p.p.t. adversary to recover x uniformly distributed over $\mathbb{Z}_p$ given as input $(G, xG, x^2G, \dots, x^qG)$. Its advantage w.r.t. an adversary A is denoted $\text{Adv}_{\text{GrGen}, A}^{q\text{-dl}}(\lambda)$. The q -DDHI assumption holds for the group generator GrGen if it is hard for any p.p.t. adversary to distinguish $(xG, x^2G, \dots, x^qG, x^{-1}G)$ from $(xG, x^2G, \dots, x^qG, Z)$ for $x \leftarrow \$ \mathbb{Z}_p$ and $Z \leftarrow \$ \mathbb{G}$. Its advantage w.r.t. an adversary A is denoted $\text{Adv}_{\text{GrGen}, A}^{q\text{-ddhi}}(\lambda)$.

Algebraic Group Model. In the algebraic group model [FKL18], adversaries are assumed to know a representation of any group element they return. This means that, after having received group elements $Z_1, \dots, Z_n$, whenever the adversary returns a group element X , it must also return coefficients $\zeta_1, \dots, \zeta_n$ so that $X = \sum_i \zeta_i Z_i$. We call such adversaries *algebraic*.

Generic Group Model. In the generic group model [Sho97] the adversary is only given access to randomly chosen encodings of group elements, and an oracle that executes the group operation: the oracle takes two encodings and returns the encoding of the sum. If the oracle supports a pairing operation, an oracle for the pairing map is also given.

3.2 Algebraic message authentication codes

Definition 3.1. An *algebraic message authentication code (MAC)* for n attributes over message family $\mathbb{M} = \{\mathbb{M}_\lambda\}_\lambda$ is a tuple $\text{MAC} = (\text{S}, \text{K}, \text{M}, \text{V})$:

Game $\text{UF-CMVA}_{\text{MAC},A}(\lambda, n)$	Oracle $\text{SIGN}(\vec{m})$
$Q_{rs} := \emptyset; \text{ crs} \leftarrow \text{MAC.S}(\lambda, n)$	$Q_{rs} := Q_{rs} \cup \{\vec{m}\}$
$(sk, pp) \leftarrow \text{MAC.K}(\text{crs})$	return $\text{MAC.M}(sk, \vec{m})$
$(\vec{m}^*, \sigma^*) \leftarrow A^{\text{SIGN}, \text{VERIFY}}(pp)$	Oracle $\text{VERIFY}(\vec{m}, \sigma)$
return $\vec{m}^* \notin Q_{rs} \wedge \text{MAC.V}(pp, \vec{m}^*, \sigma^*) = 1$	return $\text{MAC.V}(sk, \vec{m}, \sigma)$

Figure 5: Unforgeability of an algebraic message authentication code MAC.

- $\text{crs} \leftarrow \text{MAC.S}(1^\lambda, n)$ The setup algorithm, which takes as input the security parameter in unary form and the number of attributes, and outputs a common-reference string crs .

The setup algorithm implicitly defines an attribute space $\mathbb{M}_{\text{crs}} = \mathbb{M}$ (we drop the “index” crs to ease notation) describing the family of messages supported by the MAC scheme.

- $(sk, pp) \leftarrow \text{MAC.K}(\text{crs})$ The key generation algorithm, which takes as input the crs and outputs a secret key sk and some public parameters pp .

In the remainder of this paper, we will omit crs from inputs of the algorithms below for brevity, and assume that it can be implicitly derived from either sk or pp .

- $\sigma \leftarrow \text{MAC.M}(sk, \vec{m})$ The MAC algorithm, which takes as input the signing key sk and attributes $\vec{m} \in \mathbb{M}^n$ and outputs a message authentication code σ .
- $0/1 := \text{MAC.V}(sk, \vec{m}, \sigma)$ The (deterministic) verification algorithm, which takes as input the signing key sk , a message $\vec{m}$, and a MAC σ , and outputs 1 if the MAC is valid, and 0 otherwise.

MAC must satisfy correctness (every honestly generated MAC verifies) and unforgeability (it is hard to forge MACs). We denote the unforgeability advantage of A with respect to the algebraic message authentication code MAC on n attributes as $\text{Adv}_{\text{MAC},A}^{\text{ufcmva}}(\lambda, n) := |\Pr[\text{UF-CMVA}_{\text{MAC},A}(\lambda, n)]|$, where $\text{UF-CMVA}_{\text{MAC},A}(\lambda, n)$ is illustrated in Figure 5.

Remark 3.2. Differently from MACs, algebraic MACs are randomized algorithms. It is possible to construct a “de-randomized” algebraic MAC in the random oracle model using the message to seed the randomness, i.e. letting $H(\vec{m})$ be the random coins for the MAC algorithm.

3.3 Zero-knowledge arguments

A non-interactive proof system ZKP for a relation family $\mathcal{R} = \{R_\lambda\}_\lambda$ consists of the following three algorithms:

- $\text{crs} \leftarrow \text{ZKP.S}(1^\lambda)$, the setup algorithm, which takes as input the security parameter in unary form and outputs a common reference string crs . It implicitly selects a relation $R \in \mathcal{R}$.
- $\pi \leftarrow \text{ZKP.P}(\text{crs}, x, w)$, a prover which takes as input $(x, w) \in R$ and outputs a proof π .
- $0/1 \leftarrow \text{ZKP.V}(\text{crs}, x, \pi)$, a verifier that, given as input an instance x together with a proof π outputs 0 if the proof is rejected and 1 otherwise.

Game $\text{OMUF}_{\text{AT}, \mathbf{A}}(\lambda, n)$	Oracle $\text{SIGN}(\mu)$
$q := 0$	$q = q + 1$
$\text{crs} \leftarrow \text{AT.S}(1^\lambda, n)$	return $\text{AT.I.Srv}(sk, \mu)$
$(sk, pp) \leftarrow \text{AT.K}(\text{crs})$	
$(\vec{m}_i, \sigma_i)_{i=1}^{q+1} \leftarrow \mathbf{A}^{\text{SIGN}, \text{VERIFY}}(pp)$	Oracle $\text{VERIFY}(\vec{m}, \sigma)$
return $\forall i \neq j: \vec{m}_i \neq \vec{m}_j \wedge \forall i \in [q+1]: \text{AT.V}(sk, \vec{m}_i, \sigma_i) = 1$	return $\text{AT.V}(sk, \vec{m}, \sigma)$

Figure 6: One-more unforgeability game for an anonymous token scheme AT with non-interactive issuance. The variable “ μ ” denotes the issuance request message from the user.

For clarity, we will talk about a relation $\mathbf{R}$ assuming it is indexed by the range of crs , and omit the common reference string from the prover and verifier algorithms. A proof system is complete if every correctly-generated proof for an element of $\mathbf{R}$ verifies.

A proof system is a **knowledge-sound argument** [BLCL91] if there exists an extractor Ext that takes as input the random coins and the code of the p.p.t. adversary $\mathbf{A}$ (optionally, a trapdoor for the crs , but not required in our instantiations) such that, whenever $\mathbf{A}(\text{crs})$ outputs (x, π) , then Ext outputs w . The adversary wins if $\text{ZKP.V}(\text{crs}, x, \pi) = 1 \wedge (x, w) \notin \mathbf{R}$. The advantage is denoted as $\text{Adv}_{\text{ZKP}, \text{Ext}, \mathbf{A}}^{\text{ksnd}}(\lambda)$. In the proof of MAC unforgeability, we will require a stronger notion of knowledge-soundness, where the adversary is allowed to output simulated proofs, and the extractor has to provide (in addition to the witness), a candidate statement. See Sections 5.5 and 6.5.

A proof system is **zero-knowledge** [GMR89] if there exists a simulator Sim such that, for any adversary $\mathbf{A}$,

$$\text{Adv}_{\text{ZKP}, \mathbf{A}}^{\text{zk}}(\lambda) := \left| \Pr \left[b' = 1 : \begin{array}{l} \text{crs} \leftarrow \text{ZKP.S}(1^\lambda) \\ b' \leftarrow \mathbf{A}^{\text{PROVE}_0}(\text{crs}) \end{array} \right] - \Pr \left[b' = 1 : \begin{array}{l} \text{crs} \leftarrow \text{ZKP.S}(1^\lambda) \\ b' \leftarrow \mathbf{A}^{\text{PROVE}_1}(\text{crs}) \end{array} \right] \right| \leq \text{negl}(\lambda) .$$

where $\text{PROVE}_b(x, w)$ checks if $(x, w) \in \mathbf{R}$ and outputs $\text{ZKP.P}(\text{crs}, x, w)$ if $b = 0$ and $\text{Sim}(\text{crs}, x)$ if $b = 1$. We assume that both adversary and simulator have access to a random oracle, and that the simulator can explicitly re-program the random oracle. A proof system is (strongly) **simulation-extractable** if it is knowledge-sound even when the adversary has access to simulated proofs, and the output pair (x, π) was not previously returned by the zero-knowledge simulator. For the formal definitions, see Dao and Grubbs [DG23, Fig. 2, 3, and 4].

3.4 Anonymous Tokens

An anonymous token [KLOR20] is a “blind MAC” (with an optional private metadata bit). More formally, an anonymous token scheme AT for $n > 0$ attributes over attribute family $\mathbb{M}$ is a tuple $\text{AT} = (\text{S}, \text{K}, \text{I}, \text{V})$ where $\text{AT.S}(1^\lambda, n)$ outputs a crs ; $\text{AT.K}(\text{crs})$ outputs a signing key sk and some public parameters pp ; the issuance protocol involves a user $\text{AT.I.Usr}(pp, \vec{m})$ and a server $\text{AT.I.Srv}(sk)$ and produces a token σ for the user; the verification algorithm $\text{AT.V}(sk, \vec{m}, \sigma)$ returns 0/1 if the token is accepted for the message $\vec{m}$. We demand anonymous tokens to be correct, one-more unforgeable, and unlinkable.

Correctness means that all tokens generated via $\text{AT.I.}\{\text{Usr}, \text{Srv}\}$ for messages in the family successfully verify. One-more unforgeability asks that, after blindly issuing at most q tokens, no adversary can successfully present $q + 1$ valid message/token pairs. More formally, for a two-move

Game $\text{UNF}_{\text{KVAC}, \mathbf{A}}(\lambda, n)$	Oracle $\text{SIGN}(\vec{m})$
$Qrs := \emptyset$	$Qrs := Qrs \cup \{\vec{m}\}$
$crs \leftarrow \text{KVAC.S}(1^\lambda, n)$	return $\text{KVAC.M}(sk, \vec{m})$
$(sk, pp) \leftarrow \text{KVAC.K}(crs)$	
$(\phi^*, \rho^*) \leftarrow \mathbf{A}^{\text{SIGN}, \text{PRESENT}}(pp)$	Oracle $\text{PRESENT}(\phi, \rho)$
$b \leftarrow \text{KVAC.P.Srv}(sk, \phi^*, \rho^*)$	return $\text{KVAC.P.Srv}(sk, \phi, \rho)$
return $(b = 1) \wedge (\forall \vec{m} \in Qrs: \phi^*(\vec{m}) = 0)$	

Figure 7: Canonical unforgeability game for a keyed-verification credential system KVAC [CMZ14]. The variables “ ϕ, ϕ^* ” denotes the credential predicate to be shown on $\vec{m}$ (cf. Theorem 4.1) while “ ρ ” denotes the presentation message.

protocol, given an anonymous token scheme AT for $n > 0$ attributes over attribute family $\mathbb{M}$, an adversary $\mathbf{A}$ has one-more unforgeability advantage

$$\text{Adv}_{\text{AT}, \mathbf{A}}^{\text{omuf}}(\lambda) := \Pr[\text{OMUF}_{\text{AT}, \mathbf{A}}(\lambda, n) = 1]$$

where $\text{OMUF}_{\text{AT}, \mathbf{A}}(\lambda, n)$ is illustrated in Figure 6.

Unlinkability (similarly to blindness for blind signatures) demands that any malicious issuer, after blindly issuing q tokens to honest users and then observing a permutation of the issued tokens, cannot do better than guessing the link between a token and its issuance. Looking ahead, keyed-verification token systems (Theorem 4.2) are also anonymous tokens: the syntax is the same (except for predicates) but the security guarantees are stronger: instead of asking for one-more unforgeability, we demand extractability (Theorem 4.5). We will exploit the weaker security requirements of unforgeability to design more efficient schemes, but will maintain the stronger anonymity properties and therefore omit the definition of unlinkability in this work, and direct the curious reader to Kreuter et al. [KLOR20, Fig. 4] and Chase, Durak, Vaudenay [DVC22, Fig. 3].

4 Keyed-verification credential systems

We first generalize the issuance algorithm of credential systems (procedures `BlindIssue` and `BlindObtain` in Chase et al. [CMZ14] and here $\text{KVAC.I.}\{\text{Usr}, \text{Srv}\}$) so that arbitrary predicates about the attributes over which a credential is about to be issued can be proven. Then, we re-define security and give a unique notion unforgeability and anonymity.

4.1 Syntax

We enrich the classical definition of keyed-verification anonymous credentials (KVAC) with issuance with predicates beyond partial disclosure of attributes. In this setting, at issuance time the user can partially disclose some attributes satisfying a predicate $\phi \in \Phi$ over them. For the sake of simplicity, we use the same predicate family employed in the presentation protocol, and make it explicit when they are not. This is a generalization of the traditional definition, where the predicate ϕ is concerned with disclosure of only some attributes, and where the issuer has no memory of previously seen credentials.

Definition 4.1. A **credential predicate** is an efficiently-computable function ϕ mapping some elements $\vec{m} \in \mathbb{M}^n$ ($\mathbb{M}$ referred to as attribute space) to a boolean value (1 if the predicate is satisfied, 0 otherwise). With ϕ_1 we denote the trivial predicate that always returns 1. A **predicate family** is a non-empty set of credential predicates $\Phi = \{\phi: \mathbb{M}^n \rightarrow \{0, 1\}\}$ containing the trivial predicate and closed under conjunction of statements (i.e., if $\phi, \phi' \in \Phi$ then $\phi \wedge \phi' \in \Phi$).

Definition 4.2. A **keyed-verification credential system** $\text{KVAC} = (\text{S}, \text{K}, \text{I}, \text{P})$ for predicate family Φ over a message family $\mathbb{M} = \{\mathbb{M}_\lambda\}_\lambda$ for $n \in \mathbb{N}$ attributes is a tuple of algorithms:

- $\text{crs} \leftarrow \text{KVAC.S}(1^\lambda, n)$ The setup algorithm, which takes as input the security parameter in unary form and the max number of attributes $n > 0$, and outputs a common-reference string crs .

The setup algorithm implicitly defines an attribute space $\mathbb{M}$ and a predicate family Φ .

- $(\text{sk}, \text{pp}) \leftarrow \text{KVAC.K}(\text{crs})$ The key generation algorithm, which given as input the crs , produces a signing key sk and some public issuer parameters pp .

In the remainder of this work, we will omit crs from inputs to the algorithms below for brevity, and assume that it can be implicitly derived from either sk or pp . We will also assume that it is possible to efficiently test whether some given keypair (sk, pp) is correctly generated, i.e., $(\text{sk}, \text{pp}) \in [\text{KVAC.K}(\text{crs})]$. This is without loss of generality: it is always possible to consider the secret key sk as the random coins used in the key generation algorithm as a syntactical change.

- $\sigma \leftarrow (\text{KVAC.I.Usr}(\text{pp}, \vec{m}, \phi) \Rightarrow \text{KVAC.I.Srv}(\text{sk}, \phi))$ The issuance algorithm allows a user to obtain a credential σ for a set of attributes $\vec{m}$, kept hidden from the issuer, but satisfying a predicate $\phi \in \Phi$.

All future issuance algorithm will be one-round, i.e. the issuer will receive a message from the user and respond with a (blinded) credential, that will be further processed by the user. Therefore, to simplify the description, we will split the protocol into 3 non-interactive algorithms:

- $(st_u, \mu) \leftarrow \text{KVAC.I.Usr}_1(pp, \vec{m}, \phi)$, producing the first protocol message μ and the user state st_u .
- $\sigma' \leftarrow \text{KVAC.I.Srv}(sk, \phi, \mu)$, producing the issuer's blinded credential σ' .
If the issuer does not accept the user's message, the issuer will return $\sigma' = \perp$.
- $\sigma \leftarrow \text{I.Usr}_2(st_u, \sigma')$, producing the credential σ given as input the state st_u and the issuer message σ' .

The canonical definition of keyed-verification anonymous credentials admits selective disclosure of attributes, which is a special case of our definition. That is, for a vector $\vec{a} \in (\mathbb{M} \cup \{\star\})^n$ (where $\star$ denotes the positions to be hidden), the predicate of partial disclosure is $\phi_{\vec{a}}(\vec{m}) := (\forall j \in [n] : a_j = m_j \vee a_j = \star)$.

- $0/1 \leftarrow (\text{KVAC.P.Srv}(sk, \phi) \Rightarrow \text{KVAC.P.Usr}(pp, \vec{m}, \sigma, \phi))$ The present (or show) algorithm allows a user to prove possession of a credential σ over some attributes satisfying a predicate $\phi \in \Phi$, without revealing any information other than what can be explicitly inferred from ϕ . Optimizations related to the above special case of partial disclosure are generally straightforward to obtain and, for the schemes we present, presentation proof will depend solely on the number of hidden attributes.

All presentation algorithms in this work are one-round, i.e. the user will send a single message to the issuer, and the issuer will output a single bit. Consequently, we will split the protocol into two non-interactive algorithms:

- $\rho \leftarrow \text{KVAC.P.Usr}(pp, \vec{m}, \sigma, \phi)$, producing the presentation message ρ .
- $0/1 \leftarrow \text{KVAC.P.Srv}(sk, \phi, \rho)$, verifying the presentation message ρ for ϕ with sk .

A keyed-verification credential system KVAC satisfies correctness, anonymity, and unforgeability.

Similarly to previous works [CMZ14; CR19], we define two extra algorithms to simplify formalism:

- $\sigma \leftarrow \text{KVAC.M}(sk, \vec{m})$ generates a credential σ for the attributes $(m_1, \dots, m_n)$. This is a shorthand for running $(\mu, st) \leftarrow \text{KVAC.I.Usr}_1(pp, \vec{m}, \phi_{\vec{m}})$, $\sigma' \leftarrow \text{KVAC.I.Srv}(sk, \phi_{\vec{m}}, \mu)$, and returning $\sigma \leftarrow \text{KVAC.I.Usr}_2(st, \sigma')$.
- $0/1 \leftarrow \text{KVAC.V}(sk, \vec{m}, \sigma)$ verifies that the credential σ for the attributes $\vec{m}$ using sk . This is a shorthand for running $\rho \leftarrow \text{KVAC.P.Usr}(pp, \vec{m}, \sigma, \phi_{\vec{m}})$ and returning $\text{KVAC.P.Srv}(sk, \phi_{\vec{m}}, \rho)$.

The notation clash with the MAC and verification algorithms of a MAC scheme is on purpose, as we will focus on credentials derived from algebraic MACs.

4.2 Correctness

Correctness informally states that messages satisfying the issuance predicate should lead to correct credential issuance, and credential presentation should succeed for any valid statement.

Definition 4.3 (Correctness). A keyed-verification credential system KVAC for a family of $n \leq \text{poly}(\lambda)$ attributes and nontrivial predicates $\phi \supseteq \{\phi_{\vec{a}} : \vec{a} \in (\mathbb{M} \cup \star)^n\}$ and a non-empty message family $\mathbb{M}$ is **correct** if for any p.p.t. adversary $\mathbf{A}$, for any $\text{crs} \in [\text{KVAC.S}(1^\lambda, n)]$ and $(sk, pp) \in [\text{KVAC.K}(\text{crs})]$ and for any $\phi, \phi' \in \Phi$, $\vec{m} \in \mathbb{M}^n$ such that $\phi(\vec{m}) = \phi'(\vec{m}) = 1$:

$$\Pr \left[b = 1 : \begin{array}{l} \sigma \leftarrow (\text{KVAC.I.Usr}(pp, \vec{m}, \phi) \Rightarrow \text{KVAC.I.Srv}(sk, \phi)) \\ b \leftarrow (\text{KVAC.P.Srv}(sk, \phi') \Rightarrow \text{KVAC.P.Usr}(pp, \vec{m}, \sigma, \phi')) \end{array} \right]$$

is overwhelming in λ .

4.3 Anonymity

Anonymity is captured by an indistinguishability game where the adversary, given as input crs , is asked to distinguish between the interaction with an honest user and a simulator that does not know any of the attributes (for issuance and presentation). Users for which the issuance/presentation predicates hold are then indistinguishable. More formally, the simulator is a pair of procedures $\text{Sim} = (\text{Sim.I}, \text{Sim.P})$ where $\text{Sim.I}(pp, \phi)$ is an interactive procedure whose distribution is meant to be indistinguishable from KVAC.I.Usr , returning at the end some simulator state st_{Sim} , and similarly $\text{Sim.P}(st_{\text{Sim}}, \phi)$ returns a “simulated” presentation that is meant to be indistinguishable from KVAC.I. . The adversary has also access to a presentation oracle, either consisting of the user procedure or of Sim.P .

Definition 4.4. A keyed-verification credential system KVAC for a family of $n \leq \text{poly}(\lambda)$ attributes, predicate family Φ , and a non-empty message family $\mathbb{M}$ is **anonymous** if there exists a simulator $\text{Sim} = (\text{Sim.I}, \text{Sim.P})$ such that, for all $crs \in [\text{KVAC.S}(1^\lambda, n)]$ and $(sk, pp) \in [\text{KVAC.K}(crs)]$, $\vec{m} \in \mathbb{M}_{crs}^n$, $\phi \in \Phi$ such that $\phi(\vec{m}) = 1$, adversaries A, D , the advantage $\text{Adv}_{\text{KVAC}, A, D}^{\text{anon}}(\lambda, n)$ defined as:

$$\left| \Pr \left[b' = 1 : \begin{array}{l} (\sigma; st_A) \leftarrow (\text{KVAC.I.Usr}(pp, \vec{m}, \phi) \Rightarrow A(sk, pp, \phi, \vec{m})) \\ b' \leftarrow D^{\text{PRESENT}_0}(st_A) \end{array} \right] - \Pr \left[b' = 1 : \begin{array}{l} (st_{\text{Sim}}; st_A) \leftarrow (\text{Sim.I}(pp, \phi) \Rightarrow A(sk, pp, \phi, \vec{m})) \\ b' \leftarrow D^{\text{PRESENT}_1}(st_A) \end{array} \right] \right|$$

is negligible in λ , where the $\text{PRESENT}_b(\phi')$ oracle checks if $\phi'(\vec{m})$ holds for $\vec{m}$, and if so returns $\text{KVAC.P.Usr}(pp, \vec{m}, \sigma, \phi)$ outputs (if $b = 0$) or $\text{Sim.P}(st_{\text{Sim}}, \phi)$ (if $b = 1$)

KVAC has **statistical anonymity** if $\text{Adv}_{\text{KVAC}, A, D}^{\text{anon}}(\lambda, n)$ is negligible for unbounded adversaries A, D . A KVAC has **everlasting forward anonymity** if $\text{Adv}_{\text{KVAC}, A, D}^{\text{anon}}(\lambda)$ is negligible when D is unbounded.

Comparison with original KVACs [CMZ14]. The canonical definition of anonymity for keyed-verification credentials is concerned solely with the anonymity of different presentation sessions. It seems quite weak since it doesn’t consider the server that issues malicious credentials leading to non-anonymous showings. This is in (only in part) part deferred to the “blind issuance” property, which has never been formally proven for the schemes we examine. Formally, it asks that for any p.p.t. adversary A , for any $crs \in [\text{KVAC.S}(1^\lambda, n)]$ and $(sk, pp) \in [\text{KVAC.K}(crs)]$ and for any $\phi \in \Phi$, $\vec{m} \in \mathbb{M}_{crs}^n$ such that $\phi(\vec{m}) = 1$,

$$\left| \Pr \left[A(sk, pp, \phi, \rho) = 1 : \begin{array}{l} \sigma \leftarrow \text{KVAC.M}(sk, \vec{m}) \\ \rho \leftarrow \text{KVAC.P.Usr}(pp, \vec{m}, \sigma, \phi) \end{array} \right] - \Pr \left[A(sk, pp, \phi, \rho) = 1 : \rho \leftarrow \text{Sim.P}(sk, \phi) \right] \right|$$

is negligible in λ . In some credentials such as Chase–Meiklejohn–Zaverucha, where anonymity is only computational, this difference is perceptible. (In [CMZ14], other notions such as key-parameter consistency and blind issuance are concerned with user anonymity.)

Game $\text{EXT}_{\text{KVAC}, \text{Ext}, \text{A}}(\lambda, n)$ $Qrs := \emptyset; \boxed{PQrs := []; \text{Usrs} := []; \text{ctr} := 0}$ $\text{crs} \leftarrow \text{KVAC.S}(1^\lambda, n)$ $(sk, pp) \leftarrow \text{KVAC.K}(\text{crs})$ $(\phi^*, \rho^*) \leftarrow \text{A}^{\text{ISSUE, PRESENT}, \boxed{\text{NEWUSR, PRESENTUSR}}}(pp)$ $\vec{m}^* := \text{Ext.P}(sk, \phi^*, \rho^*)$ return $\text{KVAC.P.Srv}(sk, \phi^*, \rho^*) = 1 \wedge \boxed{(\phi^*, \rho^*) \notin PQrs \wedge (\vec{m}^* \notin Qrs \vee \phi^*(\vec{m}^*) = 0)}$	Oracle $\text{NEWUSR}(\vec{m})$ $\sigma \leftarrow \text{KVAC.M}(sk, \vec{m})$ $\text{Usrs}[\text{ctr}] := (\vec{m}, \sigma)$ return $\text{ctr} := \text{ctr} + 1$ Oracle $\text{PRESENTUSR}(i, \phi)$ $(\vec{m}, \sigma) := \text{Usrs}[i]$ $\rho \leftarrow \text{KVAC.P.Usr}(pp, \vec{m}, \sigma, \phi)$ $PQrs := PQrs \cup \{(\phi, \rho)\}$ return ρ	Oracle $\text{ISSUE}(\phi, \mu)$ $\sigma' \leftarrow \text{KVAC.I.Srv}(sk, \phi, \mu)$ if $\sigma' = \perp$: return $\perp$ $\vec{m} := \text{Ext.I}(sk, \phi, \mu)$ if $\phi(\vec{m}) = 0$: abort $Qrs := Qrs \cup \{\vec{m}\}$ return σ' Oracle $\text{PRESENT}(\phi, \rho)$ return $\text{KVAC.P.Srv}(sk, \phi, \rho)$
---	---	---

Figure 8: Extraction game for a keyed-verification credential system KVAC with extractors Ext.I and Ext.P, and adversary A. The variables “ ϕ ” and “ ϕ^* ” denote the predicate to be shown on $\vec{m}$ (cf. Theorem 4.1) whereas “ μ ” denotes the issuance message, and “ ρ ” the presentation message. $\boxed{}$, the experiment in the multi-user setting with man-in-the-middle attacks.

4.4 Extractability

In keyed-verification credential systems, unforgeability states that an adversary should not be able to present a credential for a predicate ϕ that does not hold over any of the previously issued credentials. The notion we propose is stronger: we require an extractor that can recover the attributes from issuance/presentation messages, and consider man-in-the-middle adversaries. The adversary wins if at the end of the game it presents a valid credential from which the extracted message was not previously issued to them. The adversary is allowed to observe and copy presentation messages from honest users on arbitrary attributes.

Definition 4.5. A keyed-verification credential system KVAC for a family of $n \leq \text{poly}(\lambda)$ attributes, predicate family Φ , and a non-empty message family $\mathbb{M}$ is **extractable** if there exists $\text{Ext} = (\text{Ext.I}, \text{Ext.P})$ such that, for any p.p.t. adversary A:

$$\text{Adv}_{\text{KVAC}, \text{Ext}, \text{A}}^{\text{ext}}(\lambda) := \Pr[\text{EXT}_{\text{KVAC}, \text{Ext}, \text{A}}(\lambda, n) = 1]$$

is negligible in λ , where $\text{EXT}_{\text{KVAC}, \text{Ext}, \text{A}}(\lambda, n)$ is defined in Figure 8.

In the game, the adversary receives as input some public parameters (from which it is possible also to infer the common reference string crs) and can interact with issuance and presentation oracles. The oracles NEWUSR and PRESENTUSR deal with issuance and presentation of credentials of *other* users, and are meant to model man-in-the-middle attackers that might re-use presentation messages for other presentation predicates. The oracles ISSUE and PRESENT instead deal with issuance and presentation of a credential: the ISSUE oracle returns a server message σ' representing the blinded credential that the adversary may unblind. The PRESENT oracle returns the result of the presentation protocol. The adversary is then asked to output a presentation message ρ^* and a predicate ϕ^* that was not queried to the oracles, and that satisfies the presentation algorithm.

Comparison with original KVACs [CMZ14]. The original definition of unforgeability for keyed-verification anonymous credentials (illustrated in Figure 7) is implied by extractability.

Any q -query bound adversary A for the game $\text{UNF}_{\text{KVAC}, \text{A}}(\lambda, n)$ can be used to construct an adversary B for the game $\text{EXT}_{\text{KVAC}, \text{Ext}, \text{B}}(\lambda, n)$. B will receive as input pp and internally run $\text{A}(pp)$. For every query of the form $\text{SIGN}(\vec{m})$ made by A, then B will run $\mu \leftarrow \text{KVAC.I.Usr}_1(pp, \vec{m}, \phi_{\vec{m}})$

and query the issuance oracle. Let $\vec{m}^*$ be the extracted message. If $\vec{m}^* \notin Qrs$ then B wins. Otherwise, $\vec{m}^* = \vec{m}_j$ for some $j \in [q]$ and $\phi^*(\vec{m}) = \phi(\vec{m}_j) = 0$ and thus B wins. Therefore $\text{Adv}_{\text{KVAC}}^{\text{unf}}(\lambda) \leq \text{Adv}_{\text{KVAC}}^{\text{ext}}(\lambda)$.

Comparison with PABSs [CKLMNP16]. Extraction is similar to the unforgeability experiment for *privacy-enhancing attribute-based signatures* (PABSs), with some syntactical differences, mostly due to the modularity of PABSs, where each attribute is committed with an *opening extractable* commitment (a commitment equipped with a zero-knowledge proof of knowledge for the opening relation). Their notion of unforgeability schemes demands that there exist extractors (E^c, E^s) recovering the attributes from respectively issuance and presentation. For any adversary that outputs a valid credential presentation message (composed of a signature presentation token spt satisfying SignTokenVf for public attributes $\vec{a}$, committed attributes $\vec{c}$, and a message M), one of the following is true: (i) the extractor E^s failed to recover a valid opening for one of the commitments returned by the adversary (ComOpenVf); (ii) the extractor E^s failed to recover a valid signature from the adversary’s output (SigVf); (iii) the message and credentials (as extracted from E^c) were not previously queried. Throughout the game, the issuance oracle checks that the commitment verification proof is satisfied, which by opening extraction, implies that the issuance message is correctly extracted similarly to the abort condition in ISSUE . Items (i) and (ii) ask that a valid message is extracted every time that the credential verification equation is successful (similarly to our request on Ext.P); item (iii) is similar to our request of the forgery not being present in Qrs , the set of credentials extracted from Ext.I .

Strategy employed in our analysis. When proving our keyed-verification credential systems, we do not consider the multi-user setting. All credentials that we study (the ones including the boxed lines in the figures) rely on a proof system ZKP that is zero-knowledge and knowledge-sound, and it is sufficient to replace knowledge-soundness with strong simulation extractability to prove the definition of extractability as presented in Figure 8. In fact, the output of PRESENTUSR is always a prover message ρ composed of commitments and proofs (the commitments, included in ρ , are part of the statement of the proofs). Therefore, the additional winning condition guarantees extraction in the strong simulation extractability game.

5 Chase–Meiklejohn–Zaverucha credentials

Figure 9 formally illustrates our credential system μCMZ . It can be seen as a variant of MAC_{GGM} , secure in the generic group model, as introduced by Chase, Meiklejohn, and Zaverucha [CMZ14].

5.1 Protocol description

Message authentication code. The key generation procedure samples random $(x_0, x_r, \vec{x}) \leftarrow \mathbb{Z}_p^{n+1}$, and sets the public parameters $(X_0 = x_0H, X_r = x_rG, X_1 = x_1G, \dots, X_n = x_nG)$. The signing algorithm computes $V = (x_0 + x_r + \sum_i m_i x_i)U$. The special property of this MAC is that they are randomizable, i.e. given a MAC (U, V) for $\vec{m}$, then $(U', V') := (rU, rV)$ for $r \neq 0$ still satisfies the verification equation $V' = (x_0 + x_r + \sum_i m_i x_i)U$. The verification algorithm $\mu\text{CMZ.V}((x_0, x_r, \vec{x}), \vec{m}, (U, V))$ checks $U \neq 0G$ and $V = (x_0 + x_r + \sum_i x_i m_i)U$.

Blind issuance. The user commits to the attributes as $C = \sum_i m_i X_i + sH$, where s is sampled uniformly at random in $\mathbb{Z}_p$. Then, it proves that $\vec{m}$ satisfies the issuance predicate ϕ , i.e., that $((C, \vec{X}, \phi), (\vec{m}, s))$ is in the relation:

$$R_{\text{cmz.iu}} := \left\{ ((C', \vec{X}, \phi), (\vec{m}, s)) : C' = \sum_i m_i X_i + sG \wedge \phi(\vec{m}) = 1 \right\} \quad (9)$$

and sends a proof of knowledge π_{iu} for $(\vec{m}, s)$. Public messages don't need to be committed (nor proven in zk) and the server can add them itself using the homomorphic properties of C' . The issuer computes a MAC for the message $\vec{m}$ and proves its correctness via a proof π_{is} for the relation:

$$R_{\text{cmz.is}} := \left\{ ((X_0, C'', U', V'), (x_0, u)) : \begin{array}{l} U' = uG \wedge X_0 = x_0H \wedge \\ V' = x_0U' + uC'' \end{array} \right\} \quad (10)$$

Credential presentation. To present a credential, the user first re-randomizes the MAC $\sigma = (U, V)$ into $\sigma' = (U' = rU, V' = rV)$ with $r \leftarrow \mathbb{Z}_p^\times$, produces commitments C_i to each message m_i , and finally prove that the committed messages satisfy a predicate ϕ .⁵ The user will prove the relation:

$$R_{\text{cmz.p}} := \left\{ ((U', X_1, \dots, X_n, C_1, \dots, C_n, Z, \phi), (r', \vec{r}, \vec{m})) : \begin{array}{l} (\forall i \in [n] \ C_i = m_i U' + r_i G) \wedge \\ Z = \sum_{i=1}^n r_i X_i - r' H \wedge \phi(\vec{m}) = 1 \end{array} \right\}. \quad (11)$$

Efficient partial disclosure. The case of partial disclosure of attributes, where only a subset of attributes $J \subseteq [n]$ is meant to be hidden, can be optimized in a straightforward way: the user will commit only to the attributes in J and the server will compute $Z = (x_0 + x_1)U' + \sum_{i \in J} x_i C_i + \sum_{i \in [n] \setminus J} m_i U' - C_V$, using the hidden attributes as input to the proof. The case of full disclosure of messages is also straightforward: it consists in sending $(U', V' + r'G)$ and proving the relation fixing $r_i = 0$ for all $i \in [n]$.

Moreover, if the predicate is $\phi_{\vec{a}}$ for some $\vec{a} \in (\mathbb{M} \cup \{\star\})^n$, then the presentation proof message can be simplified via standard batching techniques. The user will still send to the server commitments for all hidden attributes, but instead of proving knowledge of an opening for each C_i , proceeds as follows. Compute $\hat{C} = \sum_{i \in J} \mu^i C_i$ where $\mu := H(\vec{C})$ and prove knowledge instead of $\hat{m} = \sum_{i \in J} \mu^i m_i$ and $\hat{r} = \sum_{i \in J} \mu^i r_i$.

⁵Despite not explicit in [CMZ14], the check $U' \neq 0G$ is required as otherwise a user having a credential for $\vec{m} = 0$ will verify for $\vec{m} = \vec{0}, r' = 0, r_i = 0$.

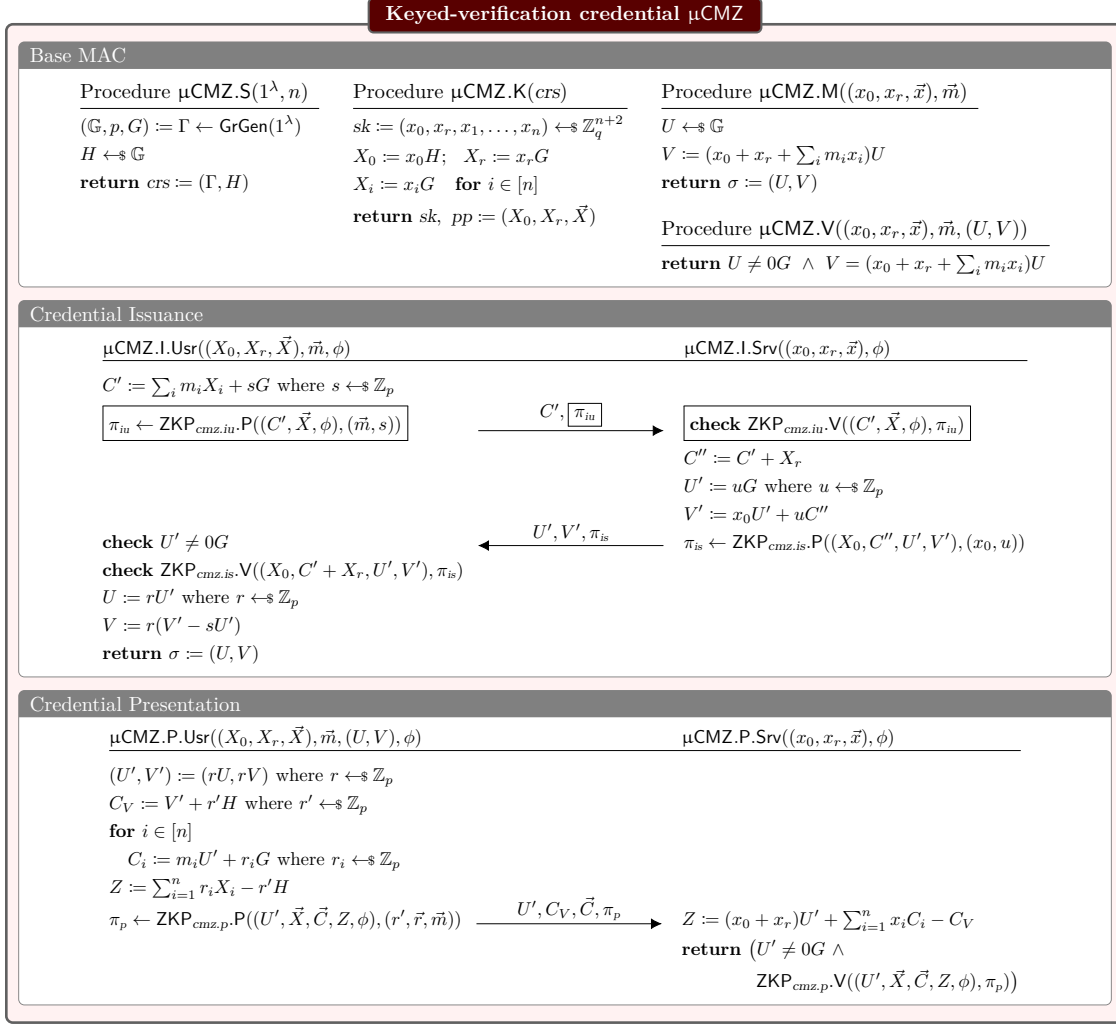


Figure 9: The keyed-verification credential system μCMZ . Boxed, the part that may be removed for anonymous tokens (one-more unforgeability). The variable ϕ denotes the arbitrary predicates that must be enforced on the attributes during issuance or presentation. The relations to be proven are defined in Equations (9) to (11).

5.2 Theorems

We formally state (below) the core theorems of μCMZ , starting from the underlying MAC, then prove them.

Algebraic MAC. The core theorem resulting from Theorem 5.4 and Theorem 5.5 is the following.

Theorem 5.1. *In the algebraic group model, μCMZ is an algebraic MAC for $n = \text{poly}(\lambda)$ attributes over $\mathbb{Z}_p$ with advantage:*

$$\text{Adv}_{\mu\text{CMZ}}^{\text{ufcmva}}(\lambda, n) \leq \text{Adv}_{\text{GrGen}}^{3\text{-dl}}(\lambda) + \text{Adv}_{\text{GrGen}}^{\text{dl}}(\lambda) + \frac{3}{p} ,$$

The proof is done in two parts: we first study the case $n = 1$ (Theorem 5.4) and then reduce the case $n > 1$ to the case $n = 1$ (Theorem 5.5). We actually will prove a stronger claim: the scheme is unforgeable even if the adversary is given one additional oracle $\text{HELP}(A_0, \vec{A}, Z)$ that returns 1 if $Z = (x_0 + x_r)A_0 + \sum_i x_i A_i$ and 0 otherwise. This additional requirement does not change affect the bound and it will simplify the future analysis of the keyed-verification credential scheme.

Keyed-verification credential. Let $R_{\text{cmz}} := R_{\text{cmz.iu}} \cup R_{\text{cmz.is}} \cup R_{\text{cmz.p}}$. We prove the following:

Theorem 5.2. *If ZKP is a proof system for the relation $R \supseteq R_{\text{cmz}}$, then μCMZ is a keyed-verification extractable credential for $n = \text{poly}(\lambda)$ attributes with anonymity advantage:*

$$\text{Adv}_{\mu\text{CMZ}}^{\text{anon}}(\lambda, n) \leq \text{Adv}_{\text{ZKP}_{\text{cmz.iu}}}^{\text{zk}}(\lambda) + \text{Adv}_{\text{ZKP}_{\text{cmz.p}}}^{\text{zk}}(\lambda) + \text{Adv}_{\text{ZKP}_{\text{cmz.is}}}^{\text{ksnd}}(\lambda) ,$$

and extractability advantage:

$$\text{Adv}_{\mu\text{CMZ}}^{\text{ext}}(\lambda, n) \leq \text{Adv}_{\mu\text{CMZ}}^{\text{ufcmva}}(\lambda, n) + \text{Adv}_{\text{ZKP}_{\text{cmz.is}}}^{\text{zk}}(\lambda) + \text{Adv}_{\text{ZKP}_{\text{cmz.iu}}}^{\text{ksnd}}(\lambda) + \text{Adv}_{\text{ZKP}_{\text{cmz.p}}}^{\text{ksnd}}(\lambda) .$$

In particular, $\mu\text{CMZ}[\text{ZKP} = \Sigma]$ has statistical anonymity if we assume that Σ is statistically knowledge sound.

Anonymous token. Let μCMZ_{AT} denote the variant of μCMZ in Figure 9 where the issuance algorithm does not return the proof π_{iu} (that is, the boxed areas are removed). From the above, and Theorem 5.11, we also have:

Theorem 5.3. *If ZKP is a proof system for the relation $R \supseteq R_{\text{cmz.p}} \cup R_{\text{cmz.is}}$, then μCMZ_{AT} is an anonymous token for $n = \text{poly}(\lambda)$ attributes with anonymity advantage:*

$$\text{Adv}_{\mu\text{CMZ}_{AT}}^{\text{anon}}(\lambda, n) \leq \text{Adv}_{\text{ZKP}_{\text{cmz.iu}}}^{\text{zk}}(\lambda) + \text{Adv}_{\text{ZKP}_{\text{cmz.p}}}^{\text{zk}}(\lambda) + \text{Adv}_{\text{ZKP}_{\text{cmz.is}}}^{\text{ksnd}}(\lambda) ,$$

and (in the algebraic group model) one-more unforgeability advantage:

$$\text{Adv}_{\mu\text{CMZ}_{AT}}^{\text{omuf}}(\lambda, n) \leq \text{Adv}_{\text{GrGen}}^{2\text{dl}}(\lambda) + q\text{Adv}_{\text{GrGen}}^{\text{dl}}(\lambda) + \frac{q+5}{p} ,$$

where q is the number of queries to the issuance oracle.

5.3 Algebraic MAC

We prove that μCMZ is an algebraic MAC first for the special case $n = 1$ in Theorem 5.4, and then reduce the case $n > 1$ to the case $n = 1$ in Theorem 5.5.

Lemma 5.4. *In the algebraic group model, μCMZ is an algebraic MAC for $n = 1$ attributes in $\mathbb{Z}_p$ with advantage:*

$$\text{Adv}_{\mu\text{CMZ}}^{\text{ufcmva}}(\lambda, 1) = \text{Adv}_{\text{GrGen}}^{3\text{-dl}}(\lambda) + \text{Adv}_{\text{GrGen}}^{\text{dl}}(\lambda) + \frac{1}{p}.$$

The unforgeability security experiment (Figure 5) provides the adversary with access to signing and verification oracles. In the proof below, the adversary has also access to an oracle $\text{HELP}(A_0, A_1, Z)$ which checks whether $Z = (x_0 + x_r)A_0 + x_1A_1$. The oracle returns 1 if the equation is satisfied and 0 otherwise. This stronger notion will simplify the proof of Theorem 5.11.

Proof. Statistical correctness follows by inspection; we prove unforgeability. Let $\mathbf{A}$ be an adversary that receives as input some public parameters $pp = (X_0, X_r, \vec{X})$ has access to a sign oracle SIGN , a verification oracle VERIFY , and a helper oracle HELP . It outputs a forgery $(m^*, (U^*, V^*))$. The j -th oracle query to $\text{SIGN}(m_j)$ takes as input $m_j \in \mathbb{Z}_p$ and outputs a MAC $\sigma_j = (U_j, V_j)$. Let q be the number of signing queries made during the execution of the adversary. The output of an algebraic adversary is accompanied by an algebraic representation $\vec{\alpha}, \vec{\beta}$ of the form

$$\begin{aligned} U^* &= \alpha_g G + \alpha_h H + \alpha_0 X_0 + \alpha_{x_r} X_r + \alpha_1 X_1 + \sum_j^q \alpha_{u,j} U_j + \alpha_{v,j} V_j, \\ V^* &= \beta_g G + \beta_h H + \beta_0 X_0 + \beta_{x_r} X_r + \beta_1 X_1 + \sum_j^q \beta_{j,u} U_j + \beta_{j,v} V_j. \end{aligned}$$

The verification equation requires that U^* is non-zero and that the following polynomial equation in $\mathbb{Z}_p[\eta, \mathbf{x}_0, \mathbf{x}_r, \mathbf{x}_1, \mathbf{u}_1, \dots, \mathbf{u}_q]$

$$\begin{aligned} &\left(\alpha_g + \alpha_h \eta + \alpha_0 \mathbf{x}_0 \eta + \alpha_r \mathbf{x}_r + \alpha_1 \mathbf{x}_1 + \sum_j^q \alpha_{u,j} \mathbf{u}_j + \alpha_{v,j} \mathbf{u}_j (\mathbf{x}_0 + \mathbf{x}_r + m_j \mathbf{x}_1) \right) (\mathbf{x}_0 + \mathbf{x}_r + m^* \mathbf{x}_1) = \\ &\quad \beta_g + \beta_h \eta + \beta_0 \mathbf{x}_0 \eta + \beta_r \mathbf{x}_r + \beta_1 \mathbf{x}_1 + \sum_j^q \beta_{u,j} \mathbf{u}_j + \beta_{v,j} \mathbf{u}_j (\mathbf{x}_0 + \mathbf{x}_r + m_j \mathbf{x}_1) \end{aligned} \quad (12)$$

holds when evaluated in $\mathbf{x}_0 = \log_H X_0$, $\mathbf{x}_1 = \log_G X_1$, $\eta = \log_G H$ and $\mathbf{u}_j = \log_G U_j$. We first study the case that Equation (12) holds over the polynomial ring, and then argue equality over the field. For the equation to hold, we must have that all monomials on the left-hand side must appear also on the right-hand side of the equation. In particular this implies:

- $\beta_g = 0$, $\beta_h = 0$, $\beta_{u,j} = 0$ since no monomials in (respectively) 1, η , and $\mathbf{u}_j$ (for $j \in [q]$) appear on the left-hand side of the equation;
- $\alpha_g = 0$, $\alpha_h = 0$, $\alpha_0 = 0$, $\alpha_r = 0$, $\alpha_1 = 0$, $\alpha_{v,j} = 0$ since no monomials in (respectively) $\mathbf{x}_0$, $\eta \mathbf{x}_r$, $\mathbf{x}_0^2 \eta$, $\mathbf{x}_r \mathbf{x}_0$, $\mathbf{x}_1 \mathbf{x}_0$ and $\mathbf{u}_j \mathbf{x}_0^2$ appear on the right-hand side of the equation.

Thus, it can be re-written as:

$$\left(\sum_j \alpha_{u,j} \mathbf{u}_j \right) (\mathbf{x}_0 + \mathbf{x}_r + m_j \mathbf{x}_1) = \beta_0 \mathbf{x}_0 \eta + \beta_r \mathbf{x}_r + \beta_1 \mathbf{x}_1 + \sum_j^q \beta_{v,j} \mathbf{u}_j (\mathbf{x}_0 + \mathbf{x}_r + m^* \mathbf{x}_1),$$

which can be further simplified to

$$\left(\sum_j^q \alpha_{u,j} \mathbf{u}_j\right)(\mathbf{x}_0 + \mathbf{x}_r + m^* \mathbf{x}_1) = \sum_j^q \beta_{v,j} \mathbf{u}_j(\mathbf{x}_0 + \mathbf{x}_r + m_j \mathbf{x}_1) .$$

which in turn means that $\alpha_{u,j} = \beta_{v,j}$ and $\alpha_{u,j} \cdot m^* = \alpha_{u,j} \cdot m_j$ for all $j \in [q]$. Since U^* is not the identity (by MAC verification), then $\exists j \in [q]: \alpha_{u,j} \neq 0$, and therefore $m^* = m_j$ for some j . This is not possible, since m^* was not queried to the signing oracle. It must therefore be the case that, for a winning adversary, Equation (12) does not hold over $\mathbb{Z}_p[\eta, \mathbf{x}_0, \mathbf{x}_r, \mathbf{x}_1, \mathbf{u}_1, \dots, \mathbf{u}_q]$ but does hold when evaluated in the relative discrete logarithms.

We build a reduction $\mathbf{B}$ to 3-DL. The adversary $\mathbf{B}$ takes as input some group description Γ and $(X, X', X'') \in \mathbb{G}^3$. It samples random $a_h, b_h, a_0, b_0, a_r, b_r, a_1, b_1 \leftarrow \mathbb{Z}_p$ and invokes $\mathbf{A}$ with public parameters

$$\begin{aligned} pp := & (\Gamma, \\ & H = a_h G + b_h X, \\ & X_0 = a_h a_0 G + (a_h b_0 + b_h) X + b_h b_0 X', \\ & X_r = a_r G + b_r X, \\ & X_1 = a_1 G + b_1 X) , \end{aligned} \tag{13}$$

which is identically distributed to the one of an honest signer. The adversary can make queries to the oracles SIGN, VERIFY, HELP.

- For the j -th signing query on a message m_j , the reduction $\mathbf{B}$ samples $a_{u,j}, b_{u,j} \leftarrow \mathbb{Z}_p$ and returns

$$\begin{aligned} (U_j := & a_{u,j} G + b_{u,j} X, \\ V_j := & a_{u,j}(a_h a_0 + a_h + a_1 m_j) G \\ & + (b_{u,j}(a_0 + a_r + a_1 m_j) + a_{u,j}(b_0 + b_r + b_1 m_j)) X \\ & + b_{u,j}(b_0 + b_r + b_1 m_j) X' . \end{aligned} \tag{14}$$

The distribution of U_j, V_j is identically distributed to the one of the security experiment.

- A query to the verification oracle VERIFY receives as input a message m and a MAC (U, V) , accompanied by the algebraic representation $\vec{\alpha}, \vec{\beta}$ such that:

$$\begin{aligned} U = & \alpha_g G + \alpha_h H + \alpha_0 X_0 + \alpha_r X_r + \alpha_1 X_1 + \sum_j^{n_s} \alpha_{u,j} U_j + \alpha_{v,j} V_j, \\ V = & \beta_g G + \beta_h H + \beta_0 X_0 + \beta_r X_r + \beta_1 X_1 + \sum_j^{n_s} \beta_{u,j} U_j + \beta_{v,j} V_j \end{aligned}$$

where $n_s < q$ is the current number of queries made to SIGN. The oracle tests the verification equation plugging Equations (13) and (14) into the verification equation and checking it using (X, X') as the maximum degree of the resulting polynomial is 2. If the equation is not satisfied, the oracle returns 0. Otherwise, it returns 1. The output is the same as the one of the security experiment.

- Any HELP query of the form $(A_0, A_1, Z) \in \mathbb{G}^3$ has algebraic representation:

$$\begin{aligned}
A_0 &= \gamma_g^{(0)}G + \gamma_h^{(0)}H + \gamma_0^{(0)}X_0 + \gamma_r^{(0)}X_r + \gamma_1^{(0)}X_1 + \sum_j \gamma_{u,j}^{(0)}U_j + \gamma_{v,j}^{(0)}V_j, \\
A_1 &= \gamma_g^{(1)}G + \gamma_h^{(1)}H + \gamma_0^{(1)}X_0 + \gamma_r^{(1)}X_r + \gamma_1^{(1)}X_1 + \sum_j \gamma_{u,j}^{(1)}U_j + \gamma_{v,j}^{(1)}V_j, \\
Z &= \delta_g G + \delta_h H + \delta_0 X_0 + \delta_r X_r + \delta_j X_1 + \sum_j \delta_{u,j} U_j + \delta_{v,j} V_j.
\end{aligned} \tag{15}$$

Similarly to the verification oracle, **B** can respond to the query evaluating the associated polynomial in X via (G, X, X', X'') , as the maximum degree of the resulting expression is 3. At the end, the adversary **A** returns a forgery $(m^*, (U^*, V^*))$ satisfying the verification equation and such that $m^* \neq m_i$ for all $i \in [q]$. From Equation (12) we know that (since $m^* \neq m_i$ for all i 's):

$$\begin{aligned}
\varphi(\eta, \mathbf{x}_0, \mathbf{x}_1, \mathbf{u}_1, \dots, \mathbf{u}_q) &:= (\alpha_g + \alpha_h \eta + \alpha_0 \mathbf{x}_0 \eta + \alpha_r \mathbf{x}_r + \alpha_1 \mathbf{x}_1 + \\
&\quad \sum_j^q \alpha_{u,j} \mathbf{u}_j + \alpha_{v,j} \mathbf{u}_j (\mathbf{x}_0 + \mathbf{x}_r + m_j \mathbf{x}_1)) (\mathbf{x}_0 + \mathbf{x}_r + m^* \mathbf{x}_1) - \\
&\quad \beta_g + \beta_h \eta + \beta_0 \mathbf{x}_0 \eta + \beta_r \mathbf{x}_r + \beta_1 \mathbf{x}_1 + \sum_j^q \beta_{u,j} \mathbf{u}_j + \beta_{v,j} \mathbf{u}_j (\mathbf{x}_0 + \mathbf{x}_r + m_j \mathbf{x}_1)
\end{aligned}$$

is non-zero. Since the $b_h, b_0, b_r, b_1, b_{u,1}, \dots, b_{u,q}$'s are all uniformly random and perfectly hidden by the respective $a_h, a_0, a_1, a_{u,1}, \dots, a_{u,q}$'s, the partial evaluation

$$\psi(\chi) = \varphi(a_h + \chi b_h, a_0 + \chi b_0, a_r + \chi b_r, a_1 + \chi b_1, a_{u,1}, \dots, a_{u,q} + \chi b_{u,q}) \tag{16}$$

is still a non-zero polynomial in $\mathbb{Z}_p[\chi]$, of degree at most 3, except with probability $\frac{1}{p}$. If the given polynomial is nontrivial, since the verification equation is satisfied, $\psi(\log_G X) = 0$. Therefore, one of the roots of ψ is in the field and is the discrete logarithm of X with respect to G . The reduction finds the root of ψ satisfying the discrete logarithm of X , and returns it. $\square$

Lemma 5.5. μCMZ is an algebraic MAC for $n = \text{poly}(\lambda)$ attributes in $\mathbb{Z}_p$ with advantage:

$$\text{Adv}_{\mu\text{CMZ}}^{\text{ufcmva}}(\lambda, n) = \text{Adv}_{\text{GrGen}}^{3\text{-dl}}(\lambda) + \text{Adv}_{\text{GrGen}}^{\text{dl}}(\lambda) + \frac{3}{p}.$$

The unforgeability security experiment (Figure 5) provides the adversary with access to signing and verification oracles. In the proof below, the adversary has also access to an oracle $\text{HELP}(A_0, \vec{A}, Z)$ which checks whether $Z = (x_0 + x_r)A_0 + \sum_i x_i A_i$. The oracle returns 1 if the equation is satisfied and 0 otherwise. This stronger notion will simplify the proof of Theorem 5.11.

Proof. Let **A** be an adversary against MAC unforgeability, taking as input the public parameters $(\Gamma, H, X_0, \dots, X_n)$ and outputting some forgery $(\vec{m}^*, (U^*, V^*))$. **A** has access to the oracles **SIGN**, **VERIFY**, and **HELP**. **SIGN** outputs a MAC (U_j, V_j) for a queried message $\vec{m}_j$. **VERIFY** outputs the verification output of μCMZ . Let q be the number of queries to **SIGN**. We distinguish two valid forgeries:

- (i) $\sum_{i=1}^n m_i^* X_i = \sum_{i=1}^n m_{j,i} X_i$ for some previously queried message $\vec{m}_j$ ($j \in [q]$). In this case, by Theorem 5.6 below, the adversary **A** can solve the gap DL problem with advantage

$$\text{Adv}_{\text{GrGen}}^{\text{gapdl}}(\lambda) + \frac{1}{p}.$$

- (ii) $\sum_{i=1}^n m_i^* X_i \neq \sum_{i=1}^n m_{j,i} X_i$ for all $j \in [q]$. In this case, by Theorem 5.7, the adversary A can forge a MAC with advantage

$$\text{Adv}_{\mu\text{CMZ}}^{\text{ufcmva}}(\lambda, 1) + \frac{1}{p}.$$

Thus, the overall advantage is

$$\begin{aligned} \text{Adv}_{\mu\text{CMZ}}^{\text{ufcmva}}(\lambda, n) &= \text{Adv}_{\mu\text{CMZ}}^{\text{ufcmva}}(\lambda, 1) + \text{Adv}_{\text{GrGen}}^{\text{gapdl}}(\lambda) + \frac{2}{p} && \text{(by Theorems 5.6 and 5.7)} \\ &= \text{Adv}_{\text{GrGen}}^{3\text{-dl}}(\lambda) + \text{Adv}_{\text{GrGen}}^{\text{dl}}(\lambda) + \frac{3}{p}. && \text{(by Theorem 5.4)} \end{aligned}$$

□

Claim 5.6. *Item (i) happens with advantage $\text{Adv}_{\text{GrGen}}^{\text{gapdl}}(\lambda) + 1/p$.*

We build a reduction B that solves gap DL for GrGen every time A returns a forgery $\vec{m}^*$ such that $\sum_i m_i^* X_i = \sum_i m_{j,i} X_i$ for some $j \in [q]$. The adversary B has access to an oracle $\text{DDH}(A, Z)$ that returns 1 if Z is the DH of X and A . B takes as input (Γ, X) with $X \in \mathbb{G}$ and samples $a_i, b_i \leftarrow \mathbb{Z}_p$ and computes $X_i = a_i G + b_i X$ for $i \in [n]$ and $X_0 = zH$ for some $z \leftarrow \mathbb{Z}_p$. It invokes A with $pp = (\Gamma, (X_0, X_r, X_1, \dots, X_n))$. The reduction B treats the oracle queries as follows:

- For every signing query on message $\vec{m}_j$, B returns

$$(U := uG, \quad V := (z + \sum_i a_i m_{j,i})U + u(\sum_i b_i m_i)X)$$

for $u \leftarrow \mathbb{Z}_p$.

- For every $\text{VERIFY}(\vec{m}, (U, V))$ query, it checks if $\sum_i b_i m_i = 0$. If $\sum_i b_i m_i = 0$, it returns 1 if $zU + \sum_i a_i m_i U = V$ and 0 otherwise. If $\sum_i b_i m_i \neq 0$, it returns the output bit of $\text{DDH}(U, (\sum_i b_i m_i)^{-1}V - zU)$.
- For every $\text{HELP}(A_0, \vec{A}, Z)$ query, it returns the output bit of $\text{DDH}(\sum_i b_i A_i, Z - zA_0 - \sum_i a_i A_i)$.

At the end, if the adversary A returns $\vec{m}^*$ and a forged MAC (U^*, V^*) satisfying

$$\sum_i m_{j,i}(a_i G + b_i X) = \sum_i m_i^*(a_i G + b_i X)$$

and $\vec{m}^* \neq \vec{m}_j$, for some $j \in [q]$. Thus, $(\sum_i (m_{j,i} - m_i^*)a_i)G = (\sum_i (m_{j,i} - m_i^*)b_i)X$. The equation is non-trivial with overwhelming probability $1/p$, since the b_i 's are uniformly random and perfectly hidden in X_i 's by a_i . Therefore, except with negligible probability $1/p$ the reduction B solves the above equation and returns $x = (\sum_i a_i (m_{j,i} - m_i^*))(\sum_i b_i (m_{j,i} - m_i^*))^{-1}$, the discrete logarithm of X . It follows that the adversary wins (i) with advantage at most

$$\text{Adv}_{\text{GrGen}}^{\text{gapdl}}(\lambda) + 1/p.$$

Claim 5.7. *Item (ii) happens with advantage at most $\text{Adv}_{\mu\text{CMZ}}^{\text{ufcmva}}(\lambda, 1) + 1/p$.*

We reduce to the security of μCMZ for $n = 1$. The reduction B gets as input the public parameters $(\Gamma, H, X_0, X_r, X_1)$. Sample $z_i \leftarrow \mathbb{Z}_p$ and compute $X_i = z_i X_1$ for $i \in [2, n]$. Assuming $X_1 \neq 0G$ (which happens with probability $1/p$), all the X_i 's are uniformly distributed. The reduction then internally runs the adversary A for the unforgeability game with public parameters $(X_0, X_1, \dots, X_n)$. The reduction treats the oracle queries as follows:

- For every MAC query of the form $\vec{m}$, forward the query to the oracle SIGN at disposal to B for the message $m_1 + \sum_i^n z_i m_i$, obtaining (U, V) . Return (U, V) to A.
- For every verification query for a message $\vec{m}$ and a MAC (U, V) , forward the query to the oracle VERIFY with $(m_1 + \sum_i z_i m_i, (U, V))$.
- Similarly, proceed for HELP queries.

Given an output forgery (U^*, V^*) for a message $\vec{m}^*$, we have that $\sum_i m_i^* X_i \neq \sum_i m_{j,i} X_i$ for all $j \in [q]$, B outputs the forged message $m_1 + \sum_{i=2}^n z_i m_i$ along with the MAC (U^*, V^*) . Therefore, $m_1^* + \sum_{i=2}^n m_i^* z_i \neq m_{j,1} + \sum_{i=2}^n m_{j,i} z_i$ for all $j \in [q]$ and thus the output message was never queried to the SIGN oracle; by construction of the X_i 's we have that

$$(U^*, V^*) = (U, x_0 U + \sum_{i=1}^n m_i^* x_i U) = (U, x_0 U + (m_1^* + \sum_{i=2}^n z_i m_i^*) x_1 U)$$

where $x_0 = \log_H X_0$, $x_1 = \log_G X_1$, and $z_i x_1 = \log_G X_i$ for $i \in [2, n]$. Hence, the forgery is valid and B produces a forgery every time A produces a forgery.

5.4 Anonymity

Theorem 5.8. *If ZKP a proof system for the relation $R \supseteq R_{\text{cmz}}$, then μCMZ is anonymous for $n = \text{poly}(\lambda)$ attributes with advantage:*

$$\text{Adv}_{\mu\text{CMZ}, A, D}^{\text{anon}}(\lambda, n) \leq \text{Adv}_{\text{ZKP}_{\text{cmz}, \text{iu}}, A'}^{\text{zk}}(\lambda) + \text{Adv}_{\text{ZKP}_{\text{cmz}, D'}}^{\text{zk}}(\lambda) + \text{Adv}_{\text{ZKP}_{\text{cmz}, \text{is}}, A''}^{\text{ksnd}}(\lambda),$$

where A, D are adversaries in the anonymity game and A', A'', D' are described in the proof.

Overall, the technical challenge here is making sure that extracting x_0 is sufficient for simulating proofs, and keeping track of the different adversaries in the game.

Proof. We define the simulator Sim as:

- The issuance simulator $\text{Sim.I}((X_r, X_0, \vec{X}), \phi)$ samples a random group element $C' \leftarrow \$ \mathbb{G}$ and simulates a representation proof π_{iu} for the instance $(C, \vec{X}, \phi)$. Upon receiving the response from the server $(U', V', \pi_{\text{is}})$, the simulator checks if the proof is valid and $U' \neq 0$. If so, it invokes the extractor for the instance (X_0, U', V') and proof π_{is} recovering (x_0, u) satisfying $x_0 = \log_H X_0$ and $u = \log_G U$. Note that x_0 is the unique value associated to the public parameter's X_0 . Finally, it stores the state as $st_{\text{Sim}} = (x_0, \vec{X})$.
- The presentation simulator $\text{Sim.P}((X_0, X_r, \vec{X}), \vec{m})$, which does not know the message. It samples random group elements $U', C_V \leftarrow \$ \mathbb{G}^2$, and $(C_1 = \gamma_1 G, \dots, C_n := \gamma_n G)$ where $\gamma_i \leftarrow \$ \mathbb{Z}_p$ for $i \in [n]$, and computes π_p via $\text{ZKP}_{\text{cmz}, P}.\text{Sim}((U', X_0, \vec{X}, \vec{C}, Z))$ where $Z = x_0 U' + \sum_i \gamma_i X_i - C_V$. The simulator outputs $(U', C_V, \vec{C}, \pi_p)$.

Let A be a p.p.t. machine taking as input a keypair (sk, pp) , a predicate $\phi \in \Phi$, and a message $\vec{m}$ such that $\phi(\vec{m}) = 1$.

Indistinguishability is shown via a hybrid argument.

H_0 This is the real interaction.

H₁ Instead of generating π_{iu} using the witness, do so via the zero-knowledge simulator Sim with input $(C', \vec{X}, \phi)$. If A 's output is noticeably different after this hybrid change, then it is possible to build an adversary A' that is able to distinguish real from simulated proofs from $\text{ZKP}_{\text{cmz.iu}}$. Therefore:

$$\text{Adv}_{A,D}^{H_1}(\lambda, n) \geq \text{Adv}_{A,D}^{H_0}(\lambda, n) - \text{Adv}_{\text{ZKP}_{\text{cmz.iu}}, A'}^{\text{zk}}(\lambda) .$$

H₂ Instead of generating π_p using the attributes $\vec{m}$ and MAC σ , use the zero-knowledge simulator Sim with input $(U', X_0, \vec{X})$. Similarly to the previous hybrid, there exists an adversary D' that wins every time D 's output is noticeably different. Therefore:

$$\text{Adv}_{A,D}^{H_2}(\lambda, n) \geq \text{Adv}_{A,D}^{H_1}(\lambda, n) - \text{Adv}_{\text{ZKP}_{\text{cmz.iu}}, D'}^{\text{zk}}(\lambda) .$$

H₃ Use the extractor of π_{is} for the statement (X_0, U', V') and check its validity.

The extractor will produce a witness (x_0, u) satisfying:

$$\begin{bmatrix} H & 0 \\ 0 & G \\ U' & C'' \end{bmatrix} \begin{bmatrix} x_0 \\ u \end{bmatrix} = \begin{bmatrix} X_0 \\ U' \\ V' \end{bmatrix}$$

except with probability $\text{Adv}_{\text{ZKP}_{\text{cmz.is}}, A''}^{\text{ksnd}}(\lambda)$, where A'' is the adversary in the knowledge soundness game that runs A internally and returns the proof π_{is} for which extraction fails. In other words,

$$\text{Adv}_{A,D}^{H_3}(\lambda, n) \geq \text{Adv}_{A,D}^{H_2}(\lambda, n) - \text{Adv}_{\text{ZKP}_{\text{cmz.is}}, A''}^{\text{ksnd}}(\lambda) .$$

H₄ At issuance time, instead of computing C' using the message as $C' = \sum_i m_i X_i + sG$ for some $s \leftarrow \mathbb{Z}_p$, send a random element $C' \leftarrow \mathbb{G}$ and extract from the response π_{is} the witness (u, x_0) . Then, using the extracted witness, compute a fresh MAC $(U = ruG, x_0U + urX_r + \sum_i rum_i X_i)$ for $r \leftarrow \mathbb{Z}_p$ and return it. The distribution in this hybrid is identical to the previous one: U is distributed identically, and V is the (unique) group element satisfying $V = (\log_H X_0)U + (\log_G U) \cdot (X_r + \sum_i m_i X_i)$. Hence:

$$\text{Adv}_{A,D}^{H_4}(\lambda, n) = \text{Adv}_{A,D}^{H_3}(\lambda, n) .$$

H₅ At presentation time, sample $U', C_V \leftarrow \mathbb{G}^2$ and $\vec{C}$ such that $C_i = \gamma_i G$ with $\gamma_i \leftarrow \mathbb{Z}_p$. The (simulated) proof π_p is made for the statement $(U', C_V, \vec{C}, Z)$ where Z is computed as:

$$Z = x_0 U' + \sum_i \gamma_i X_i - C_V .$$

The elements $\vec{C}, C_V$ are identically distributed to the real ones, by the perfect hiding property of Pedersen commitments. The element U' is also perfectly indistinguishable from the real one, since r is uniformly distributed and $U \neq 0G$. Finally, the value Z , despite being computed differently, is exactly the same in both distributions since $\gamma_i X_i = x_i C_i$ where $x_i = \log_G X_i$. Hence:

$$\text{Adv}_{A,D}^{H_5}(\lambda, n) = \text{Adv}_{A,D}^{H_4}(\lambda, n) .$$

The last hybrid is the simulated interaction; the result follows. $\square$

Remark 5.9 (Knowledge soundness vs simulation extractability). *To extract from π_{iu} after seeing (simulated) issuance proofs π_{is} , one would formally need to make sure that the proofs π_{is} cannot be turned into proofs for $R_{cmz.iu}$. This is generally achieved by proofs that are simulation extractable, by making sure that different proof types are with respect to different setup strings (or random oracles). In many cases the actual requirement can be weaker. For instance, in Schnorr proofs, the vector sizes mismatch and thus knowledge soundness suffices.*

5.5 Extractability

Theorem 5.10. *If ZKP is a proof system for the relation $R \supseteq R_{cmz}$, then μCMZ is extractable for $n = \text{poly}(\lambda)$ attributes with advantage:*

$$\text{Adv}_{\mu\text{CMZ}}^{\text{ext}}(\lambda, n) \leq \text{Adv}_{\mu\text{CMZ}}^{\text{ufcmva}}(\lambda, n) + \text{Adv}_{\text{ZKP}_{cmz.is}}^{\text{zk}}(\lambda) + \text{Adv}_{\text{ZKP}_{cmz.iu}}^{\text{ksnd}}(\lambda) + \text{Adv}_{\text{ZKP}_{cmz.p}}^{\text{ksnd}}(\lambda) .$$

Roughly speaking, the extractor matches the zero-knowledge extractor and a reduction is made to unforgeability of the underlying MAC. However, the term “Z” is computed during presentation verification using the MAC key $(x_0, x_r, \vec{x})$, which is not available to the reduction to unforgeability. To circumvent this, we are going to assume that the proof itself can provide a candidate instance “Z”, and that the instance can be check for correctness via the oracle HELP (introduced in Section 5.3). In practice, this requirement is trivially satisfied for Schnorr proofs, where it is possible to recover the statement from the actual proof by looking at the trace of random oracle queries.

Proof. We use the extractors of ZKP to define the extractors for the credential system.

- $\text{Ext.I}(\phi, (C', \pi_{iu}))$ is the zero-knowledge extractor of $\text{ZKP}_{cmz.iu}$ which takes as input the proof π_{iu} and an instance (C', X, ϕ) and $(\vec{m}, s)$ such that $\phi(\vec{m}) = 1$ and

$$\begin{bmatrix} X_1 & X_2 & \dots & X_n & G \end{bmatrix} \begin{bmatrix} m_1 \\ m_2 \\ \vdots \\ m_n \\ s \end{bmatrix} = C' .$$

If no such message is found, the extractor returns $\perp$ and the game aborts.

- $\text{Ext.P}(\phi^*, (U', C_V, \vec{C}, \pi_p))$ runs the extractor of $\text{ZKP}_{cmz.p}$, which takes as input the proof π_p , and recovers an instance $(U', \vec{X}, \vec{C}, Z, \phi^*)$ and a witness $(\vec{m}, \vec{r}, r')$ such that $\phi(\vec{m}) = 1$ and

$$\begin{bmatrix} U' & & & G & & & \\ & U' & & & G & & \\ & & \ddots & & & \ddots & \\ & & & X_1 & X_2 & \dots & X_n & -H \end{bmatrix} \begin{bmatrix} m_1 \\ \vdots \\ m_n \\ r_1 \\ \vdots \\ r_n \\ r' \end{bmatrix} = \begin{bmatrix} C_1 \\ \vdots \\ C_n \\ Z \end{bmatrix} .$$

If the extracted witness does not satisfy the above, the extractor returns $\perp$ and the game aborts. Internally, the extractor checks that Z was correctly computed via the secret key $(x_0, x_r, \vec{x})$.

Let A be an adversary for extraction that gets as input $((\Gamma, H), (X_0, X_r, X_1, \dots, X_n))$ and has access to oracles $\text{ISSUE}, \text{PRESENT}$. At the end the adversary wins if one of the following holds:

- (i) There exists a query to ISSUE of the form $(\phi, (C', \pi_{iu}))$ such that $\text{ZKP}_{\text{cmz.iu}}.V((C', \vec{X}, \phi), \pi_{iu})$, but extraction failed (in particular this happens if $\phi(\vec{m}) = 0$);
- (ii) The adversary returned a pair $(\phi^*, \rho^* = (U', C_V, C_1, \dots, C_n, \pi_p))$ such that π_p verifies, but extraction failed (in particular this happens if $\phi^*(\vec{m}) = 0$);
- (iii) The adversary returned a pair $(\phi^*, \rho^* = (U', C_V, C_1, \dots, C_n, \pi_p))$ and extraction recovered a valid witness $(\vec{m}^*, \vec{r}, r')$ such that $\phi^*(\vec{m})$ is satisfied and $\vec{m}^*$ has not been previously queried.

From Items (i) and (ii) it is immediate to build an adversary for knowledge soundness of the zero-knowledge proof system ZKP . Item (iii) can be reduced to unforgeability of the underlying MAC as follows. Let B be an adversary that gets as input $pp = ((\Gamma, H), (X_0, X_r, X_1, \dots, X_n))$ and has access to the oracles $\text{SIGN}, \text{VERIFY}$ of the MAC unforgeability game. B internally runs $A(pp)$ and handles oracle queries as follows:

- For every $\text{ISSUE}(C', \pi_{iu})$ query, it recovers a valid witness $\vec{m}, s$. It queries $\text{SIGN}(\vec{m})$ and obtains (U, V) . Returns $(U, V + sU)$ and simulates the proof π_{is} .
- For every query $\text{PRESENT}(\phi, (U', C_V, \vec{C}, \pi_p))$ from the adversary, B uses the zero-knowledge extractor to recover $(\vec{m}, \vec{r}, r')$ and returns $\text{VERIFY}(\vec{m}, (U', C_V - r'H))$. At the end A returns a value $\phi^*, \rho^* = (U', C_V, \vec{C}, \pi_p)$. B runs the extractor to obtain m^* and returns $(U', C_V - r'H)$. For presentation messages, we have:

$$C_V - r'H = (x_0 + x_r)U' + \sum_i x_i C_i - Z - r'H = (x_0 + x_r + \sum_i x_i m_i)U' ,$$

where the first equality follows from the answer of HELP and the second follows from the representation proof π_p .

- Signing queries are handled in the same way as above.

Thus, the output $(U', C_V - r'H)$ is a valid MAC only if C_V is a valid presentation message that the server accepted. Validity of the returned forgery from B follows from the winning condition $\vec{m}^* \notin Q_{rs}$. $\square$

5.6 One-more unforgeability

Let μCMZ_{AT} be the μCMZ scheme of Figure 9 where the boxed parts are removed.

Theorem 5.11. *In the algebraic group model, μCMZ_{AT} is an anonymous token for $n = \text{poly}(\lambda)$ attributes in $\mathbb{Z}_p$ with one-more unforgeability advantage:*

$$\text{Adv}_{\mu\text{CMZ}_{AT}}^{\text{omuf}}(\lambda, n) \leq \frac{q+6}{p} + (q+1)\text{Adv}_{\text{GrGen}}^{\text{dl}}(\lambda) + 3\text{Adv}_{\text{GrGen}}^{2\text{-dl}}(\lambda) + \text{Adv}_{\text{GrGen}}^{\text{gapdl}}(\lambda) ,$$

where q is the number of signing queries allowed to the adversary.

Proof. Correctness is straightforward; anonymity follows from Theorem 5.8. We are left with one-more unforgeability. We prove first the case $n = 1$, as the case for $n = \text{poly}(\lambda)$ can be shown later using techniques similar to the ones of the previous section.

The case $n = 1$. Let $\mathbf{A}$ be an q -query bound algebraic adversary that receives as input $(\Gamma, H, (X_0, X_r, X_1))$. Each query to SIGN has as input $C_j \in \mathbb{G}$ (for $j \in [q]$), accompanied by an algebraic representation $\vec{\gamma}^{(j)}$ satisfying:

$$C_j = \gamma_g^{(j)} G + \gamma_h^{(j)} H + \gamma_0^{(j)} X_0 + \gamma_r^{(j)} X_r + \gamma_1^{(j)} X_1 + \sum_{k < j} \gamma_{u,k}^{(j)} U_k + \gamma_{v,k}^{(j)} V_k \quad (17)$$

Consider the associated polynomial in $\mathbb{Z}_p[\eta, \mathbf{x}_0, \mathbf{x}_r, \mathbf{x}_1, \mathbf{u}_1, \dots, \mathbf{u}_q]$:

$$c_j(\eta, \mathbf{x}_0, \mathbf{x}_1, \mathbf{u}_1, \dots, \mathbf{u}_{j-1}) = \gamma_g^{(j)} \eta + \gamma_h^{(j)} \eta + \gamma_0^{(j)} \mathbf{x}_0 \eta + \gamma_r^{(j)} \mathbf{x}_r + \gamma_1^{(j)} \mathbf{x}_1 + \sum_{k < j} \gamma_{u,k}^{(j)} \mathbf{u}_k + \gamma_{v,k}^{(j)} \mathbf{u}_k (\mathbf{x}_0 + \mathbf{x}_r + c_k) \quad (18)$$

where $\eta = \log_G H$, $\mathbf{x}_0 = \log_H X_0$, $\mathbf{x}_r = \log_G X_r$, $\mathbf{x}_1 = \log_G X_1$, $\mathbf{u}_k = \log_G U_k$. As an example, for the first 3 queries we have:

$$\begin{aligned} c_1(\eta, \mathbf{x}_0, \mathbf{x}_r, \mathbf{x}_1) &= \gamma_g^{(1)} \eta + \gamma_h^{(1)} \eta + \gamma_0^{(1)} \mathbf{x}_0 \eta + \gamma_r^{(1)} \mathbf{x}_r + \gamma_1^{(1)} \mathbf{x}_1 \\ c_2(\eta, \mathbf{x}_0, \mathbf{x}_r, \mathbf{x}_1, \mathbf{u}_1) &= \gamma_g^{(2)} \eta + \gamma_h^{(2)} \eta + \gamma_0^{(2)} \mathbf{x}_0 \eta + \gamma_r^{(2)} \mathbf{x}_r + \gamma_1^{(2)} \mathbf{x}_1 + \gamma_{u,1}^{(2)} \mathbf{u}_1 + \gamma_{v,1}^{(2)} \mathbf{u}_1 (\mathbf{x}_0 + \mathbf{x}_r + c_1) \\ c_3(\eta, \mathbf{x}_0, \mathbf{x}_r, \mathbf{x}_1, \mathbf{u}_1, \mathbf{u}_2) &= \gamma_g^{(3)} \eta + \gamma_h^{(3)} \eta + \gamma_0^{(3)} \mathbf{x}_0 \eta + \gamma_r^{(3)} \mathbf{x}_r + \gamma_1^{(3)} \mathbf{x}_1 + \\ &\quad \gamma_{u,1}^{(3)} \mathbf{u}_1 + \gamma_{v,1}^{(3)} \mathbf{u}_1 (\mathbf{x}_0 + \mathbf{x}_r + c_1) + \gamma_{u,2}^{(3)} \mathbf{u}_2 + \gamma_{v,2}^{(3)} \mathbf{u}_2 (\mathbf{x}_0 + \mathbf{x}_r + c_2) \end{aligned}$$

and so on. Equation (18) lives in $\mathbb{Z}_p[\eta, \mathbf{x}_0, \mathbf{x}_r, \mathbf{x}_1, \mathbf{u}_1, \dots, \mathbf{u}_{j-1}]$, no (individual) quadratic terms are present, and the total degree of at most $i + 1$. At the end of the execution, $\mathbf{A}$ returns the MACs $(m_i^*, (U_i^*, V_i^*))_{i=1}^{q+1}$ followed by algebraic representations $\vec{\alpha}^{(i)}, \vec{\beta}^{(i)}$ such that

$$\begin{aligned} U_i^* &= \alpha_g^{(i)} G + \alpha_h^{(i)} H + \alpha_0^{(i)} X_0 + \alpha_r^{(i)} X_r + \alpha_1^{(i)} X_1 + \sum_{j=1}^q \alpha_{u,j}^{(i)} U_j + \alpha_{v,j}^{(i)} V_j, \\ V_i^* &= \beta_g^{(i)} G + \beta_h^{(i)} H + \beta_0^{(i)} X_0 + \beta_r^{(i)} X_r + \beta_1^{(i)} X_1 + \sum_{j=1}^q \beta_{u,j}^{(i)} U_j + \beta_{v,j}^{(i)} V_j. \end{aligned} \quad (19)$$

From the verification equation and Equation (18), $U^* \neq 0G$ and the following polynomial equality:

$$\begin{aligned} \left(\alpha_g^{(i)} + \alpha_h^{(i)} \eta + \alpha_0^{(i)} \mathbf{x}_0 \eta + \alpha_r^{(i)} \mathbf{x}_r + \alpha_1^{(i)} \mathbf{x}_1 + \sum_{j=1}^q \alpha_{u,j}^{(i)} \mathbf{u}_j + \alpha_{v,j}^{(i)} \mathbf{u}_j (\mathbf{x}_0 + \mathbf{x}_r + c_j) \right) (\mathbf{x}_0 + \mathbf{x}_r + m_i^* \mathbf{x}_1) = \\ \beta_g^{(i)} + \beta_h^{(i)} \eta + \beta_0^{(i)} \mathbf{x}_0 \eta + \beta_r^{(i)} \mathbf{x}_r + \beta_1^{(i)} \mathbf{x}_1 + \sum_{j=1}^q \beta_{u,j}^{(i)} \mathbf{u}_j + \beta_{v,j}^{(i)} \mathbf{u}_j (\mathbf{x}_0 + \mathbf{x}_r + c_j) \end{aligned} \quad (20)$$

holds for $j \in [q + 1]$ when evaluated in $\eta = \log_G H$, $\mathbf{x}_0 = \log_H X_0$, $\mathbf{x}_r = \log_G X_r$, $\mathbf{x}_1 = \log_G X_1$ and $\mathbf{u}_j = \log_G U_j$. We study the equation in the polynomial ring $\mathbb{Z}_p[\eta, \mathbf{x}_0, \mathbf{x}_r, \mathbf{x}_1, \mathbf{u}_1, \dots, \mathbf{u}_q]$ first. For the equation to hold:

- For all $i \in [q + 1]$, $\beta_g^{(i)} = 0$, $\beta_h^{(i)} = 0$, since no monomials in (respectively) 1 , η (for $j \in [q]$) appear on the left-hand side of the equation;
- For all $i \in [q + 1]$ $\alpha_g^{(i)} = 0$, $\alpha_h^{(i)} = 0$, $\alpha_0^{(i)} = 0$, $\alpha_r^{(i)} = 0$, $\alpha_1^{(i)} = 0$, since no monomials in (respectively) $\mathbf{x}_0$, $\eta \mathbf{x}_r$, $\mathbf{x}_0^2 \eta$, $\mathbf{x}_r \mathbf{x}_0$, $\mathbf{x}_1 \mathbf{x}_0$ appear on the right-hand side of the equation.

- For all $i \in [q+1]$ and $j \in [q]$, $\alpha_{v,j}^{(i)} = 0$ since no term $\mathbf{x}_0^2 \mathbf{u}_j$ is present on the right-hand side. (In particular, c_j is defined in Equation (18) and by inspection one can observe that the right-hand side equation has no monomials in $\mathbf{x}_0^2$.)

We therefore have:

$$\begin{aligned} & \left(\sum_{j=1}^q \alpha_{u,j}^{(i)} \mathbf{u}_j \right) (\mathbf{x}_0 + \mathbf{x}_r + m_i^* \mathbf{x}_1) = \\ & \beta_0^{(i)} \mathbf{x}_0 \eta + \beta_r^{(i)} \mathbf{x}_r + \beta_1^{(i)} \mathbf{x}_1 + \sum_{j=1}^q \beta_{u,j}^{(i)} \mathbf{u}_j + \beta_{v,j}^{(i)} \mathbf{u}_j (\mathbf{x}_0 + \mathbf{x}_r + c_j), \end{aligned} \quad (21)$$

which in turn implies that $\beta_0^{(i)} = 0$, $\beta_r^{(i)} = 0$ and $\beta_1^{(i)} = 0$, since no monomials in $\mathbf{x}_0 \eta$, $\mathbf{x}_r$, and $\mathbf{x}_1$ appear on the left-hand side of the equation. Plugging Equation (18) in the above, for all $i \in [q+1]$ and $j \in [q]$:

- $\beta_{v,j}^{(i)} \gamma_{u,k}^{(j)} = 0$, $\beta_{v,j}^{(i)} \gamma_{v,k}^{(j)} = 0$, $\beta_{v,j}^{(i)} \gamma_g^{(j)} = 0$ for all $k < j$, since no monomial in (respectively) $\mathbf{u}_j \mathbf{u}_k$, $\mathbf{u}_j$ appears on the left-hand side (cf. Equation (18)).
- $\beta_{v,j}^{(i)} \gamma_h^{(j)} = 0$, and $\beta_{v,j}^{(i)} \gamma_0^{(j)} = 0$ since no monomial in (respectively) $\mathbf{u}_j \eta$, and $\mathbf{u}_j \eta \mathbf{x}_0$ appears on the left-hand side of the equation.

Therefore:

$$\sum_{j=1}^q \alpha_{u,j}^{(i)} \mathbf{u}_j \mathbf{x}_0 + \alpha_{u,j}^{(i)} \mathbf{u}_j \mathbf{x}_r + \alpha_{u,j}^{(i)} m_i^* \mathbf{u}_j \mathbf{x}_1 = \sum_{j=1}^q (\beta_{u,j}^{(i)} + \beta_{v,j}^{(i)} \gamma_g^{(i)}) \mathbf{u}_j + \beta_{v,j}^{(i)} \mathbf{u}_j \mathbf{x}_0 + \beta_{v,j}^{(i)} \gamma_r^{(j)} \mathbf{u}_j \mathbf{x}_r + \beta_{v,j}^{(i)} \gamma_1^{(j)} \mathbf{u}_j \mathbf{x}_1$$

which implies that $\alpha_{u,j}^{(i)} = \beta_{u,j}^{(i)}$ (when looking at the terms in $\mathbf{u}_j \mathbf{x}_0$) and there exists at least one non-zero j -indexed term since $U_i^* \neq 0G$. Furthermore, $m_i^* \alpha_{u,j} = \beta_{v,j} \gamma_1^{(j)}$ (looking at the terms in $\mathbf{u}_j \mathbf{x}_1$) implies $m_i^* = \gamma_1^{(j)}$. By the pigeonhole principle, since $i \in [q+1]$ and $j \in [q]$, two forgeries must be on the same message, which contradicts the winning condition that requires all forgeries to be on distinct messages. It must therefore be the case that, for a winning adversary, Equation (12) does not hold over $\mathbb{Z}_p[\eta, \mathbf{x}_0, \mathbf{x}_r, \mathbf{x}_1, \mathbf{u}_1, \dots, \mathbf{u}_q]$ but does hold when evaluated in the relative discrete logarithms.

Denote with $\vec{\alpha}^{(i)}, \vec{\beta}^{(i)}$ the algebraic representation of the i -th forgery of the adversary (U_i^*, V_i^*) . Form the above argument we have that for the (non-zero) polynomial

$$\begin{aligned} & \varphi_i(\eta, \mathbf{x}_0, \mathbf{x}_r, \mathbf{x}_1, \mathbf{u}_1, \dots, \mathbf{u}_q) := \\ & \left(\alpha_g^{(i)} + \alpha_h^{(i)} \eta + \alpha_0^{(i)} \mathbf{x}_0 \eta + \alpha_r^{(i)} \mathbf{x}_r + \alpha_1^{(i)} \mathbf{x}_1 + \sum_{j=1}^q \alpha_{u,j}^{(i)} \mathbf{u}_j + \alpha_{v,j}^{(i)} \mathbf{u}_j (\mathbf{x}_0 + \mathbf{x}_r + c_j) \right) (\mathbf{x}_0 + \mathbf{x}_r + m_i^* \mathbf{x}_1) - \\ & \left(\beta_g^{(i)} + \beta_h^{(i)} \eta + \beta_0^{(i)} \mathbf{x}_0 \eta + \beta_r^{(i)} \mathbf{x}_r + \beta_1^{(i)} \mathbf{x}_1 + \sum_{j=1}^q \beta_{j,u}^{(i)} \beta_{u,j}^{(i)} \mathbf{u}_j + \beta_{v,j}^{(i)} \mathbf{u}_j (\mathbf{x}_0 + \mathbf{x}_r + c_j) \right) \end{aligned} \quad (22)$$

the evaluation in $\log_G H, \log_H X_0, \log_G X_r, \log_G X_1, \log_G U_1, \dots, \log_G U_q$ is zero. The polynomials $\varphi_1, \dots, \varphi_{q+1}$ are all multivariate polynomials of maximum total degree $q+1$. Let $i \in [q+1]$.

- In Theorem 5.12, we show that A wins unforgeability and φ_i has a non-zero monomial in $\mathbf{u}_j$ for $j \in [q]$ with advantage:

$$q \left(\frac{1}{p} + \text{Adv}_{\text{GrGen}}^{\text{dl}}(\lambda) \right).$$

- (ii) In Theorem 5.13, we show that $\mathbf{A}$ wins unforgeability and φ_i has a non-zero monomial in η with advantage:

$$\frac{1}{p} + \text{Adv}_{\text{GrGen}}^{\text{dl}}(\lambda) .$$

- (iii) In Theorem 5.14, we show that $\mathbf{A}$ wins unforgeability and φ_i has a non-zero monomial in $\mathbf{x}_0$, $\mathbf{x}_1$, or $\mathbf{x}_r$ with advantage:

$$3 \left(\frac{1}{p} + \text{Adv}_{\text{GrGen}}^{2\text{-dl}}(\lambda) \right) .$$

Since at least one of the above coefficients must be non-zero, we have a bound on the adversary's winning probability and therefore:

$$\text{Adv}_{\mu\text{CMZ}_{AT}}^{\text{omuf}}(\lambda, 1) \leq \frac{q+4}{p} + (q+1)\text{Adv}_{\text{GrGen}}^{\text{dl}}(\lambda) + 3\text{Adv}_{\text{GrGen}}^{2\text{-dl}}(\lambda) . \quad (23)$$

The general case $n > 1$. The strategy is to distinguish the two winning events:

- (i) The adversary returns forgeries $(m_k^*, (U_k^*, V_k^*))_{k=1}^{q+1}$ such that $\forall k_1 \neq k_2 : \sum_i m_{i,k_1}^* X_i \neq \sum_i m_{i,k_2}^* X_i$.
- (ii) The adversary returns forgeries $(m_k^*, (U_k^*, V_k^*))_{k=1}^{q+1}$ such that $\exists k_1 \neq k_2 : \sum_i m_{i,k_1}^* X_i = \sum_i m_{i,k_2}^* X_i$.

The first case reduces to Theorem 5.11, while the second case can be reduced to DL using an argument similar to Theorem 5.5. Therefore:

$$\begin{aligned} \text{Adv}_{\mu\text{CMZ}_{AT}}^{\text{omuf}}(\lambda, n) &= \text{Adv}_{\mu\text{CMZ}}^{\text{omuf}}(\lambda, 1) + \text{Adv}_{\text{GrGen}}^{\text{gapdl}}(\lambda) + \frac{2}{p} && \text{(by Theorem 5.5)} \\ &= \frac{q+6}{p} + (q+1)\text{Adv}_{\text{GrGen}}^{\text{dl}}(\lambda) + 3\text{Adv}_{\text{GrGen}}^{2\text{-dl}}(\lambda) + \text{Adv}_{\text{GrGen}}^{\text{gapdl}}(\lambda) . \\ &&& \text{(from Equation (23))} \end{aligned}$$

□

Claim 5.12. *Item (i) happens with probability at most $q(1/p + \text{Adv}_{\text{GrGen}}^{\text{dl}}(\lambda))$.*

Proof. We seek a reduction $\mathbf{B}$ that turns an adversary $\mathbf{A}$ providing forgeries satisfying Item (i) into a solution for DL. Fix $\iota \in [q]$. This index will determine the coefficient where we embed the DL challenge.

$\mathbf{B}$ receives as input a group description Γ and a DL challenge $X \in \mathbb{G}$. The adversary samples at random $a, b \leftarrow \$ \mathbb{Z}_p$. $\mathbf{B}$ internally runs $\mathbf{A}$ with public parameters generated as in μCMZ : sample $\eta, x_0, x_r, x_1 \leftarrow \$ \mathbb{Z}_p$ and invoke $\mathbf{A}(\Gamma, H := \eta G, X_0 = x_0 H, X_1 = x_1 G)$. Queries to $\text{SIGN}(C_j, \vec{\gamma}^{(j)})$ – where $\vec{\gamma}^{(j)}$ being the algebraic representation of C_j – are responded as follows:

- For all $j < \iota$, the j -th query is responded following the issuance procedure: sample $u_j \leftarrow \$ \mathbb{Z}_p$ and return $(U_j := u_j G, V_j := (x_0 + x_r)U_j + u_j C_j)$. Note that each signing or verification query up to this point contains an algebraic representation in $H, X_0, X_r, X_1, U_1, \dots, U_{\iota-1}$ and that therefore the discrete logarithm of every group element sent by the adversary is known. We denote by $\hat{\gamma}^{(j)} = \log_G C_j$ (for $j \leq \iota$) the discrete logarithm of the j -th queried element. $\hat{\gamma}^{(j)}$ is computed with the inner product $\langle \vec{\gamma}, (1, \eta, x_0, x_r, x_1, u_1, \dots, u_{j-1}) \rangle$.
- Upon receiving the ι -th query, return $(U_\iota = aG + bX, V_\iota = x_0 aG + x_0 bX + aC + b\hat{\gamma}^{(\iota)} X)$

- For all $\iota < j \leq q$, the j -th query is responded as in the issuance procedure. Sample $u_j \leftarrow \$ \mathbb{Z}_p$ and return $(U_j = u_j G, V_j = x_0 U_j + u_j C_j)$.

Each $\text{VERIFY}(m, (U, V))$ query is responded with 1 if $U \neq 0$ and $V = (x_0 + x_r + m x_1)U$, 0 otherwise. At the end of the execution, $\mathbf{A}$ returns forgeries $(m_i^*, (U_i^*, V_i^*))_{i=1}^{q+1}$ and algebraic representations $\vec{\alpha}^{(i)}, \vec{\beta}^{(i)}$. If $\exists i \in [q+1]$ such that

$$\phi_i^{(u, \iota)}(\chi) = \varphi_i(\eta, x_0, x_r, x_1, u_1, \dots, a + b\chi, u_{\iota+1}, \dots, u_q) \neq 0 ,$$

then it is possible to solve the linear equation and find the discrete logarithm of X . Note that, if φ of Equation (22) has a monomial in $\mathbf{u}_\iota$ with a non-zero coefficient, then the polynomial $\phi(\mathbf{x})$ is non-zero with high probability, since b is uniformly distributed and perfectly hidden by a . Therefore, Item (i) happens with probability at most

$$q \left(\frac{1}{p} + \text{Adv}_{\text{GrGen}}^{\text{dl}}(\lambda) \right) .$$

□

Claim 5.13. *Item (ii) happens with probability at most $1/p + \text{Adv}_{\text{GrGen}}^{\text{dl}}(\lambda)$.*

Proof. We now prove that if φ has a monomial in η , then a solution for DL can be found. Similarly to before, let $X \in \mathbb{G}$ be a DL challenge. The adversary $\mathbf{B}$ samples $a, b \leftarrow \$ \mathbb{Z}_p$ and invokes $\mathbf{A}(\Gamma, H := aG + bX, X_0 = x_0 aG + x_0 bX, X_1 = x_1 G)$. Signing and verification queries are responded as prescribed in the protocol description. At the end, if $\exists i \in [q+1]$ such that

$$\phi_i^{(h)}(\chi) = \varphi_i(a + b\chi, x_0, x_r, x_1, u_1, \dots, \dots, u_q) \neq 0 ,$$

it is possible to solve the linear equation and find the discrete logarithm of X with overwhelming probability. (As before, since b is uniformly distributed and perfectly hidden by a .) □

Claim 5.14. *Item (iii) happens with probability at most $3(1/p + \text{Adv}_{\text{GrGen}}^{2\text{-dl}}(\lambda))$.*

Proof. We prove that if φ has a monomial in $\mathbf{x}_0$, then a solution for 2-DL can be found. (The cases $\mathbf{x}_r$ and $\mathbf{x}_1$ are almost identical.) Let $\mathbf{B}(\Gamma, X, X')$ be an adversary for 2-DL. The adversary samples $\eta, a, b, x_r, x_1 \leftarrow \$ \mathbb{Z}_p$ and internally runs the one-more unforgeability adversary as $\mathbf{A}(\Gamma, H := \eta G, X_0 := aH + b\eta X, X_r := x_r G, X_1 := x_1 G)$. During its execution, the adversary may query the signing oracle with query $\text{SIGN}(C_j)$ which is responded as $(U_j := u_j G, (a + x_r)U_j + bu_j Xu_j C_j)$. The adversary may also query the verification oracle with query $\text{VERIFY}(m_i, (U_i, V_i))$ which is responded with the help of the algebraic representation and X, X' : if $U_i \neq 0$ then its algebraic representation has degree one in $\mathbf{x}_0$ at most, and therefore the verification equation can be tested with the help of X' for the quadratic term. At the end of the execution, the adversary returns forgeries $(m_i^*, (U_i^*, V_i^*))_{i=1}^{q+1}$ and, if $\phi_i^{(x_0)}(\mathbf{x})$ similar to the above is non-zero, then a solution for DL can be found with high probability. □

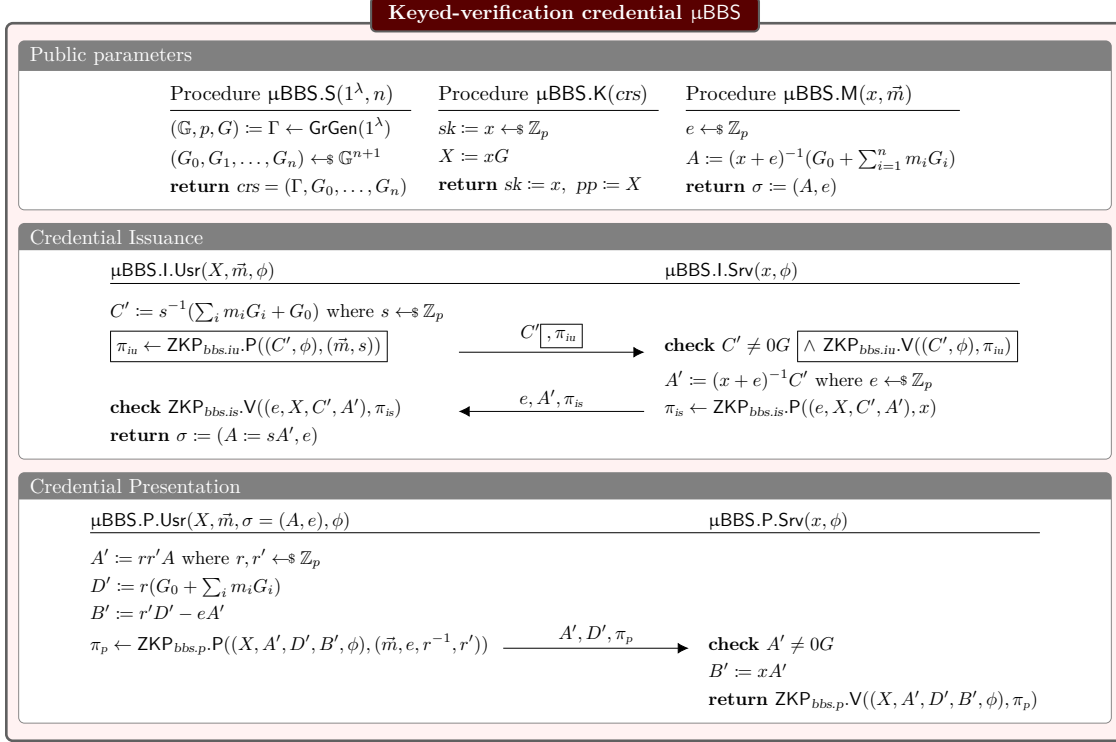


Figure 10: The keyed-verification credential system μ BBS. The variable ϕ denotes the arbitrary predicates that must be enforced on the attributes during issuance or presentation. Boxed, the part that may be removed for anonymous tokens (one-more unforgeability). The zero-knowledge relations associated to the proof system (for user issuance, server issuance, and presentation respectively) are defined in Equations (24) to (26).

6 Boneh–Boyen–Shacham credentials

Anonymous credentials from Boneh–Boyen–Shacham signatures were first introduced as BBS+ within the framework of k -times anonymous authentication primitive [ASM06], and as algebraic MACs by Barki et al. [BBDT17]. We propose a variant in Figure 10 and study it in the algebraic group model.

6.1 Protocol description

Message authentication code. A μ BBS signature is similar to BBS signatures: a MAC (A, e) for a message $\vec{m}$ is

$$(A = (x + e)^{-1}(G_0 + \sum_{i=1}^n m_i G_i), e \leftarrow \mathbb{Z}_p) .$$

Verification consists in checking $(x + e)A = G_0s + \sum_{i=1}^n m_i G_i$. This is identical to the MAC presented by Barki et al. [BBDT17] when setting $s = 0$. For the purpose of the proof, it is actually not needed that e is randomly chosen and the sole requirement for it is that they are different for different MACs. This property can be formalized with a stateful procedure EGen that takes as input some internal state, the input message $\vec{m}$, and outputs an e -value for $\vec{m}$. Some examples are:

- The issuer picks e uniformly at random from $\mathbb{Z}_p$ for each signature (as displayed in Figure 10). In this case, **EGen** does not require a state nor a message. In the case where $p > 2^{2\lambda}$, **EGen** may just sample a uniformly random string of length 2λ and read it as an integer mod p .
- The issuer can hash the message into $e = H_s(\vec{m})$. In this case, **EGen** does not require a state.
- The issuer can be stateful and set e to be a counter. In this case, **EGen** does not require a message, but it is stateful; the counter concretely may be much smaller than 2λ bits – e.g., 64 bits would suffice, and the probability that any two e -values are equal is zero.

We denote the collision advantage for this procedure as $\delta(\lambda, q)$, that is, the probability that a q -query bound adversary A with oracle access to $H_e : \mathbb{M}^n \rightarrow \{0, 1\}^\lambda$ makes two different queries that collide under H_e . In [TZ23], this quantity is indicated as $\delta_{eS, eG}$.

A special property of BBS signatures is that it is possible to extend the credential system for more attributes without requiring a new key generation or re-issuance of a credential, and that the public parameters pp are a single element. We prove the following theorem:

Theorem 6.1. *In the algebraic group model, μBBS is an algebraic MAC for $n = \text{poly}(\lambda)$ attributes in $\mathbb{Z}_p$ with advantage:*

$$\text{Adv}_{\mu\text{BBS}}^{\text{ufcmva}}(\lambda, n) = \text{Adv}_{\text{GrGen}}^{(q+2)\text{-dl}}(\lambda) + \text{Adv}_{\text{GrGen}}^{\text{dl}}(\lambda) + \frac{3}{p} + \delta(\lambda, q) ,$$

where q is the number of signing queries made by the adversary.

In particular, when e is sampled as a uniformly-random 2λ -bit string, we have $\delta(\lambda, q) \leq q^2/2^{2\lambda}$. The major difference with the previous proof of Tessaro and Zhu [TZ23] is that we need to provide the adversary additional oracles: one for (keyed-)verification, and a helper oracle that will be required for proving the credential system.

Blind issuance. The user commits to the message as $C' = s^{-1}(\sum_i m_i G_i + G_0)$ where $s \leftarrow \mathbb{Z}_p$, and proves a representation of C' in bases $(G_0, G_1, \dots, G_n)$ via $\text{ZKP}_{bbs.iu}$, a simulation-extractable proof system for the relation

$$R_{bbs.iu} := \{((C', \phi), (\vec{m}, s)) : G_0 = sC' - \sum_i m_i G_i \wedge \phi(\vec{m}) = 1\} \quad (24)$$

The issuer then computes $A' = (x + e)^{-1}C'$ and proves knowledge of the discrete logarithm of (e, X, C', A') via $\text{ZKP}_{bbs.is}$, a simulation-extractable proof system for the relation:

$$R_{bbs.is} := \{((e, X, A', C'), x) : X = xG \wedge C' - eA' = xA'\} . \quad (25)$$

Remark 6.2. *This is different from the original approach of Barki et al. [BBDT17] where the user commits to the message as $C' = \sum_i m_i G_i + sG$ and proves knowledge of a representation of C' under $(G_0, G_1, \dots, G_n)$.*

Presentation. The core idea for presentation is to note that:

$$xA - G_0 = \sum_{i=1}^n m_i G_i - eA$$

where the left-hand side can be computed by the issuer. So, all the user has to do is provide a proof of knowledge of the discrete logarithm of $xA - G_0$ in basis $G_1, \dots, G_n, A$.

To provide anonymity across presentation, the user must also “blind” A . More formally π_p is a zero-knowledge proof of knowledge for:

$$\mathbf{R}_{bbs.p} := \left\{ ((X, A', D', B', \phi), (\vec{m}, e, r'', r')) : \begin{array}{l} B' = r' D' - e A' \wedge G_0 = r'' D' - \sum_i m_i G_i \wedge \\ \phi(\vec{m}) = 1 \end{array} \right\} \quad (26)$$

as shown by Tessaro and Zhu [TZ23].

Remark 6.3 (Difference with [BBDT17]). In Barki et al., the user algorithm takes as input the private attributes $\vec{m}$ along with the tuple (A, e, s, C) , where $(A, e) \in \mathbb{G} \times \mathbb{Z}_p$ is the standard BBS signature (denoted “ (A, r) ”), $s \in \mathbb{Z}_p$ is the nonce attribute (denoted “ s_u ”), and $C \in \mathbb{G}$ is the commitment to the message (denoted “ $\tilde{C}_m$ ”). To present the credential, the user sends $(B_0 := lA, C_0 := lC - eB_0, E := l^{-1}C_0 + tF)$ for $l, t \leftarrow \mathbb{Z}_p$. The guarantees are the same, the proof is only slightly less efficient.

Remark 6.4 (Differences with [BBS04]). The original BBS paper introduces a protocol for proving knowledge of a signature in the scope of group signatures. The main difference is in that the honest-verifier zero-knowledge of their protocol is only computational while here it is perfect.

6.2 Theorems

We formally state (below) the core theorems of μBBS , starting from the underlying MAC, then prove them.

Algebraic MAC. The core theorem resulting from Theorem 6.8 and Theorem 6.9

Theorem 6.5. μBBS is an algebraic MAC for $n = \text{poly}(\lambda)$ attributes in $\mathbb{Z}_p$ with advantage:

$$\text{Adv}_{\mu\text{BBS}}^{\text{ufcmva}}(\lambda, n) \leq \text{Adv}_{\text{GrGen}}^{(q+2)\text{-dl}}(\lambda) + \text{Adv}_{\text{GrGen}}^{\text{dl}}(\lambda) + \frac{3}{p} + \delta(\lambda, q) .$$

Keyed-verification credential. Let $\mathbf{R}_{bbs} := \mathbf{R}_{bbs.iu} \cup \mathbf{R}_{bbs.is} \cup \mathbf{R}_{bbs.p}$.

Theorem 6.6. If ZKP is a proof system for $\mathbf{R} \supseteq \mathbf{R}_{bbs}$, then μBBS is a keyed-verification extractable credential for $n = \text{poly}(\lambda)$ attributes with anonymity advantage:

$$\text{Adv}_{\mu\text{BBS}}^{\text{anon}}(\lambda, n) \leq \text{Adv}_{\text{ZKP}_{bbs.iu}}^{\text{zk}}(\lambda) + \text{Adv}_{\text{ZKP}_{bbs.p}}^{\text{zk}}(\lambda) + \text{Adv}_{\text{ZKP}_{bbs.is}}^{\text{ksnd}}(\lambda) + \text{Adv}_{\text{GrGen}}^{\text{dl}}(\lambda) + \frac{1}{p} ,$$

and extractability advantage:

$$\text{Adv}_{\mu\text{BBS}}^{\text{ext}}(\lambda, n) \leq \text{Adv}_{\mu\text{BBS}}^{\text{ufcmva}}(\lambda, n) + \text{Adv}_{\text{ZKP}_{bbs.is}}^{\text{zk}}(\lambda) + \text{Adv}_{\text{ZKP}_{bbs.iu}}^{\text{ksnd}}(\lambda) + \text{Adv}_{\text{ZKP}_{bbs.p}}^{\text{ksnd}}(\lambda) .$$

In particular, $\mu\text{BBS}[\text{ZKP} = \Sigma]$ has everlasting anonymity (Theorem 4.4).

Anonymous token. Let μBBS_{AT} denote the variant of μBBS in Figure 10 where the issuance algorithm does not return the proof π_{iu} (that is, the boxed areas are removed).

Theorem 6.7. If ZKP is a proof system for the relation $\mathbf{R} \supseteq \mathbf{R}_{bbs.is} \cup \mathbf{R}_{bbs.p}$, then μBBS_{AT} is an anonymous token system for $n = \text{poly}(\lambda)$ attributes with anonymity advantage:

$$\text{Adv}_{\mu\text{BBS}_{AT}}^{\text{anon}}(\lambda, n) \leq \text{Adv}_{\text{ZKP}_{bbs.iu}}^{\text{zk}}(\lambda) + \text{Adv}_{\text{ZKP}_{bbs.p}}^{\text{zk}}(\lambda) + \text{Adv}_{\text{ZKP}_{bbs.is}}^{\text{ksnd}}(\lambda) + \text{Adv}_{\text{GrGen}}^{\text{dl}}(\lambda) + \frac{1}{p} ,$$

and (in the algebraic group model) one-more unforgeability advantage:

$$\text{Adv}_{\mu\text{BBS}_{AT}}^{\text{omuf}}(\lambda, n) \leq \text{Adv}_{\text{GrGen}}^{(q+2)\text{-dl}}(\lambda) + \text{Adv}_{\text{GrGen}}^{\text{dl}}(\lambda) + \frac{3}{p} + \delta(\lambda, q) .$$

6.3 Algebraic MAC

We prove that μBBS is a MAC for $n = 1$ attributes and then reduce the case of $n = \text{poly}(\lambda)$ attributes to it. The proof will actually be about a stronger game where the adversary may also query an oracle $\text{DDH}(A, B)$ to check if $B = xA$.

Lemma 6.8. *In the algebraic group model, μBBS is an algebraic MAC for $n = 1$ attributes in $\mathbb{Z}_p$ with advantage:*

$$\text{Adv}_{\mu\text{BBS}}^{\text{ufcmva}}(\lambda, 1) \leq \text{Adv}_{\text{GrGen}}^{(q+2)\text{-dl}}(\lambda) + \frac{1}{p} + \delta(\lambda, q) ,$$

where q is the number of signing queries made by the adversary.

Proof. An algebraic adversary A for the unforgeability game receives as input the public parameters $((\Gamma, G_0, G_1, \dots, G_n), X)$ and during its execution can query an oracle $\text{SIGN}(\vec{m})$ that returns a MAC (A, e) valid on $\vec{m}$ and a verification oracle $\text{VERIFY}(\vec{m}, (A, e))$ that returns 1 if the MAC is valid and 0 otherwise. During its execution, it makes at most q queries to the oracle SIGN . The output forgery is a message m^* not previously queried, a MAC (A^*, e^*) accompanied by an algebraic representation $\vec{\alpha}$ such that

$$A^* = \alpha_{g,0}G_0 + \alpha_{g,1}G_1 + \alpha_g G + \alpha_x X + \sum_{j=1}^q \alpha_{a,j} A_j , \quad (27)$$

where q is the number of MACs produced so far. Form the verification equation, we have that the equality

$$A^* = \frac{\mathbf{g}_0 + m^* \mathbf{g}_1}{\mathbf{x} + e^*} G \quad (28)$$

holds when $\mathbf{g}_0, \mathbf{g}_1, \mathbf{x}$ are the discrete logarithms of the respective group elements G_0, G_1, X base G . Putting together Equations (27) and (28), we obtain the rational function in $\mathbf{g}_0, \mathbf{g}_1, \mathbf{x}$ over $\mathbb{Z}_p$

$$\frac{\mathbf{g}_0 + m^* \mathbf{g}_1}{\mathbf{x} + e^*} = \alpha_g + \alpha_{g,0} \mathbf{g}_0 + \alpha_{g,1} \mathbf{g}_1 + \alpha_x \mathbf{x} + \sum_{j=1}^q \alpha_{a,j} \frac{\mathbf{g}_0 + m_j \mathbf{g}_1}{\mathbf{x} + e_j}$$

we have two possibilities: $e^* \in \{e_1, \dots, e_q\}$ or $e^* \notin \{e_1, \dots, e_q\}$.

In the first case, let $f(\mathbf{x}) = \prod_j (\mathbf{x} + e_j)$ and $f_k(\mathbf{x}) = \prod_{j \neq k} (\mathbf{x} + e_j) = f(\mathbf{x}) / (\mathbf{x} + e_k)$. Let the e_i 's be all different (this will always happen except with probability $\delta(\lambda, q)$). Without loss of generality, let $e^* = e_1$. We have:

$$((1 - \alpha_{a,1}) \mathbf{g}_0 + (m^* - \alpha_{a,1} m_1) \mathbf{g}_1) f_1(\mathbf{x}) = (\alpha_g + \alpha_{g,0} \mathbf{g}_0 + \alpha_{g,1} \mathbf{g}_1 + \alpha_x \mathbf{x}) f(\mathbf{x}) + \sum_{j=2}^q \alpha_{a,j} (\mathbf{g}_0 + m_j \mathbf{g}_1) f_j(\mathbf{x})$$

and since $(\mathbf{x} + e_1)$ divides the right-hand side but not the left-hand side, then $\alpha_{a,1} = 1$ and $m_1 = m^*$ which leads to a contradiction.

If $e^* \notin \{e_1, \dots, e_q\}$, then let for simplicity $e_{q+1} = e^*$ and let $f^* = \prod_j^{q+1} (\mathbf{x} + e_j)$ and, similarly to before, $f_k^* = f(\mathbf{x}) / (\mathbf{x} + e_k)$. Then we have

$$(\mathbf{g}_0 + m^* \mathbf{g}_1) f_{q+1}^*(\mathbf{x}) = (\alpha_g + \alpha_{g,0} \mathbf{g}_0 + \alpha_{g,1} \mathbf{g}_1 + \alpha_x \mathbf{x}) f^*(\mathbf{x}) + \sum_{j=1}^q \alpha_{a,j} (\mathbf{g}_0 + m_j \mathbf{g}_1) f_j^*(\mathbf{x})$$

Similarly to the previous case, we have that the right-hand side is divisible by $(\mathbf{x} + e_{q+1})$, but the left-hand side is not, leading to a contradiction. This implies that, in the above polynomial equation, at least two monomials are different.

We build a reduction to $(q+2)$ -DL. The reduction $\mathbf{B}$ receives as input $(\Gamma = (\mathbb{G}, p, G), X^{(1)}, X^{(2)}, \dots, X^{(q+1)}, x^{(q+2)})$. Let $X^{(0)} := G$. $\mathbf{B}$ samples $e_1, \dots, e_q$ from EGen . (In the figure, $e_i \leftarrow \mathbb{Z}_p$.) In the fortunate case where $\log_G X \in \{0, 1, -e_1, \dots, -e_q\}$, $\mathbf{B}$ stops and returns the discrete logarithm found. Let $f = \prod_{j=1}^q (\mathbf{x} + e_j)$ and $f_k = f(\mathbf{x})/(\mathbf{x} + e_k)$ for $k \in [q]$. Note that, f and f_k can be evaluated the group $\mathbb{G}$ using the “powers of X ” (the $X^{(j)}$ ’s) given as input. Sample $a_0, b_0, a_1, b_1 \leftarrow \mathbb{Z}_p$ and let $G' := f(X)$, $X := X^{(1)}$, $G_0 := ((a_0 + \mathbf{x}b_0)f)(X)$ and $G_1 := ((a_1 + \mathbf{x}b_1)f)(X)$. (By $f(X)$ we mean $\sum_i f_i X^{(i)}$.) The degree of both polynomials is at most $q + 1$. Invoke $\mathbf{A}$ with public parameters $((\Gamma, G_0, G_1), X)$. To respond the j -th query to the signing oracle, denoted $\text{SIGN}(m_j)$, compute

$$A_j = (a_0 + m_j a_1) \sum_{i=0}^q f_{j,i} X^{(i)} + (b_0 + m_j b_1) \sum_{i=0}^q f_{j,i} X^{(i+1)}$$

and return (A_j, e_j) . For any verification query $\text{VERIFY}(m, (A, e, \vec{\alpha}))$, where $\vec{\alpha}$ denotes the algebraic representation of A (similar to Equation (27)) check the verification equation over the algebraic representation. To do so, consider the following multivariate polynomial in $\mathbb{Z}_p[\mathbf{g}_0, \mathbf{g}_1, \mathbf{x}]$

$$\varphi_{m,e}(\mathbf{g}_0, \mathbf{g}_1, \mathbf{x}) := (\mathbf{g}_0 + m\mathbf{g}_1)f(\mathbf{x}) - (\mathbf{x} + e) \left(\alpha_g f(\mathbf{x}) + \alpha_{g,0}\mathbf{g}_0 + \alpha_{g,1}\mathbf{g}_1 - \alpha_x \mathbf{x} + \sum_j \alpha_{a,j}(\mathbf{g}_0 + \mathbf{g}_1 m_j) f_j(\mathbf{x}) \right)$$

The polynomial $\varphi_{m,e}(a_0 + \mathbf{x}b_0, a_1 + \mathbf{x}b_1, \mathbf{x})$ can be tested for zero when evaluated in $\log_G X$ using the inputs $G, X^{(1)}, \dots, X^{(q+2)}$ since the resulting polynomial is of degree at most $q + 2$. For every DDH oracle (A, B) , consider the algebraic representation of A , which is similar to Equation (27). Replacing each group element with its representation in the DL challenge $(G, X^{(1)}, X^{(2)}, \dots, X^{(q+1)})$ we obtain a vector $\vec{\alpha}'$ such that $A = \alpha'_0 G + \sum_{j=1}^{q+1} \alpha'_j X^{(j)}$. Return 1 if $B = \sum_{j=1}^{q+2} \alpha'_j X^{(j)}$. At the end the adversary returns a forgery $(m^*, (A^*, e^*))$.

From the argument at the beginning of the proof, we have $\varphi_{m^*, e^*}(\mathbf{g}_0, \mathbf{g}_1, \mathbf{x}) \neq 0$. Consider then the partial evaluation $\phi(\mathbf{x}) := \varphi((a_0 + b_0 \mathbf{x}), (a_1 + b_1 \mathbf{x}), \mathbf{x})$ and note that the resulting polynomial is of degree at most $q + 2$ and is with overwhelming probability non-trivial too since b_0, b_1 are uniformly random and perfectly hidden in G_0, G_1 by a_0, a_1 . One of the root of this polynomial must be the discrete logarithm of X . The reduction $\mathbf{B}$ uses Berlekamp’s algorithm to factor ϕ and recovers the discrete logarithm of X . $\square$

Lemma 6.9. μBBS is an algebraic MAC for $n = \text{poly}(\lambda)$ attributes in $\mathbb{Z}_p$ with advantage:

$$\text{Adv}_{\mu\text{BBS}}^{\text{ufcmva}}(\lambda, n) = \text{Adv}_{\mu\text{BBS}}^{\text{ufcmva}}(\lambda, 1) + \text{Adv}_{\text{GrGen}}^{\text{dl}}(\lambda) + \frac{2}{p}.$$

Proof. Let $\mathbf{A}$ be a p.p.t. adversary for unforgeability. It receives as input the public parameters $pp = (\Gamma, (G_0, \dots, G_n), X)$, during its executions it queries SIGN with messages $\vec{m}_j$ for $j \in [q]$ (q denoting the number of queries of $\mathbf{A}$); finally, it returns a forgery satisfying one of the following:

- (i) the output $(\vec{m}^*, (A^*, e^*))$ is a valid forgery and $\sum_i m_i^* G_i = \sum_i m_{j,i} G_i$ for some $j \in [q]$
- (ii) the output $(\vec{m}^*, (A^*, e^*))$ is a valid and $\sum_i m_i^* G_i \neq \sum_i m_{j,i} G_i$ for all $j \in [q]$

We claim that (i) happens at most with negligible probability. To do so, we build an adversary B that solves DL every time A wins the game with a forgery satisfying $\sum_i (m_i^* - m_{j,i})G_i = 0G$. Let $B(\Gamma, Y)$ be an adversary for $DL_{GrGen}(\lambda)$. B samples at random $a_0, a_1, \dots, a_n, b_1, \dots, b_n, x \leftarrow \mathbb{Z}_p$ and sets $G_0 := a_0$, $G_i = a_i G + b_i Y$ for all $i \in [n]$, and finally sets $X := xG$ for a random $x \leftarrow \mathbb{Z}_p$. It invokes $A((\Gamma, G_0, \vec{G}), X)$. Queries to the oracles $SIGN(\vec{m}_j)$ are answered with (A_j, e_j) , where $e_j \leftarrow \mathbb{Z}_p$ and $A_j = (x + e_j)^{-1}(G_0 + \sum_i m_i G_i)$; queries $VERIFY(\vec{m}, (A, e))$ are answered testing the equality $(x + e)A = G_0 + \sum_i m_i G_i$. To any $DDH(A, B)$ query, it responds 1 if $xA = B$. Given as output a forgery satisfying Item (i), we have that there exists a $j \in [q]$ such that:

$$\sum_{i=1}^n (m_i^* - m_{j,i})a_i G = \sum_{i=1}^n (m_i^* - m_{j,i})b_i Y$$

and the above equation is non-trivial with overwhelming probability since $\vec{m}^* \neq \vec{m}_j$ (and thus $\exists i. m_i^* \neq m_{j,i}$) and the b_i 's are uniformly distributed and perfectly hidden. Therefore, with overwhelming probability B can compute $\log_G Y = (\sum_i (m_i^* - m_{j,i})b_i)^{-1}(\sum_i (m_i^* - m_{j,i})a_i)$.

The claim Item (ii) reduces to unforgeability of μBBS for $n = 1$. Let B be an unforgeability adversary receiving as input Γ, G_0, G_1, X . Let $G_i = a_i G_1$ for $i \in [2, n]$ with $a_i \leftarrow \mathbb{Z}_p$. The two are identically distributed if $G_1 \neq 0G$. Invoke $A((\Gamma, G_0, G_1, \dots, G_n), X)$. For every signing query for a message $\vec{m}_j$, respond with $SIGN(\sum_i a_i m_{j,i})$. For every verification query for a message $\vec{m}$, respond with $VERIFY(\sum_i a_i m_i)$. All DDH queries are sent to the challenger as-is. At the end, we obtain an output forgery satisfying Item (ii) and therefore $(\sum_i a_i m_i^*, (A^*, e^*))$ is a valid forgery also for μBBS for $n = 1$. $\square$

6.4 Anonymity

Theorem 6.10. *Let ZKP be a proof system for the relation $R \supseteq R_{bbs}$, then μBBS is anonymous for $n = \text{poly}(\lambda)$ attributes over $\mathbb{Z}_p$ with advantage:*

$$\text{Adv}_{\mu BBS, A, D}^{\text{anon}}(\lambda, n) \leq \text{Adv}_{ZKP_{bbs, iu}, A'}^{\text{zk}}(\lambda) + \text{Adv}_{ZKP_{bbs, is}, A''}^{\text{ksnd}}(\lambda) + \text{Adv}_{ZKP_{bbs, p}, D'}^{\text{zk}}(\lambda) + \text{Adv}_{GrGen, A'''}^{\text{dl}}(\lambda) + \frac{1}{p},$$

where A, D are adversaries in the anonymity game and A', A'', A''', D are described in the proof.

Proof. We define the simulator Sim as follows.

- Let Sim.I sample a uniformly-random group element $C' \leftarrow \mathbb{G}$ and invoke $ZKP_{bbs, iu}.\text{Sim}$ on the statement (C', ϕ) obtaining a simulated proof π_{iu} . Upon receiving the server response (e, A', π_{is}) , verify the zero-knowledge proof via $ZKP.V((e, X, A', C'), \pi_{is})$ and (if the output is 1) proceed extracting a witness x from π_{is} via the extractor for $ZKP_{bbs, is}$ for the statement (e, X, A', C') . Store the simulator state $st_{\text{Sim}} = (x)$.
- The presentation simulator $\text{Sim.P}(x, \phi)$ samples random A', D' and simulates the zero-knowledge proof π_p for the statement (X, A', D', B', ϕ) where $B' = xA'$. It returns (A', D', π_p) .

We show that the real and simulated distributions are indistinguishable via a hybrid argument.

H_0 This first hybrid is the honest interaction, where issuance and presentation are done honestly using the input message $\vec{m}$.

H₁ Instead of generating π_{iu} using the witness, we do so via the zero-knowledge simulator of $\text{ZKP}_{bbs.iu}$ with input (C', ϕ) . If A's output is noticeably different after this hybrid change, then it is possible to build an adversary A' that is able to distinguish real from simulated proofs for $\text{ZKP}_{bbs.iu}$. Therefore:

$$\text{Adv}_{A,D}^{H_1}(\lambda, n) \geq \text{Adv}_{A,D}^{H_0}(\lambda, n) - \text{Adv}_{\text{ZKP}_{bbs.iu}, A'}^{\text{zk}}(\lambda) .$$

H₂ Instead of generating π_p using the attributes $\vec{m}$ and MAC σ , use the zero-knowledge simulator of $\text{ZKP}_{bbs.p}$ with input (X, A', D', B', ϕ) . Similarly to the previous hybrid, there exists an adversary D' that wins every time D's output is noticeably different. Therefore:

$$\text{Adv}_{A,D}^{H_2}(\lambda, n) \geq \text{Adv}_{A,D}^{H_1}(\lambda, n) - \text{Adv}_{\text{ZKP}_{bbs.p}, A'}^{\text{zk}}(\lambda) .$$

H₃ Extract from π_{is} a witness for the statement (e, X, A', C') and check its validity. The extractor will produce a witness x satisfying:

$$x \begin{bmatrix} G \\ A' \end{bmatrix} = \begin{bmatrix} X \\ C' - eA' \end{bmatrix} ,$$

except with probability $\text{Adv}_{\text{ZKP}_{bbs.is}, A''}^{\text{ksnd}}(\lambda)$ where A'' is the adversary in the knowledge soundness game that runs A internally and returns the proof π_{is} for which extraction fails. In other words:

$$\text{Adv}_{A,D}^{H_1}(\lambda, n) \geq \text{Adv}_{A,D}^{H_3}(\lambda, n) - \text{Adv}_{\text{ZKP}_{bbs.is}, A''}^{\text{ksnd}}(\lambda) .$$

H₄ If $G_0 + \sum_i m_i G_i = 0G$, abort.

In the real distribution, if $G_0 + \sum_i m_i G_i = 0G$ the user will always send $C' = 0G$, whereas in the simulated case the simulator aborts. The above bad event happens only with advantage $\text{Adv}_{\text{GrGen}}^{\text{dl}}(\lambda)$. Let (Γ, Y) be a DL challenge and A be an adversary for the anonymity game. The reduction A''' invokes A with inputs

$$pp := (\Gamma, X := xG, G_0 = a_0G + b_0Y, \dots, G_n := a_nG + b_nY)$$

where $x, a_0, \dots, a_n, b_0, \dots, b_n \leftarrow \mathbb{Z}_p$. At issuance time the adversary makes queries a message $\vec{m}$ and A''' checks if $G_0 + \sum_i m_i G_i = 0G$. If so, then we have $(a_0 + \sum_i m_i a_i)G = (b_0 + \sum_i m_i b_i)Y$ and the equation is non-trivial with overwhelming probability $1/p$. The reduction A''' can thus recover the discrete logarithm of Y . Therefore:

$$\text{Adv}_{A,D}^{H_3}(\lambda, n) \geq \text{Adv}_{A,D}^{H_4}(\lambda, n) - \text{Adv}_{\text{GrGen}, A'''}^{\text{dl}}(\lambda) .$$

H₅ Replace C' with a uniformly-distributed group element $C' \leftarrow \mathbb{G}$. The two distributions are perfectly indistinguishable, that is:

$$\text{Adv}_{A,D}^{H_4}(\lambda, n) = \text{Adv}_{A,D}^{H_5}(\lambda, n) .$$

H₆ Finally, we replace the responses of the presentation oracle. Sample $A', D' \leftarrow \mathbb{G}$ and compute B' differently: instead of setting $B' := rr'((G_0 + \sum_i m_i G_i) - eA)$ with $r, r' \leftarrow \mathbb{Z}_p$ and (A, e) the MAC obtained from the server, we set $B' := xA'$. The distribution of the two is exactly

the same. In fact, fixed A', D' , for any message $\vec{m} \in \mathbb{Z}_p^n$ and $e \in \mathbb{Z}_p$ there exist $(\rho', \rho'') \in \mathbb{Z}_p^2$ such that:

$$\begin{aligned} B' &= xA' = \rho' D' - eA' , \\ \rho'' D' &= \sum_i m_i G_i + G_0 , \end{aligned}$$

if $D' \neq 0G$. Thus:

$$\text{Adv}_{\mathbf{A}, \mathbf{D}}^{\text{H}_5}(\lambda, n) = \text{Adv}_{\mathbf{A}, \mathbf{D}}^{\text{H}_6}(\lambda, n) + \frac{1}{p} .$$

The last hybrid is running the simulator code. By the difference lemma, the result follows. $\square$

6.5 Extractability

Theorem 6.11. *If ZKP is a proof system for the relation $R \supseteq R_{\text{bbs}}$, then μBBS for $n = \text{poly}(\lambda)$ is extractable with advantage:*

$$\text{Adv}_{\mu\text{BBS}}^{\text{ext}}(\lambda, n) \leq \text{Adv}_{\mu\text{BBS}}^{\text{ufcmva}}(\lambda, n) + \text{Adv}_{\text{ZKP}_{\text{bbs.is}}}^{\text{zk}}(\lambda) + \text{Adv}_{\text{ZKP}_{\text{bbs.iu}}}^{\text{ksnd}}(\lambda) + \text{Adv}_{\text{ZKP}_{\text{bbs.p}}}^{\text{ksnd}}(\lambda) .$$

Roughly speaking, the extractor matches the zero-knowledge extractor and a reduction is made to unforgeability of the underlying MAC. However, the term “ B' ” is computed during presentation verification using the MAC key x , which is not available to the reduction to unforgeability. To circumvent this, we are going to assume that the proof itself can provide a candidate instance “ B' ”, and that the instance can be check for correctness via the oracle DDH. In practice, this requirement is trivially satisfied for Schnorr proofs, where it is possible to recover the statement from the actual proof by looking at the trace of random oracle queries.

Proof. We define the extractors for the credential system as follows.

- **Ext.I** receives as input $(x, \phi, C', \pi_{\text{in}})$ and recovers a witness $(\vec{m}, s)$ for the instance (C', ϕ) such that $\phi(\vec{m}) = 1$ and

$$\begin{bmatrix} C' & -G_1 & \cdots & -G_n \end{bmatrix} \begin{bmatrix} s \\ m_1 \\ \vdots \\ m_n \end{bmatrix} = G_0 . \quad (29)$$

If extraction fails the extractor aborts and the adversary wins.

- **Ext.P** receives as input (x, ϕ) and a prover message (A', D', π_p, B') . It checks if B' is well-formed via DDH and, if that is the case, it internally runs $\text{ZKP}_{\text{bbs.p}}.\text{Ext}$ on input (X, A', D', B', ϕ) where $X = xG$ and obtain a witness $(\vec{m}, e, r'', r')$ such that $\phi(\vec{m}) = 1$ and

$$\begin{bmatrix} -G_1 & -G_2 & \cdots & -G_n & & D' & -A' \end{bmatrix} \begin{bmatrix} m_1 \\ \vdots \\ m_n \\ e \\ r'' \\ r' \end{bmatrix} = \begin{bmatrix} G_0 \\ B' \end{bmatrix} . \quad (30)$$

If the recovered witness does not satisfy the above equation, return $\perp$. A wins if one of the following happens:

- (i) The adversary returns $(\phi, \mu = (C', \pi_{iu}))$ such that $\text{ZKP}_{bbs.iu}$ verifies π_{iu} for the statement (C', ϕ) but extraction failed.
- (ii) The adversary returns $(\phi^*, \rho^* = (A', D', \pi_p, B'))$ such that extraction fails for the instance (A', D', B', ϕ^*) .
- (iii) The presentation message returned by the adversary $\rho^* = (A', D', \pi_p, B')$ satisfies $\mu\text{BBS.P.Srv}(x, \phi^*, \rho^*)$ and the message $\vec{m}^*$ extracted from π_p is such that $\vec{m} \notin Qrs$.

Items (i) and (ii) can be reduced to knowledge soundness of the proof system. We are left with Item (iii), which reduces to unforgeability of μBBS . We do so by building an adversary B for unforgeability that internally uses the adversary A.

The reduction B receives as input the public parameters $pp = (\Gamma, G_0, G_1, \dots, G_n, X)$ and internally invokes the adversary A. For every $\text{ISSUE}(\phi, (C', \pi_{iu}))$ query, it runs the extractor and obtains $(\vec{m}, s)$ satisfying Equation (29). It queries $\text{SIGN}(\vec{m})$ obtaining a MAC (A, e) . Returns $(A' := s^{-1}A, e)$ (or $\perp$ if $s = 0$). For every $\text{PRESENT}(\phi^*, (A', D', \pi_p))$ query, B runs the extractor to obtain a candidate instance B'' – which is checked via a query $\text{DDH}(A', B')$ to ensure $B'' = xA$ – and a witness $(\vec{m}, e, r'', r')$ satisfying Equation (30). It queries the verification oracle on the message $\vec{m}$ and MAC $(A = r''/r' A', e)$. It proceeds similarly for the returned message, outputting a forgery $\vec{m}^*$ (the message extracted from ρ^*) and $(A = r''/r' A', e)$.

The output of SIGN follow exactly the same distribution as in the original game, since from Equation (29) (and commutativity in $\mathbb{Z}_p$)

$$\frac{1}{x+e}C' = s^{-1} \left(\frac{1}{x+e}(G_0 + \sum_{i=1}^n m_i G_i) \right)$$

The unfortunate case where $s = 0$ implies (via Equation (29)) that $G_0 + \sum_i m_i G_i = 0$ and therefore $C' = 0$, so we return $\perp$ in both cases. The output of VERIFY is also well-formed since, from π_p :

$$B' = xA' = r'D' - eA' = \frac{r'}{r''}(\sum_i m_i G_i + G_0) - eA' \implies A' = \frac{r'}{r''} \cdot \frac{1}{x+e} \cdot (\sum_i m_i G_i + G_0) .$$

The claim follows. □

6.6 One-more unforgeability analysis

Lemma 6.12. *There exists a reduction for the game $\text{OMUF}_{\mu\text{BBS}_{AT}, A}(\lambda, n)$ to $q\text{-DL}_{\text{GrGen}, A}(\lambda)$ with: q signing queries, $O(q^3)$ group operations, $O(q)$ space.*

We construct an adversary A that, given as input $X \in \mathbb{G}$ and access to a signing oracle $\text{SIGN}(\cdot)$ performing a MAC on the input group element, outputs a $q\text{-DL}$ challenge $(P, xP, \dots, x^q P)$ for some generator $P \in \mathbb{G}$.

Let $A_0 := G$. For each $i \in [q]$, the attacker A queries $\text{SIGN}(A_{i-1})$ obtaining the MACs (A_i, e_i) . We have the invariant

$$\mathbf{x}A_i = A_{i-1} - e_i A_i , \tag{31}$$

for each $i \in [q]$, with $\mathbf{x}$ denoting the unknown. In particular, we have that:

$$A_{q-k} = \prod_{i=k}^q (\mathbf{x} + e_i) A_q = \sum_i \nu_i^{(k)} \mathbf{x}^i A_q$$

where the polynomial coefficients $\nu_i^{(k)}$ can be computed in time $O(q^2)$ naïvely for each $k \in [q]$. In particular, the k -th polynomial is monic, and the leading term can always be isolated by recursively applying Equation (31) to obtain an expression independent of $\mathbf{x}$, that is:

$$\begin{aligned} \mathbf{x}A_q &= A_{q-1} - e_q A_q =: B_1 \\ \mathbf{x}^2 A_q &= A_{q-2} - e_{q-1} A_{q-1} - e_q B_1 =: B_2 \\ &\vdots \end{aligned}$$

(Since $i < q$, then $\mathbf{x}^i A_q$ will stop when replacing the term A_{q-i}). $\mathbf{A}$ returns $(A_q, B_1, B_2, \dots, B_{q-1})$.

Corollary 6.13. *There exists a p.p.t. adversary $\mathbf{A}$ for one-more unforgeability of μBBS_{AT} that can recover the signing key in time $O(\sqrt{p/d} + \sqrt{d})$ and space $\max\{\sqrt{p/d}, \sqrt{d}\}$ where $d \mid p \pm 1$ is the number of signing queries allowed.*

7 Designated-verifier fully-succinct SNARKs without pairings

A polynomial commitment scheme (PCS) is designated-verifier (dv) if only the holder of the *secret* verification key vk can verify the proof, as opposed to standard PCS definitions where vk can be efficiently computed from the proving key pk . The definition below, is an adaptation of [CHMMVW20].

Definition 7.1. A *designated-verifier polynomial commitment scheme* (PCS) over the field family $\mathcal{F} = \{\mathbb{F}_\lambda\}_{\lambda \in \mathbb{N}}$ is a tuple of algorithms $\text{PCS} = (\text{S}, \text{K}, \text{C}, \text{E}, \text{V})$ where

- $\text{crs} \leftarrow \text{PCS.S}(1^\lambda, d)$ given as input the security parameter in unary form and a maximum degree $d > 0$, outputs a common reference string crs and defines the field $\mathbb{F} \in \mathcal{F}$.
- $(pk, vk) \leftarrow \text{PCS.K}(\text{crs}, d)$ outputs a key pair (pk, vk) called respectively proving key and verification key. The proving key allows committing to polynomials over $\mathbb{F}$ up to degree d .
- $(C, r) \leftarrow \text{PCS.C}(pk, f)$ outputs a commitment C to a polynomial $f(\mathbf{x}) \in \mathbb{F}[\mathbf{x}]$ represented via its coefficients f_i 's, together with some opening information r .
When clear from the context, to facilitate exposition, we will write $C \leftarrow \text{PCS.C}(f; r)$, omit pk and using the random coins r .
- $\pi \leftarrow \text{PCS.E}(pk, (f, r), z, y)$ outputs an evaluation proof π , showing that $y = f(z)$.
- $0/1 \leftarrow \text{PCS.V}(vk, C, z, y, \pi)$ outputs 1 if the evaluation proof is correct and 0 otherwise.

A PCS satisfies completeness, extractability, and hiding.

Completeness requires that all honestly-generated proofs verify. More precisely, for any $f \in \mathbb{Z}_p[\mathbf{x}]$ of degree $d > 0$, $z \in \mathbb{Z}_p$ and $y = f(z)$:

$$\Pr \left[\begin{array}{l} \text{PCS.V}(vk, C, z, y, \pi) = 1 : \\ \text{crs} \leftarrow \text{PCS.S}(1^\lambda, d) \\ (pk, vk) \leftarrow \text{PCS.K}(\text{crs}) \\ (C, r) \leftarrow \text{PCS.C}(pk, f) \\ \pi \leftarrow \text{PCS.E}(pk, (f, r), z, y) \end{array} \right] = 1 .$$

Extractability asks that there exists an extractor Ext such that, for any p.p.t. adversary A , B , and query sampler Q ⁶ sharing the same random coins:

$$\Pr \left[\begin{array}{l} \text{PCS.V}(vk, C, z, y, \pi) = 1 \\ \Downarrow \\ \deg(f) \leq d \wedge y = f(z) \end{array} : \begin{array}{l} \text{crs} \leftarrow \text{PCS.S}(1^\lambda, d) \\ (pk, vk) \leftarrow \text{PCS.K}(\text{crs}) \\ \rho \leftarrow_{\$} \{0, 1\}^{A.\text{rl}(\lambda)} \\ C := A(pk; \rho) \\ f := \text{Ext}(pk; \rho) \\ z := Q(pk; \rho) \\ (\pi, y) := B(pk; \rho) \end{array} \right] \leq \text{negl}(\lambda) .$$

⁶A query sampler for an interactive oracle proof is an efficient algorithm that, given as input the random coins of the verifier, returns the point at which the oracle messages of the prover is queried. See [CHMMVW20] for a more formal definition.

Hiding asks that for any $d > 0$, adversaries A , D , and any $crs \in [\text{PCS.S}(1^\lambda, d)]$, $(pk, vk) \in [\text{PCS.K}(crs)]$ following experiments are indistinguishable:

$$\left| \Pr \left[b' = 1 : \begin{array}{l} f \leftarrow A(pk, vk) \\ (C, r) \leftarrow \text{PCS.C}(pk, f) \\ b' \leftarrow D^{\text{EVAL}}(pk, C) \end{array} \right] - \Pr \left[b' = 1 : \begin{array}{l} f \leftarrow A(pk, vk) \\ C \leftarrow \text{Sim.C}(vk, \deg(f)) \\ b' \leftarrow D^{\text{EVAL}}(pk, C) \end{array} \right] \right| \leq \text{negl}(\lambda) ,$$

where $\text{EVAL}(z)$ sets $y := f(z)$ and, if the condition holds, in the real game returns $\text{PCS.E}(pk, (f, r), z, y)$ and in the simulated game $\text{Sim.E}(vk, z, y)$.

For the remainder of this work, we will consider homomorphic polynomial commitments, that is, $\text{PCS.C}(f; r) + \text{PCS.C}(g; s) = \text{PCS.C}(f + g; r + s)$, where r, s are the randomness of the commit algorithms. In particular the opening algorithm trivially recomputes the commitment with the randomness provided and checks that it matches the commitment given as input.

Malicious designated-verifier zero-knowledge [QRW19]. In practical applications, we would like the verifier to publish the proving key pk and privately store the secret key sk without relying on a trusted third party. In the context of keyed-verification credential systems, in fact, the verifier is also the entity publishing the public parameters. While the definition of hiding above holds for every keypair (pk, vk) , one must make sure that the keypair is in the range of the setup algorithm to guarantee that hiding holds against malicious servers. (The proving key of the next section, for instance, holds a specific structure and proving pk is a correct output of the key generation is not immediate.) Our approach here will be to prove security for a proving key that is within the range of PCS.S , and then expect the verifier publishing the proving key to prove that it is indeed in the range of PCS.S .

Remark 7.2. *The compiler of Chiesa et al. [CHMMVW20] introduces another procedure Trim which, given as input a prover key pk for a PCS of size D , and an integer $d < D$, it returns a new prover key pk' for a PCS of size d . This is trivial to implement for our schemes and is a technicality of the compiler, which requires the prover key to be generated before seeing the actual size of the index. The opening algorithm PCS.O and verification algorithms PCS.V (called *Open* and *Check* in the literature) can also be generalized as in [CHMMVW20; BDFG20] to allow for batch opening and evaluation checking. Given polynomials $f_1(\mathbf{x}), \dots, f_m(\mathbf{x}) \in \mathbb{F}[\mathbf{x}]$ claimed to evaluate in $y_1, \dots, y_m$ in points $z_1, \dots, z_m$, and given a random challenge $r \in \mathbb{F}$ one can consider euclidean division of the batch polynomial $f(\mathbf{x}) := \sum_i r^i f_i(\mathbf{x})$ by $Z(\mathbf{x}) = \prod_i (\mathbf{x} - z_i)$:*

$$f(\mathbf{x}) = Z(\mathbf{x})q(\mathbf{x}) + r(\mathbf{x}) ,$$

where $q(\mathbf{x})$ is the polynomial that will be sent as proof to the verifier, and the polynomial $r(\mathbf{x})$ of degree $m - 1$ is the polynomial defined by interpolation as $r(z_i) = y_i$.

7.1 Designated-verifier Kate–Zaverucha–Goldberg commitments

We describe a variant of KZG meant for designated-verifier proof systems. We assume that the polynomial is given in coefficient form as a vector of field elements.

Procedure $\text{dvKZG.S}(1^\lambda, d)$	Procedure $\text{dvKZG.K}(crs)$	Procedure $\text{dvKZG.C}(pk, f(\mathbf{x}))$
$\Gamma := \text{GrGen}(1^\lambda)$	$\tau \leftarrow \mathbb{Z}_p^\times$	$s \leftarrow \mathbb{Z}_p$
return $crs := (\Gamma, d)$	$T_i := \tau^i G$ for $i \in [0, d+1]$	return $C := \sum_{i=0}^d f_i T_i + sR$
	$\vec{T} := (T_0, \dots, T_d)$	
	$R := \eta G$ where $\eta \leftarrow \mathbb{Z}_p^\times$	
	return $pk := (T_0, \vec{T}, R), vk := (pk, \tau, \eta)$	

The idea for evaluation proofs stems from the observation that for a polynomial $f \in \mathbb{Z}_p[\mathbf{x}]$, the Euclidian division by $(\mathbf{x} - z)$ has remainder $y = f(z)$:

$$f(\mathbf{x}) = (\mathbf{x} - z)q(x) + y$$

The proof consists into exhibiting a quotient polynomial $q(x)$ satisfying the above (and accounting for the additional term $r(z)$ introduced in the commitment).

Procedure $\text{dvKZG.E}(pk, f(\mathbf{x}), s, z, y)$	Procedure $\text{dvKZG.V}(vk, C, z, y, (Q, D))$
$q(\mathbf{x}) := (f(\mathbf{x}) - y)/(\mathbf{x} - z)$	return $(\tau - z)Q + yG + \eta D = C$
$Q := \sum_i q_i T_i + s'R$ where $s' \leftarrow \mathbb{Z}_p$	
$D := \sum_i \delta_i T_i$ where $\delta(\mathbf{x}) := s - s'(\mathbf{x} - z)$	
return $\pi := (Q, D)$	

Knowledge soundness holds in the algebraic group model under the $(q+1)$ -DL assumption, and the proof is the same as [KT24, B.2.2]. We now focus on hiding.

Theorem 7.3. *dvKZG is a perfectly hiding polynomial commitment scheme.*

Proof. The simulator Sim obtains as input $vk = (pk = (T_0, \dots, T_d, R), (\tau, \eta))$ such that:

$$\begin{aligned} \eta, \tau \in \mathbb{Z}_p^\times &\iff T_1, R \neq 0G \wedge \\ &\forall \tau \in [d]: \tau T_{i-1} = T_i \end{aligned} \tag{32}$$

Given vk along with a commitment C and an evaluation proof y , samples uniformly at random $Q \leftarrow \mathbb{G}$ and computes $D := \eta^{-1}(C - (\tau - z)Q - yG)$.

We have thus a distinguishing game where an adversary $\mathbf{A}$ returns a polynomial f and $pk = (T_0, \vec{T}, R), vk = (\tau, \eta)$ are well-formed and satisfying Equation (32). $\mathbf{A}$ has to distinguish between:

- a real interaction, where:

$$C = \sum_i f_i T_i + sR = (f(\tau) + r\eta)G$$

with $s \leftarrow \mathbb{Z}_p$, $\tau := \log_G T_1$ and $\eta := \log_G R$ with $\tau, \eta \neq 0$. $\mathbf{A}$ can query on any point z the evaluation oracle for $\text{EVAL}(z)$ which in turn computes $\text{dvKZG.E}(pk, (f, s), z, f(z))$ returning (Q, D) such that:

$$\begin{aligned} Q &= \sum_i q_i T_i + s'H = (q(\tau) + s'\eta)G \\ D &= (r + s'z)T_0 - s'T_1 = (s + s'z - s'\tau)G \end{aligned} \tag{33}$$

where the right-hand side is given by Equation (32). From correctness, we have:

$$(\tau - z)(q(\tau) + s'\eta) + y + \eta((s + s'z) - s'\tau) = f(\tau) + \eta s$$

- a simulated interaction, where the simulator Sim returns a commitment $C = \alpha G$ with $\alpha \leftarrow \mathbb{Z}_p$ as a commitment, and, for every evaluation query on $z \in \mathbb{Z}_p$ and y , the proof returned is:

$$Q = \beta G$$

$$D = \eta^{-1}(C - (\tau - z)Q - y) = \eta^{-1}(\alpha - (\tau - z)\beta - y)G$$

which satisfies the verification equation since:

$$(\tau - z)\beta + y + (\alpha - (\tau - z)\beta - y) = \alpha$$

(note that, since $R \neq 0G$, η is invertible).

The two distributions are perfectly indistinguishable: both α and $f(\tau) + s\eta$ are uniformly distributed (since $\eta \neq 0$ and s is uniformly distributed), and the same can be argued about β and $q(\tau) + s'\eta$. Finally, D is uniquely determined by τ, η, C, Q, z, y and the verification equation is satisfied in both cases. $\square$

Degree-check for polynomials whose degree may be less than d can be enforced using standard techniques as described in Kohrita and Towa [KT24, p. 5.2], roughly speaking by checking instead the equation:

$$(f - z)\mathbf{x}^{d-\deg(f)} = q\mathbf{x}^{d-\deg(f)}(\mathbf{x} - y) ,$$

by having the prover commit to the polynomial $f(\mathbf{x})\mathbf{x}^{d-\deg(f)}$.

7.2 IOP compiler for designated-verifier polynomial commitments

An indexed relation is a relation $R = \{(i, x), w\}$ consisting of an index i , an instance x , and a witness w . An interactive proof for an indexed relation generally has the prover's first message depend only on the index i and not depending on the prover's randomness. The first-message function is called *indexer*, and is relevant in the context of preprocessing SNARKs [CHMMVW20].

Definition 7.4 (PIOP). A **polynomial interactive oracle proof** for an indexed relation R over a field family $\mathcal{F}$ is a tuple $\text{IOP} = (k, o, d, \text{I}, \text{P}, \text{V})$ where k, o, d are maps $\{0, 1\}^* \rightarrow \mathbb{N}$ and $\text{I}, \text{P}, \text{V}$ are algorithms called respectively *indexer*, *prover*, and *verifier*. The map k specifies the number of interaction rounds, o the number of polynomials in each round, and d the degree bound of these polynomials.

In the 0-th round (offline phase), the indexer IOP.I receives as input a field $\mathbb{F} \in \mathcal{F}$ and an index i for R , and outputs $o(0)$ polynomials $p_1^{(0)}(x), \dots, p_{o(0)}^{(0)} \in \mathbb{F}[x]$ of degree at most $d(|i|, 0, 1), \dots, d(|i|, 0, o(n))$ respectively.⁷

In any other round (online phase), given an instance x and a witness w such that $(i, x, w) \in R$, the prover IOP.P receives as input $(\mathbb{F}, i, x, w)$ and IOP.V receives as input oracle access to the polynomials output by $\text{IOP.I}(\mathbb{F}, i)$. The prover IOP.P and the verifier IOP.V interact over $k(|i|)$ rounds. For $j \in [k(i)]$, in the j -th round of interaction the verifier IOP.V sends messages $m_j \in \mathbb{F}$ to the verifier IOP.P ; the prover then replies with $o(j)$ oracle polynomials $p_1^{(j)}, \dots, p_{o(j)-1}^{(j)} \in \mathbb{F}[x]$. The verifier may query any of the polynomial it has received any number of times. A query consists

⁷The 0-th round does not depend on any particular instance or witness, and merely considers the task of encoding a given index i .

of a point $z \in \mathbb{F}$ and the prover replies with the evaluation of the polynomial at that point. After the interaction the verifier accepts or rejects. The function d determines which prover to consider for the completeness and soundness properties of the proof system. In more detail, a prover P is admissible if for every $j \in [k(|i|)]$ and $i \in [o(j)]$ the degree of $p_i^{(j)}$ is at most $d(|i|, j, i)$.

Construction. The compiler $\text{ARK}[\text{IOP}, \text{PCS}]$ that transforms the polynomial IOP IOP into an interactive argument of knowledge using the polynomial commitment scheme PCS is almost identical to the one of Chiesa et al. [CHMMVW20] with minimal syntactical variations.

- The trusted party invokes $\text{crs} \leftarrow \text{PCS.S}(1^\lambda)$. Then computes $(pk, vk) \leftarrow \text{PCS.K}(\text{crs}, d)$ where the integer d is computed as the maximum degree bound of the polynomial degrees of IOP of indices of size at most N :

$$d := \max\{d(N, j, i) : j \in [k(|i|)], i \in [o(j)]\}$$

It publishes the proving key pk and stores the (secret) verification key vk .⁸

- The indexer, given as input the index in a relation R computes the commitment to any polynomials in the 0-th round using no randomness as $C_{0,i} := \text{PCS.C}(pk, p_i^{(0)}; 0)$ for $i \in [o(0)]$. The indexer returns $\rho_0 = (i, C_{0,i}, \dots, C_{0,o(0)})$.
- The prover $P(pk, \rho_0, x, w)$ receives as input the proving key pk and the indexer's output ρ_0 , the instance x and witness w . The verifier $V(vk, \rho_0, x)$ receives as input the verification key vk , the indexer's output and the instance.

At the i -th round, the verifier internally runs the verifier to receive the challenge c_i and sends it to the prover. The prover internally runs the next-message function of the prover IOP.P obtaining the polynomials $p_i^{(j)}$ for $i \in [o(n)]$. It commits to each of them running $C_j^{(i)} \leftarrow \text{PCS.C}(pk, p_i^{(j)})$. The verifier notifies IOP.V that the round has finished.

At the end of the interactive phase, the verifier sends the randomness for the query phase $c_{k(|i|)+1}$. The verifier computes the query set $Q = Q_{\text{IOP.V}}(x; c_1, \dots, c_{n+1})$ and sends those to the prover. The prover responds with claimed evaluations $z_i^{(j)}$ for each polynomial $p_i^{(j)}$ queries at location $y_i^{(j)}$ from the query set. The verifier sends a batch opening challenge ζ and the prover replies with the batch open procedure using pk , the list of polynomials sent throughout the protocol $(p_i^{(j)})_{j \in [k(|i|)], i \in [o(j)]}$, their respective degree d , the query set Q and the batch opening challenge ζ . The verifier accepts if the verification equation is satisfied and the polynomial verification is correct with respect to PCS.V .

The proof of the following theorems is omitted as mostly identical to the one of Chiesa et al. [CHMMVW20, Thm. 8.3, 8.4].

Theorem 7.5. *Let IOP be a knowledge-sound, polynomial interactive oracle proof over field family $\mathcal{F}$ for R , and PCS be a designated-verifier extractable polynomial commitment scheme over $\mathcal{F}$. Then $\text{ARK}[\text{IOP}, \text{PCS}]$ is a designated-verifier argument of knowledge for the relation R .*

⁸In practice, we seek PCS.K to be generated by the (potentially malicious) verifier. The verifier will then prove that the key is well-formed.

Theorem 7.6. *Let IOP be a q -query bounded zero-knowledge polynomial interactive oracle proof for R over field family $\mathcal{F}$, and PCS be a designated-verifier hiding polynomial commitment scheme over $\mathcal{F}$. Then $\text{ARK}[\text{IOP}, \text{PCS}]$ is a designated-verifier argument of knowledge for the relation R . Let PCS be a designated-verifier zero-knowledge proof system.*

Let $\text{ZKP}_{\text{pcs},k}$ be a designated-verifier zero-knowledge proof system for the relation

$$R_{\text{pcs},k} = \{(vk, pk) : (vk, pk) \in [\text{PCS.K}(\text{crs})]\} .$$

Let $\text{ARK}[\text{IOP}, \text{PCS}, \text{ZKP}_{\text{pcs},k}]$ denote the compiler above where PCS.K is performed by the verifier, and pk is accompanied by a proof $\pi_{\text{pcs},k}$ that pk lives in the range of the key generation algorithm. Then, we have the following corollary.

Corollary 7.7. *Let IOP be a q -query bounded zero-knowledge polynomial interactive oracle proof for R over field family $\mathcal{F}$, and PCS be a designated-verifier hiding polynomial commitment scheme over $\mathcal{F}$, and $\text{ZKP}_{\text{pcs},k}$ as above. Then $\text{ARK}[\text{IOP}, \text{PCS}, \text{ZKP}_{\text{pcs},k}]$ is a designated-verifier argument of knowledge for the relation R .*

Example 7.8 (A designated-verifier range proof RAP). *We consider the protocol sketched by Boneh, Fisch, Gabizon, Williamson⁹ to prove that a committed value v is in a range $[0, 2^d]$. Let $H := \langle \omega \rangle$ be a subgroup of $\mathbb{F}^\times$ of size d and $Z(x)$ be the polynomial where H vanishes. A common choice here is to select the group of the d -th roots of unity for which the vanishing polynomial can be computed in $\log d$ field operations.*

The proof RAP is for a witness $v \in [0, 2^d - 1]$ opening a previously committed polynomial such that $f(\omega^d) = v$. (The polynomial f is committed using the designated-verifier polynomial commitment scheme; proving that the commitment opens to a signed value boils down to a DLEQ proof.) A common choice is $d = 64$ to prove 64-bit integers.

At a high level, the verifier generates the proving key vk, pk via $\text{dvKZG.S}(1^\lambda, d + 2)$ and then proves in $\pi_{\text{kzg},s}$ knowledge of τ such that:

$$\tau \cdot \left(\sum_{i=0}^n \mu_i G_i \right) = \sum_{i=0}^{n-1} \mu^i G_{i+1}$$

where $\mu := H_s(pk)$. Upon downloading the proving key, the prover checks that none of the elements in pk is zero and $\pi_{\text{kzg},s}$ is verified. The prover RAP.P considers the binary decomposition $(b_0, \dots, b_{d-1})$ of v and define the polynomial $g(x)$ such that $g(\omega^i) = \sum_{j < i} 2^j b_j$ for $i \in [0, d]$. The polynomial satisfies:

$$\begin{aligned} g(\omega^0) &\in \{0, 1\} \\ g(\omega^d) &= f(\omega^d) \\ g(\omega^{i+1}) - 2g(\omega^i) &\in \{0, 1\} \end{aligned} \tag{34}$$

To provide zero-knowledge for the query bound of this interactive oracle proof, interpolate g on two additional random points $\delta_1, \delta_2 \notin H$ such that $g(\delta_1) = \alpha_1 \leftarrow \$ \mathbb{Z}_p$ and $g(\delta_2) = \alpha_2 \leftarrow \$ \mathbb{Z}_p$. The prover commits to $g(\mathbf{x})$ via dvKZG.C .

⁹Descriptions of the protocols can be found in <https://hackmd.io/@dabo/B1U4kx8XI>, <https://decentralizedthoughts.github.io/2020-03-03-range-proofs-from-polynomial-commitments-reexplained/> and the SoK of Christ et al. [CBCMRW24, sec. 3.2].

Table 4: Table of range proof sizes for 64-bit integers. The column $|\pi|$ indicates the size of the range proof over a 64-bit integer, and in parentheses is the size of the range proof over secp256k1. In Sharp, which involves check of small norm for scalar elements, only concrete size is given as it can't be expressed simply in terms of group elements. Note that batch range proofs will be more performing than the naïve approach. We omit the proving key from the table, which in our case amount to 64 group elements. Timing estimates are based on `zka.1c` [Ern+24].

Scheme	$ \pi $ (64-bits)	P time	V time	Notes
Bulletproofs++ [EKRN24]	$10g + 3s$ (416B)	4ms	1.8ms	
Sharp [CGKR22]	n/a (389B)	1.17ms	0.75ms	Requires fine-tuning on the chosen elliptic curve
HashWires [CCLMR21]	n/a (200B)	0.65ms	0.61ms	Relies on a trusted party generating the commitment
This example (RAP)	6g (192B)	1.16ms	0.752ms	Designated-verifier NIZK

The verifier samples $\rho \leftarrow \mathbb{Z}_p$ and sends it to the prover.

The prover computes:

$$\begin{aligned}
w_1(\mathbf{x}) &:= \frac{Z(\mathbf{x})}{\mathbf{x}\omega^0} \cdot g(\mathbf{x}) \cdot (1 - g(\mathbf{x})) \\
w_2(\mathbf{x}) &:= \frac{Z(\mathbf{x})}{\mathbf{x}\omega^d} \cdot (g(\mathbf{x}) - f(\mathbf{x})) \\
w_3(\mathbf{x}) &:= \frac{Z(\mathbf{x})}{\prod_{i \neq d-1} \omega^i \mathbf{x}} (g(\omega \mathbf{x}) - 2g(\mathbf{x}))(1 - g(\omega \mathbf{x}) + 2g(\mathbf{x}))
\end{aligned}$$

and “batches” them into $w(\mathbf{x}) = w_1 + \rho w_2 + \rho^2 w_3$. To show that $Z \mid w$ (which implies Equation (34)) it is sufficient to exhibit $q(\mathbf{x})$ such that:

$$q(\mathbf{x})Z(\mathbf{x}) = w(\mathbf{x}) \tag{35}$$

$$= \frac{Z(\mathbf{x})}{\mathbf{x}\omega^0} \cdot g(\mathbf{x}) \cdot (1 - g(\mathbf{x})) + \frac{Z(\mathbf{x})}{\mathbf{x}\omega^d} \cdot (g(\mathbf{x}) - f(\mathbf{x})) + \tag{36}$$

$$\frac{Z(\mathbf{x})}{\prod_{i \neq d-1} \mathbf{x} - \omega^i} (g(\omega \mathbf{x}) - 2g(\mathbf{x}))(1 - g(\omega \mathbf{x}) + 2g(\mathbf{x})) \tag{37}$$

The prover sends a commitment to $q(\mathbf{x})$ to the verifier.

The verifier samples $r \leftarrow \mathbb{Z}_p$ and the prover sends openings of $g(\omega r), g(r), \hat{q}(r)$ where:

$$\hat{q}(\mathbf{x}) := \frac{Z(r)}{r - \omega^d} f(\mathbf{x}) + q(\mathbf{x})Z(r) \tag{38}$$

The verifier checks the polynomial openings well-formedness of Equation (35). Performances of the above range proof and comparisons are displayed in Table 4.

8 Building on keyed-verification credential systems

In this section we gather some simple extensions and techniques that could be of interest for future applications.

8.1 Time-based policies

To limit the lifetime of a credential and guarantee safe expiry without key rotation is a desirable feature in many applications. As a warm-up example, we study the case of credentials subject to time-based policies such as expiry.

Let KVAC be a keyed-verification credential over message family the integers modulo p satisfying $64 < \lceil \log p \rceil$, and assume time is represented in unix time as a 64-bit signed integer. During issuance, server and user agree on a timestamp t and run the issuance protocol with attributes $(\vec{m}, t)$. At presentation time, the user proves that the credential is not yet expired (that is, $t < e$ where e denotes the expiry time) via:

$$\phi_{\text{exp}}^e(\vec{m}) := ((e - t) \bmod p \in \{0, \dots, 2^{64} - 1\})$$

where t denotes the timestamp attribute and e denotes the expiry timestamp, after which credentials are to be considered stale. Christ et al. [CBCMRW24] present and detail state-of-art range proofs and the trade-offs between them. In Theorem 7.8 we present an example range proof.

No adversary in possession of an expired credential (that is, with attribute $t > e$) can have a successful presentation for the above statement as long as $\log p > 64$. Any two users with valid attributes for which ϕ_{exp} holds are indistinguishable to the server, by anonymity (which in turn implies indistinguishability of two credentials satisfying the same predicate)

Therefore, any keyed-verification credential system supporting $n > 1$, attribute space $\mathbb{M} = \mathbb{Z}_p$ with $\lceil \log p \rceil > 64$ and a predicate space Φ sufficiently expressive, can be used to implement the above feature.

Remark 8.1. *Other ways are possible to enforce expiry of a credential. One of them is to partition expiration into time frames. When issuing a credential, the user is associated to an expiration time frame and their credential embedded with it. This approach, while simpler, entails a more complex trade-off with anonymity as it may fraction the anonymity set into users per time-frame.*

8.2 Rate-limiting

In keyed-verification credentials, authorizations can be presented an indefinite number of times while preserving privacy. A valid specialization of the authorization case is to control the limited use of some finite resource, while preserving anonymity of the users. This problem is treated in *k-times anonymous authentication* [CHKLM06; TFS04; NS04], a credential system where users, a group manager (in charge of distributing credentials), and some application providers (in charge of moderating accesses) enforce rate-limiting access to users.

Here we work in a simple scenario composed only of a server and a user, without revocation to target a simple and efficient rate-limiting system similar to Privacy Pass [DGSTV18]. In this model, we don't consider revocation, nor credential hijacking. At a high level, the approach here is to have the user get a credential for a PRF key k and evaluate the PRF over a counter to generate pseudorandom "tokens" that are unlinkable. The issuer maintains a list of "spent tokens" and

enforces double-spending by ensuring no token is ever re-used for the same scope. The user, to present a valid token, proves the PRF has been correctly evaluated and that its input is “small”.

8.2.1 Syntax

A rate-limiting anonymous token $\text{RTL} = (\text{S}, \text{K}, \text{I}, \text{P})$ is composed of:

- $\text{crs} \leftarrow \text{RTL.S}(1^\lambda)$ the setup algorithm, which generates a common reference string.
- $(\text{sk}, \text{pp}) \leftarrow \text{RTL.K}(\text{crs})$ the key generation algorithm, which produces a secret key sk and some public parameters pp .
- $\hat{\sigma} \leftarrow (\text{RTL.I.Usr}(\text{pp}) \Rightarrow \text{RTL.I.Srv}(\text{sk}))$ the issuance algorithm, which produces a credential $\hat{\sigma}$. We consider the procedure to be non-interactive, that is, we consider the following 3 algorithms:
 - $(\mu, \text{st}) \leftarrow \text{RTL.I.Usr}_1(\text{pp})$ the first part of the issuance algorithm, returning μ and some user state st .
 - $\sigma' \leftarrow \text{RTL.I.Srv}(\text{sk}, \mu)$, the server signing algorithms, which returns a blinded credential σ' .
If the input was malformed, $\sigma' = \perp$.
 - $\hat{\sigma} \leftarrow \text{RTL.I.Usr}_2(\text{st}, \sigma')$ the second part of the user issuance algorithm, which returns the credential $\hat{\sigma}$.
- $0/1 \leftarrow (\text{RTL.P.Usr}(\text{pp}, \hat{\sigma}, (\ell, \text{scp}, i)) \Rightarrow \text{RTL.P.Srv}(\text{sk}, \ell, \text{scp}))$ the presentation algorithm, which takes as input a scope $\text{scp} \in \{0, 1\}^*$ identifying the resource and (for the user) a counter i for generating the i -th token. It returns a bit (and updates the server state). We assume presentation to be non-interactive and thus consider the following two algorithms:
 - $(t, \rho) \leftarrow \text{RTL.P.Usr}(\text{pp}, \sigma, (\ell, \text{scp}, i))$ the user presentation algorithm, which returns a pair (t, ρ) where t identifies the token.
 - $0/1 \leftarrow \text{RTL.P.Srv}(\text{sk}, (\ell, \text{scp}, t), \rho)$ the server presentation algorithm, which returns 1 if the token (t, ρ) is considered valid and for at most ℓ accesses and 0 otherwise.

Similarly to the case of keyed-verification credentials, we will consider the procedure $\text{RTL.M}(\text{sk})$ that generates a fresh credential using sk interacting with a mock user. This is a shorthand of a server emulating the user and returning its result.

A rate-limiting scheme must be *correct*, *unforgeable*, and *unlinkable*. Correctness means that for every $\text{crs} \in \text{RTL.S}(1^\lambda)$, $(\text{sk}, \text{pp}) \leftarrow \text{RTL.K}(\text{crs})$, $\ell > 0$ and $0 \leq i < \ell$, $\text{scp} \in \{0, 1\}^*$

$$\Pr \left[b = 1 : \begin{array}{l} \hat{\sigma} \leftarrow (\text{RTL.I.Usr}(\text{pp}) \Rightarrow \text{RTL.I.Srv}(\text{sk})) \\ b \leftarrow (\text{RTL.P.Usr}(\text{pp}, \hat{\sigma}, (\ell, \text{scp}, i)) \Rightarrow \text{RTL.P.Srv}(\text{sk}, (\ell, \text{scp}))) \end{array} \right] = 1 \ .$$

A rate-limiting anonymous token is one-more unforgeable if for every $\ell \geq 0$, and any adversary A , the advantage $\text{Adv}_{\text{RTL}, \text{A}}^{\text{omuf}}(\lambda) := \Pr[\text{OMUF}_{\text{RTL}, \text{A}}(\lambda, \ell) = 1]$ (defined in Figure 11) is negligible in λ . A rate-limiting anonymous token is unlinkable if for any $\ell > 0$ and (stateful) adversary A , the advantage $\text{Adv}_{\text{RTL}, \text{A}}^{\text{unlink}}(\lambda, \ell)$ defined in Figure 11 is negligible in λ .

Game $\text{UNF}_{\text{RTL},A}(\lambda, \ell)$ $q := 0; \text{ctr} := 0; Qrs := []$ $\text{crs} \leftarrow \text{RTL.S}(1^\lambda)$ $(sk, pp) \leftarrow \text{RTL.K}(\text{crs})$ $(\text{scp}^*, (t_i^*, \rho_i^*)_{i=1}^{\ell q+1}) \leftarrow \mathbf{A}^{\text{Issue, Present, NewUsr, PresentUsr}}(pp)$ return $\forall i: \text{RTL.P.Srv}(sk, \ell, \text{scp}^*, t_i^*, \rho_i^*) = 1$ $\forall i \neq j: t_i \neq t_j \wedge$ $\forall i: (t_i^*, \rho_i^*) \notin Qrs$	Oracle $\text{NEWUsr}()$ $\hat{\sigma} := \text{RTL.M}(sk)$ $\text{Usrs}[\text{ctr}] := \hat{\sigma}$ $\text{ctr} := \text{ctr} + 1$ return ctr Oracle $\text{PRESENTUsr}(i, \text{scp})$ $(t, \rho) \leftarrow \text{RTL.P.Usr}(pp, \text{Usrs}[i], \ell, \text{scp}, i)$ $Qrs := Qrs \cup \{(t, \rho)\}$	Oracle $\text{ISSUE}(\mu)$ $q := q + 1$ return $\text{RTL.I.Srv}(sk, \mu)$ Oracle $\text{PRESENT}(\text{scp}, t, \rho)$ return $\text{RTL.P.Srv}(sk, \ell, \text{scp}, t, \rho)$
Game $\text{UNLINK}_{\text{RTL},A,D}^b(\lambda, \ell)$ $\text{crs} \leftarrow \text{RTL.S}(\text{crs})$ $\text{Ctr} := []$ $(pp, st_A) \leftarrow \mathbf{A}(\text{crs})$ $(\sigma_0; st'_A) \leftarrow (\text{RTL.I.Usr}(pp) \Rightarrow \mathbf{A}(st_A))$ $(\sigma_1; st_D) \leftarrow (\text{RTL.I.Usr}(pp) \Rightarrow \mathbf{A}(st'_A))$ $b' \leftarrow \mathbf{D}^{\text{GET, CHAL}}(st_D)$ return $b' = 1$	Oracle $\text{GET}(\beta, \text{scp})$ $\text{Ctr}[\text{scp}][\beta] := \text{Ctr}[\text{scp}][\beta] + 1$ or 1 if $\text{Ctr}[\text{scp}][0] > \ell \vee \text{Ctr}[\text{scp}][1] > \ell$: return $\perp$ return $\text{RTL.P.Usr}(sk, \hat{\sigma}_\beta, (\ell, \text{scp}, \text{Ctr}[\text{scp}][\beta]))$ Oracle $\text{CHAL}(\text{scp})$ return $(\text{GET}(b, \text{scp}), \text{GET}(1 - b, \text{scp}))$	

Figure 11: Unforgeability and unlinkability game for a rate-limiting anonymous token RTL. The adversary of unlinkability is assumed to be stateful and to preserve state across calls.

8.2.2 Our compiler

Let ϕ_{range}^ℓ denote the predicate that checks if the input is in the range $[0, \ell - 1]$. Let $\text{PRF}(k, (x, y))$ denote a pseudo-random function (PRF) evaluation on key k and message (x, y) . The message is presented as a pair so to make it easy to prove statements about parts of the rate-limiting input. A PRF is **compatible** with the keyed-verification credential KVAC if the message space of the credential system is equal to the key space of the PRF, and $\phi_{\text{prf}}^{m,e}(k) := (\text{PRF}(k, m) = e)$ is part of the predicate family. The more general case is easy to derive from this one.

We describe a compiler that, given a keyed-verification credential KVAC and a pseudorandom function PRF compatible with KVAC, produces a rate-limiting scheme. We denote such system as $\text{cRTL}[\text{KVAC}, \text{PRF}]$ and define it as follows:

- $\text{cRTL.S}(1^\lambda)$, the setup algorithm runs $\text{crs} \leftarrow \text{KVAC.S}(1^\lambda, 1)$ and returns crs .
- $\text{cRTL.K}(\text{crs})$, the key generation algorithm returns $(sk, pp) \leftarrow \text{KVAC.K}(\text{crs})$.
- $(\text{cRTL.I.Usr}(pp) \Rightarrow \text{cRTL.I.Srv}(sk))$, the credential issuance algorithm has the server run $\text{KVAC.I.Srv}(sk, st, \phi_1)$, while the user algorithm samples a random $k \leftarrow \mathbb{M}$ and runs $\text{KVAC.I.Usr}(pp, k, \phi_1)$. At the end of the interaction, the user obtains a credential σ and returns the pair $\hat{\sigma} := (k, \sigma)$.
- $(\text{cRTL.P.Usr}(pp, \hat{\sigma} = (k, \sigma), (\ell, \text{scp}, i)) \Rightarrow \text{cRTL.P.Srv}(sk, (\ell, \text{scp})))$, the credential presentation algorithm has the server run $\text{KVAC.P.Srv}(sk, \phi_{\text{prf}}^{\text{scp}, t} \wedge \phi_{\text{range}}^\ell)$, while the user runs $\text{KVAC.P.Usr}(pp, k, \sigma, \phi_{\text{prf}}^{\text{scp}, t} \wedge \phi_{\text{range}}^\ell)$.

The next section is devoted to proving the following:

Theorem 8.2. *Let $\ell > 0$, and $\Phi \supset \{\phi_{\text{prf}}, \phi_{\text{range}}\}$. If KVAC be a keyed-verification credential for $n = 1$ attributes and predicate family Φ , and PRF is a pseudorandom function compatible with KVAC, then $\text{cRTL}[\text{KVAC}, \text{PRF}]$ is a rate-limiting anonymous token with one-more unforgeability advantage:*

$$\text{Adv}_{\text{cRTL}[\text{KVAC}, \text{PRF}]}^{\text{omuf}}(\lambda, \ell) \leq (q\ell + 1)\text{Adv}_{\text{KVAC}}^{\text{ext}}(\lambda, 1) + q\text{Adv}_{\text{KVAC}}^{\text{anon}}(\lambda, 1) + q\text{Adv}_{\text{PRF}}^{\text{prnd}}(\lambda),$$

(where q is the number of queries to the issuance oracles) and unlinkability advantage:

$$\text{Adv}_{\text{cRTL}[\text{KVAC}, \text{PRF}]}^{\text{unlink}}(\lambda, \ell) \leq 2(\text{Adv}_{\text{KVAC}}^{\text{anon}}(\lambda, 1) + \text{Adv}_{\text{PRF}}^{\text{prnd}}(\lambda)) .$$

Remark 8.3. The restriction on the PRF being compatible with KVAC may be lifted, provided that the user proves at issuance time well-formedness of the PRF key.

Remark 8.4. In the proof, extraction is required solely to carry out a counting argument after making sure that all tokens are evaluated from keys issued to the user. In fact, for the credentials μCMZ and μBBS it is still possible to extract from the zero-knowledge proofs sent at presentation time and carry out a reduction to one-more unforgeability.

Corollary 8.5. Let $\ell > 0$, and $\Phi \supset \{\phi_{\text{prf}}, \phi_{\text{range}}\}$. Consider a one-more unforgeable keyed-verification credential $\text{KVAC}_{\text{AT}} \in \{\mu\text{CMZ}_{\text{AT}}, \mu\text{BBS}_{\text{AT}}\}$, for attribute family Φ and let PRF be a pseudorandom function compatible with KVAC_{AT} , then cRTL is a rate-limiting anonymous token with one-more unforgeability advantage:

$$\text{Adv}_{\text{cRTL}[\text{KVAC}_{\text{AT}}[\text{ZKP}], \text{PRF}]}^{\text{omuf}}(\lambda, \ell) \leq \text{Adv}_{\text{KVAC}_{\text{AT}}}^{\text{omuf}}(\lambda, 1) + (q\ell + 1)\text{Adv}_{\text{ZKP}}^{\text{ksnd}}(\lambda) + q\text{Adv}_{\text{KVAC}_{\text{AT}}}^{\text{anon}}(\lambda, 1) + q\text{Adv}_{\text{PRF}}^{\text{prnd}}(\lambda) ,$$

and unlinkability advantage:

$$\text{Adv}_{\text{cRTL}[\text{KVAC}_{\text{AT}}, \text{PRF}]}^{\text{unlink}}(\lambda, \ell) \leq 2(\text{Adv}_{\text{KVAC}_{\text{AT}}}^{\text{anon}}(\lambda, 1) + \text{Adv}_{\text{PRF}}^{\text{prnd}}(\lambda)) ,$$

where q is the number of oracle queries made by the adversary.

8.2.3 Analysis

Correctness follows by inspection; unlinkability follows directly from pseudorandomness and anonymity. We focus on unforgeability.

Lemma 8.6. Let $\ell > 0$, and $\Phi \supset \{\phi_{\text{prf}}, \phi_{\text{range}}\}$. If KVAC be a keyed-verification credential for $n = 1$ attributes and predicate family Φ , and PRF is a PRF, then $\text{cRTL}[\text{KVAC}, \text{PRF}]$ has one-more unforgeability advantage:

$$\text{Adv}_{\text{cRTL}[\text{KVAC}, \text{PRF}]}^{\text{omuf}}(\lambda, \ell) \leq (q\ell + 1)\text{Adv}_{\text{KVAC}}^{\text{ext}}(\lambda, 1) + q_u\text{Adv}_{\text{KVAC}}^{\text{anon}}(\lambda, 1) + q_u\text{Adv}_{\text{PRF}}^{\text{prnd}}(\lambda) ,$$

where q is the number of queries to *ISSUE* and q_u is the number of queries to *NEWUSR*.

Proof. Consider an adversary $\mathbf{A}$ for $\text{OMUF}_{\text{cRTL}[\text{KVAC}, \text{PRF}]}(\lambda)$. $\mathbf{A}$ receives as input pp and during the execution has access to oracles:

- *ISSUE*, to issue a credential to the adversary;
- *PRESENT*, to present a credential to the adversary;
- *NEWUSR*, and *PRESENTUSR*, to respectively create new honest users, and have them produce a new presentation token.

We build a reduction $\mathbf{B}$ for extractability of KVAC. $\mathbf{B}$ receives as input pp and initializes a list $\text{Keys} = []$. It runs internally $\mathbf{A}(pp)$, responding to each oracle query as follows:

- each issuance query with input μ is responded with $\text{ISSUE}(\mu, \phi_1)$ (*ISSUE* being the issuance oracle of the extractability game);
- each present query with input (scp, t, ρ) is responded with $\text{PRESENT}(\phi_{\text{prf}}^{\text{scp}, t} \wedge \phi_{\text{range}}^\ell, \rho)$;

- each new user query is responded by sampling $k \leftarrow \mathbb{M}$, appending it to Keys , and querying $\text{NEWUSR}(k)$;
- each user presentation query with input (i, scp) has the adversary $\mathbf{B}$ compute $t := \text{PRF}(\text{Keys}[i], \text{scp})$ and respond with the answer of $\text{PRESENTUSR}(i, \phi_{\text{prf}}^{\text{scp}, t} \wedge \phi_{\text{range}}^\ell)$.

At the end of the execution, $\mathbf{A}$ outputs valid (for scp^*) forgeries $(t_i^*, \rho_i^*)_{i=1}^{\ell q+1}$ such that $t_i^* \neq t_j^*$ for all $i \neq j$ and (t_i^*, ρ_i^*) was not previously queried. We distinguish the following events:

- $\exists i \in [\ell q + 1] : t_i^* \neq \text{PRF}(k_i^*, \text{scp}^*, j_i^*)$ (for some $j_i^* \in [0, \ell - 1]$), in which case extraction failed, and the predicate does not hold. This can be shown with a reduction to the extractability game, where the reduction at the end guesses the presentation message of the adversary output that constitutes a valid forgery.
- $\exists i \in [\ell q + 1] : k_i^* \in \text{Keys}$, which means that a forgery was made from an honest user. However, it can be shown that since the PRF evaluations are pseudorandom (and independent of the keys), satisfying this event is equivalent to guessing the PRF key of a user. This can be shown with a hybrid argument, first replacing the honest user issuance and presentation sessions with simulated ones, and then replacing the PRF evaluations with random ones. At the end of both changes, the NEWUSR and PRESENTUSR oracles do not create any PRF key, nor credentials. The resulting proofs are not part of the (winning) adversary output (as they are not part of $PQrs$).

By the pigeonhole principle on $\text{PRF}(\cdot, \text{scp}, \cdot)$, where the first argument is Qrs (the set of issued credentials in the extractability game, of size q) and the last argument is in $[0, \ell - 1]$ (of size ℓ), it must be that $\exists i, j \in [\ell q + 1] : k_i^* = k_j^*$ (which contradicts one of the winning conditions) and so the adversary can never win the game in this case. $\square$

8.2.4 Instantiation

We describe a variation of the Dodis–Yampolskiy’s PRF [DY05] that makes it easy to reason and prove statements on the counter. The PRF is defined as:

$$\text{HashDY}(k, (\text{scp}, \text{ctr})) := (k + \text{ctr})^{-1} \cdot \text{H}_{\mathbb{G}}(\text{scp})$$

for $\text{ctr} \in \mathbb{Z}_p$ and $\text{ctr} \leq \ell = \text{poly}(\lambda)$ and $\text{H}_{\mathbb{G}} : \{0, 1\}^* \rightarrow \mathbb{G}$ a hash that maps elements in the group. In this case, a “token” is $T \in \mathbb{G}$ (indicated here in uppercase to match the group description). Note that the predicate:

$$\phi_{\text{prf}}^{\text{scp}, T, \ell}(\vec{m}, k, i) := ((k + i)T = \text{H}_{\mathbb{G}}(\text{scp}) \wedge 0 \leq i < \ell)$$

can be proven easily within Schnorr proofs, with the help of a range proof. We prove that HashDY is a PRF.

Theorem 8.7. *In the random oracle model, if q -DDHI holds over GrGen , then HashDY is pseudo-random with advantage:*

$$\text{Adv}_{\text{HashDY}, \mathbf{A}}^{\text{prnd}}(\lambda) \leq q \text{Adv}_{\text{GrGen}}^{q\text{-ddhi}}(\lambda) + \frac{q^2}{p} ,$$

where q is the number of evaluation queries.

Proof. Let A be a p.p.t. adversary for pseudorandomness. A has access to an oracle EVAL for arbitrary $scp \in \{0, 1\}^*$ and counters $0 \leq i < \ell$. Consider a hybrid argument where, in the i -th step, we replace the EVAL responses up to the i -th query with uniformly random group elements. Consider an adversary A_i able to distinguish the i -th hybrid. We construct an adversary B_i for the i -th hybrid able to distinguish every time that the adversary A_i 's output is different during the i -th hybrid change.

B_i receives as input a group description Γ and a q -DDHI challenge $\vec{G} = (G, \tau G, \tau^2 G, \dots, \tau^q G)$ and $P \in \mathbb{G}$ (either $P = 1/\tau \cdot G$ in $q\text{-DDHI}_{A, \text{GrGen}}^0(\lambda)$, or a uniformly distributed random element $P \leftarrow_{\$} \mathbb{G}$ in $q\text{-DDHI}_{A, \text{GrGen}}^1(\lambda)$), with $q \leq \ell$. Let $\alpha := \tau - i$ and define $\vec{A} := (G, \alpha G, \alpha^2 G, \dots, \alpha^q G)$. These can be computed using the binomial theorem using $\vec{G}$ only. Once computed, B internally runs the adversary A using the crs received as input. During its execution, the adversary A query to $H_{\mathbb{G}}$ and EVAL :

- Upon receiving the j -th query to the random oracle $H_{\mathbb{G}}$ of the form $H_{\mathbb{G}}(scp_j)$ such that $scp_j \in \{0, 1\}^*$, B checks if the element was previously queried. If that is not the case, B computes the polynomial $f(x) := \prod_{\substack{\iota=1 \\ \iota \neq i}}^{\ell} (x + \iota) = \sum_k f_k x^k$ and samples $\theta_j \leftarrow_{\$} \mathbb{Z}_p$. Finally, it computes $T_j = \theta_j f(\alpha) G = \theta_j \cdot \sum_k f_k A_k$ and stores in a table (scp_j, θ_j, T_j) and returns to the user T_j . If such a query was already made, B fetches T_j from its records and returns it the same value as before.
- Upon receiving the j -th EVAL query of the form (scp_j, ι_j) from the adversary A , the reduction B checks if scp_j was already queried in $H_{\mathbb{G}}$ and, if not, proceeds doing so. Then, it checks if ι_j is equal to i . If it is not equal, it retrieves the record (scp_j, θ_j, T_j) and returns $(\tau + \iota_j)^{-1} \cdot T_j$, computed via $h(x) := \prod_{\iota \neq i, \iota_j} (x + \iota)$ and returning $\theta_j \cdot \sum_k h_k G_k$. If $\iota_j = i$, then by euclidean remainder we can write $f(x) = h(x)(x+i) + r$ where $r = f(i)$. Therefore, $(\alpha+i)^{-1} \cdot \theta_j \cdot f(\alpha) \cdot G = \theta_j \cdot (h(\alpha) + r/(\alpha+i)) G = \theta_j (\sum_k h_k A_k + f(i)P)$.

The output distribution of $H_{\mathbb{G}}$ is uniformly random except with statistically negligible probability (when α is a root of f). In the case $q\text{-DDHI}_{A, \text{GrGen}}^0(\lambda)$ the output distribution of EVAL is identically distributed, while it is uniformly distributed in $q\text{-DDHI}_{A, \text{GrGen}}^1(\lambda)$. (Recall that $P = 1/\tau G = 1/(\alpha+i)G$ in $q\text{-DDHI}_{A, \text{GrGen}}^0(\lambda)$ and $A_i = \alpha^i G$.) Indistinguishability then is straightforward by the distance lemma over each of the hybrid changes. $\square$

8.2.5 Other methods for rate-limiting

We briefly mention below other approaches that may be used to enforce rate limiting.

Grinding. Simpler methods for rate-limiting are folklore, such as providing a proof of work associated with the request. This technique can be embedded in the rate-limiting token part t : roughly speaking, upon presentation the token t must also satisfy that t is “small”. This is sometimes referred to as *grinding* in the literature and may be implemented appending a random nonce to the scope, having then the user attempt to find a nonce such that the PRF output is small.

Batch issuance of spend-once credentials. Another common approach is to issue multiple spend-once credentials: the user sets (at issuance time) a blind attribute to be a nonce, and fully discloses it to the server upon presentation. The issuer, on the other hand, keeps a list of previously spent nonces and enforces rate-limiting by adjusting the number of spend-once credentials. This

Game $\text{ANON}_{\text{NYM}, \text{A}, \text{D}}^b(\lambda)$	Oracle $\text{GET}(scp, m)$
$crs \leftarrow \text{NYM.S}(\lambda)$	$Qrs := Qrs \cup \{(scp, m)\}$
$(pp, st_A) \leftarrow \text{A}(crs)$	for $\beta \in \{0, 1\}$
$(\sigma_0; st'_A) \leftarrow (\text{NYM.I.Usr}(pp) \Rightarrow \text{A}(st_A))$	$nym_\beta := \text{NYM.E}(pp, k_\beta, scp)$
$(\sigma_1; st_D) \leftarrow (\text{NYM.I.Usr}(pp) \Rightarrow \text{A}(st_A))$	$\rho_\beta \leftarrow \text{NYM.P.Usr}(pp, (k_\beta, \sigma_\beta), (scp, nym_\beta, m))$
$b' \leftarrow \text{D}^{\text{GET}, \text{CHAL}}(st_D)$	return $((nym_0, \rho_0), (nym_1, \rho_1))$
return b'	
	Oracle $\text{CHAL}(scp, m)$
	if $(scp, m) \in Qrs$: return $\perp$
	$(\alpha_0, \alpha_1) \leftarrow \text{GET}(scp, m)$
	return (α_b, α_{1-b})

Figure 12: Anonymity game for a keyed-verification pseudonym system NYM.

approach naïvely incurs in a n -times communication and computation overhead in the issuance phase for limiting n -accesses per-user, but classical batching techniques may be available to partially reduce this cost.

8.3 Pseudonyms

A restriction (at least at first sight) of the rate-limiting scheme is the pseudonym extension, where the user is allowed to create a pseudonym for a specific resource $scp \in \{0, 1\}^*$ and use it to authenticate to the server.

In these cases, the authentication server requires the user to adopt an identity for a specific resource denoted $scp \in \{0, 1\}^*$, and have that identity be otherwise unlinkable. Such pseudonym can be used for blocking and logging in users in a specific service without making requests to third-party services or requiring another registration process. The scope $scp \in \{0, 1\}^*$ identifies the scope to be accessed, but one may also make other valid choices. For instance, a timestamp can restrict access over time, a unique random nonce for spend-once credentials, the identifier of a group chat to select a group-dependent pseudonym, etc.

8.3.1 Syntax

A keyed-verification pseudonym system NYM consists of the following efficient procedures:

- $crs \leftarrow \text{NYM.S}(1^\lambda)$, the setup algorithm, getting as input the security parameter in unary form and returning a common reference string crs ;
- $(sk, pp) \leftarrow \text{NYM.K}(crs)$, the key-generation algorithm produces a signing key sk together with some public parameters pp ;
- $(k, \sigma) \leftarrow (\text{NYM.I.Usr}(pp) \Rightarrow \text{NYM.I.Srv}(sk))$, the issuance algorithm, producing an identity key k and a credential σ over it. We consider the issuance to be non-interactive, that is, we consider the following 3 algorithms:
 - $(\mu, st) \leftarrow \text{NYM.I.Usr}_1(pp)$ the first part of the issuance algorithm, returning μ and some user state st .

Game $\text{UNF}_{\text{NYM},A}(\lambda)$ $PQrs := []$; $Creds := []$; $Keys := []$; $q := 0$ $crs \leftarrow \text{NYM.S}(1^\lambda)$ $(sk, pp) \leftarrow \text{NYM.K}(crs)$ $(scp^*, (nym_i^*, m_i^*, \rho_i^*)_{i=1}^n) \leftarrow \mathbf{A}^{\text{ISSUE}, \text{PRESENT}, \text{NEWUSR}, \text{PRESENTUSR}}(pp)$ return $\forall i \in [n]: \text{NYM.P.Srv}(sk, (scp_i^*, nym_i^*, m_i^*), \rho_i) = 1 \wedge$ $(scp^*, nym_i^*, m_i^*) \notin PQrs \wedge$ $(\forall i \neq j: nym_i^* \neq nym_j^* \wedge n > q) \vee$ $\exists i, j: nym_i^* = \text{NYM.E}(pp, Keys[j], scp^*)$	Oracle $\text{ISSUE}(\mu)$ $q := q + 1$ return $\text{NYM.I.Srv}(sk, \mu)$ Oracle $\text{PRESENT}((scp, nym, m), \rho)$ return $\text{NYM.P.Srv}(sk, (scp, nym, m), \rho)$ Oracle $\text{EVAL}(i, scp)$ return $\text{NYM.E}(pp, Keys[i], scp)$	Oracle $\text{NEWUSR}()$ $k \leftarrow \mathbb{S} \mathbb{M}$ $\sigma \leftarrow \text{NYM.M}(sk, k)$ $(Keys[q], Creds[q]) := (k, \sigma)$ return 1 Oracle $\text{PRESENTUSR}(i, scp, m)$ $nym := \text{NYM.E}(Keys[i], scp)$ $PQrs := PQrs \cup \{(i, scp, nym, m)\}$ $\rho \leftarrow \text{NYM.P.Usr}(pp, (scp, nym, m))$ return (nym, ρ)
--	---	--

Figure 13: Unforgeability for a keyed-verification pseudonym system NYM.

- $\sigma' \leftarrow \text{NYM.I.Srv}(sk, \mu)$, the server signing algorithms, which returns a blinded credential σ' .
If the input was malformed, $\sigma' = \perp$.
- $(k, \sigma) \leftarrow \text{NYM.I.Usr}_2(st, \sigma')$ the second part of the user issuance algorithm, which returns the identity key k and credential σ .
- $nym := \text{NYM.E}(pp, k, scp)$ the evaluation algorithm generates a pseudonym nym for the scope $scp \in \{0, 1\}^*$.
- $0/1 \leftarrow (\text{NYM.P.Srv}(sk, (scp, nym, m)) \Rightarrow \text{NYM.P.Usr}(pp, (k, \sigma), (scp, nym, m)))$ the presentation algorithm, which authenticates as message m for an authorized identity nym within the scope scp , using the identity key k and credential σ . Since we consider non-interactive presentations, we can simplify the above as:
 - $\mu \leftarrow \text{NYM.P.Usr}(pp, (k, \sigma), (scp, nym, m))$,
 - $0/1 \leftarrow \text{NYM}(sk, (scp, nym, m), \mu)$.

The core security requirements of a pseudonym system are correctness (all honestly-generated nyms verify), unforgeability (that is, a user cannot generate multiple identities for the same scope or act of behalf of another user), and anonymity (that is, two users are indistinguishable across different scopes). More formally, correctness requires that, for all $scp, m \in \{0, 1\}^*$:

$$\Pr \left[b = 1 : \begin{array}{l} crs \leftarrow \text{NYM.S}(1^\lambda) \\ (sk, pp) \leftarrow \text{NYM.K}(crs) \\ (k, \sigma) \leftarrow (\text{NYM.I.Usr}(pp) \Rightarrow \text{NYM.I.Srv}(sk)) \\ nym := \text{NYM.E}(pp, k, scp) \\ b \leftarrow (\text{NYM.P.Srv}(sk, (scp, nym, m)) \Rightarrow \text{NYM.P.Usr}(pp, (k, \sigma), (scp, nym, m))) \end{array} \right] \geq 1 - \text{negl}(\lambda) .$$

Unforgeability asks that $\text{Adv}_{\text{NYM},A}^{\text{unf}}(\lambda) := \Pr[\text{UNF}_{\text{NYM},A}(\lambda) = 1]$ is negligible in λ , where $\text{UNF}_{\text{NYM},A}(\lambda)$ is defined in Figure 13. Anonymity requires that $\text{Adv}_{\text{NYM},A}^{\text{anon}}(\lambda)$ is negligible in λ , where the distinguishing game $\text{ANON}_{\text{NYM},A}^b(\lambda)$ is illustrated in Figure 12.

Remark 8.8 (Differences with VRFs.). *It is possible to re-state those properties in terms of verifiable random functions, but the syntax here is slightly different. Verifiable random functions have a verification algorithm that relies on a “public key” (which here is not used) and require uniqueness, that is, it is not possible to prove two different VRF outputs are valid for the same input and the*

same secret key. Pseudonym evaluation is a deterministic algorithm, and therefore uniqueness perfectly holds.

8.3.2 Our compiler

A keyed-verification credential KVC supports **labelled presentation** if the presentation message of the user can be labeled. This is denoted as a “dummy predicate” ϕ_{lbl}^m . For the credentials we present, this boils down to demanding ZKP to supporting labelling, e.g. with a message embedded in the Fiat–Shamir transformation.

As in the previous section, we say that a pseudorandom function PRF is **compatible** with a keyed-verification credential KVC if the attribute space of the credential system is equal to the key space of the PRF and $\phi_{\text{prf}}^{m,e}$ is in the predicate family of KVC.

We say that a keyed-verification credential KVC supports **i -rerandomizable issuance** if there exists an issuance protocol $\text{KVC.I}_{\text{rnd}}$ which, as long as at least one of the two participants (user or server) is honest, the credential at the end of the protocol is such that the i -th attribute is uniformly distributed over $\mathbb{M}$.

Let KVC be a keyed-verification credential for $n = 1$ attributes supporting re-rerandomizable issuance and labels. Let PRF be a pseudorandom function compatible with KVC. We describe a compiler $\text{cNYM}[\text{KVC}, \text{PRF}]$ and define it as follows:

- $\text{crs} \leftarrow \text{cNYM.S}(1^\lambda)$, the setup algorithm internally runs $\text{KVC.S}(1^\lambda, 1)$ and return its common reference string;
- $(\text{sk}, \text{pp}) \leftarrow \text{cNYM.K}(\text{crs})$ returns the output of the keyed-verification credential key generation;
- $(k, \sigma) \leftarrow (\text{cNYM.I}(\text{pp}) \Rightarrow \text{cNYM.I.Srv}(\text{sk}))$, the issuance algorithm has the user run $\text{KVC.I}_{\text{rnd}}.\text{Usr}(\text{pp}, [], \phi_1)$ and the server $\text{KVC.I}_{\text{rnd}}.\text{Srv}(\text{sk}, \phi_1)$. At the end of the protocol execution, the user gets a uniformly-distributed attribute k and a credential σ on it.
- $\text{nym} := \text{cNYM.E}(\text{pp}, k, \text{scp})$, the pseudonym evaluation function returns $\text{nym} := \text{PRF}(k, \text{scp})$.
- $0/1 \leftarrow (\text{cNYM.P}.\text{Usr}(\text{pp}, (k, \sigma), (\text{scp}, \text{nym}, m)) \Rightarrow \text{cNYM.P}.\text{Srv}(\text{sk}, (\text{scp}, \text{nym}, m)))$, the presentation algorithm has the user receive as input a pseudonym $\text{nym} := \text{PRF}(k, \text{scp})$ and a message m to be authenticated. It runs $\text{KVC.P}.\text{Usr}(\text{pp}, k, \sigma, \phi_{\text{prf}}^{\text{scp}, \text{nym}} \wedge \phi_{\text{lbl}}^m)$, while the server responds with whatever the underlying protocol $\text{KVC.P}.\text{Srv}(\text{sk}, (\text{scp}, \text{nym}, m))$ returns.

Theorem 8.9. *Let KVC be a keyed-verification credential for $n = 1$ attributes, with re-rerandomizable issuance and labelled presentation, for predicate family $\Phi \supset \{\phi_{\text{prf}}\}$. Let PRF be a pseudorandom function compatible with KVC. Then, $\text{cNYM}[\text{KVC}, \text{PRF}]$ is a keyed-verification pseudonym system with unforgeability advantage:*

$$\text{Adv}_{\text{cNYM}[\text{KVC}, \text{PRF}]}^{\text{unf}}(\lambda) \leq q \text{Adv}_{\text{KVC}}^{\text{ext}}(\lambda, 1) + q \text{Adv}_{\text{KVC}}^{\text{anon}}(\lambda) + q \text{Adv}_{\text{PRF}}^{\text{prnd}}(\lambda) + \frac{q^2}{|\mathbb{M}|} ,$$

(where q is the number of oracle queries) and unlinkability advantage:

$$\text{Adv}_{\text{cNYM}[\text{KVC}, \text{PRF}]}^{\text{unlink}}(\lambda) \leq 2(\text{Adv}_{\text{KVC}}^{\text{anon}}(\lambda, 1) + \text{Adv}_{\text{PRF}}^{\text{prnd}}(\lambda)) .$$

Remark 8.10. *The requirement of PRF compatibility and re-randomization of Kvac can be relaxed with an alternative construction, that we sketch below.*

At issuance time, the user generates a key $k \leftarrow \text{NYM.K}(\text{crs})$ and an evaluation $\text{nym}_0 \leftarrow \text{NYM.E}(k, \varepsilon)$ (where ε denotes the empty string). At issuance time, the user sets the key k to be one of the (hidden) credential attributes, and adds to the user message the registration identity nym_0 . Along with the predicate associated with the other attributes, the user proves knowledge of a valid PRF key $k \in \mathbb{K}$ (a hidden attribute) and that nym_0 is well-formed (via the predicate ϕ_{nym}). The issuer, on the receiving side, checks that nym_0 has not already been used in the past and that it is indeed correctly generated by verifying the predicate (via ϕ_{nym}). If both checks pass, the server proceeds with the issuing of a credential. Upon accessing any other resource $\text{scp} \neq \varepsilon$ the user can provide an identity associated to that resource computing $\text{nym} \leftarrow \text{NYM.E}(k, \text{scp})$ and proving via the predicate ϕ_{nym} that the computation was done correctly.

Remark 8.11. *In the proof, extraction is used to carry out a counting argument after making sure that nym_i^* are all correct evaluations of the function $\text{PRF}(\cdot, \text{scp}^*)$. Therefore, one may wonder if one-more unforgeability is sufficient for proving security. While we don't have at disposal the final messages, the credentials μCMZ and μBBS both rely on a presentation message that is sound, and therefore the argument still applies. In fact, by relying on knowledge soundness of the proofs π_p in both protocols, one can show that $\mu\text{CMZ}_{\text{AT}}$ and $\mu\text{BBS}_{\text{AT}}$ (the schemes without the issuance user proofs) are sufficient for achieving the desired security properties. Note that the “ q ” factor required to “guess” the forgery is now moved to the knowledge soundness experiment.*

Corollary 8.12. *Consider a one-more unforgeable keyed-verification credential $\text{KVAC}_{\text{AT}} \in \{\mu\text{CMZ}_{\text{AT}}, \mu\text{BBS}_{\text{AT}}\}$, for predicate family $\Phi \supset \{\phi_{\text{prf}}\}$. Let PRF be a pseudorandom function compatible with KVAC_{AT} . Then, $\text{cNYM}[\text{KVAC}_{\text{AT}}, \text{PRF}]$ is a keyed-verification pseudonym system with unforgeability advantage:*

$$\text{Adv}_{\text{cNYM}[\text{KVAC}_{\text{AT}}, \text{PRF}]}^{\text{unf}}(\lambda) \leq \text{Adv}_{\text{KVAC}_{\text{AT}}}^{\text{omuf}}(\lambda, 1) + q\text{Adv}_{\text{ZKP}}^{\text{ksnd}}(\lambda) + q\text{Adv}_{\text{KVAC}_{\text{AT}}}^{\text{anon}}(\lambda) + \text{Adv}_{\text{PRF}}^{\text{prnd}}(\lambda) + \frac{q^2}{|\mathbb{M}|} ,$$

(where q is the number of oracle queries) and unlinkability advantage:

$$\text{Adv}_{\text{cNYM}[\text{KVAC}, \text{PRF}]}^{\text{unlink}}(\lambda) \leq 2(\text{Adv}_{\text{KVAC}_{\text{AT}}}^{\text{anon}}(\lambda, 1) + \text{Adv}_{\text{PRF}}^{\text{prnd}}(\lambda)) .$$

8.3.3 Analysis

Correctness follows by inspection; unlinkability follows directly from pseudorandomness and anonymity. We focus on unforgeability.

Lemma 8.13. *Let Kvac be a keyed-verification credential for $n = 1$ attributes, with re-randomizable issuance and labelled presentation, for predicate family $\Phi \supset \{\phi_{\text{prf}}\}$. Let PRF be a pseudorandom function compatible with Kvac. Then, $\text{cNYM}[\text{KVAC}, \text{PRF}]$ is a keyed-verification pseudonym system with unforgeability advantage:*

$$\text{Adv}_{\text{cNYM}[\text{KVAC}, \text{PRF}]}^{\text{unf}}(\lambda) \leq q\text{Adv}_{\text{KVAC}}^{\text{ext}}(\lambda, 1) + q\text{Adv}_{\text{KVAC}}^{\text{anon}}(\lambda) + q\text{Adv}_{\text{PRF}}^{\text{prnd}}(\lambda) + \frac{q^2}{|\mathbb{M}|} ,$$

where q is the number of queries to the issuance oracles.

Proof. We consider, without loss of generality, two types of adversaries:

- a “one-more” adversary $A_1(pp)$ that returns $(scp, (nym_i^*, m_i^*, \rho_i^*)_{i=1}^{q+1})$, such that $(nym_i)_i$ are all different and q is the number of queries to the **ISSUE** oracle. This attacker can be reduced to extractability of the underlying keyed-verification credential system. To do so, we build a reduction B for the extraction game $\text{EXT}_{\text{KVAC,Ext,A}}(\lambda, 1)$.

B receives as input pp and internally runs A_1 , mapping each oracle query to the respective one in the extractability game:

- each issuance query with adversarial user message μ is forwarded as $\text{ISSUE}(\phi_{\text{prf}}^{scp, nym}, \mu)$
- each presentation query of the form $(scp, nym, m), \rho$ is forwarded as $\text{PRESENT}(\phi_{\text{prf}}^{scp, nym} \wedge \phi_{\text{lbl}}^m, \rho)$
- each new user query has B sample $k \leftarrow \$ \mathbb{M}$, runs $\text{NEWUSR}(k)$ and upon receiving a counter ctr for the given user, it stored $\text{Keys}[ctr] := k$. Finally, returns ctr .
- each user presentation query of the form (i, scp, m) is forwarded to the extractability oracle as $\text{PRESENTUSR}(i, \phi_{\text{prf}}^{scp, nym} \wedge \phi_{\text{lbl}}^m)$ where $nym := \text{NYM.E}(\text{Keys}[i], scp)$. (If the index is invalid, we assume the oracle returns $\perp$).
- finally, for each $\text{EVAL}(i, scp)$, returns $\text{PRF}(\text{Keys}[i], scp)$.

The oracles provide the exact same output distribution as in the original game. At the end, the adversary B guesses an index $j \in [q + 1]$ and returns $(scp^*, nym_j^*, m_j^*, \rho_j^*)$.

Since there are at most q different issuance queries, at most q keys have been issued. However, the adversary produced $q + 1$ different evaluations of the function $\text{PRF}(\cdot, scp)$. This implies that either one of the keys is not present as a part of the issued credentials (and the winning condition “ $\vec{m} \notin Qrs$ ” in the extractability game is satisfied) or an extracted credential does not satisfy the pseudonym predicate (so the condition “ $\phi(\vec{m}) = 0$ ” in the extractability game is satisfied).

- an adversary stealing pseudonyms from honest users $A_2(pp)$ that returns a valid forgery $(scp^*, nym^*, m^*, \rho^*)$ not previously queried, and such that there exists an index $j \in [q_u]$ (q_u being the number of queries to NEWUSR) such that $nym^* = \text{NYM.E}(\text{Keys}[j], scp^*)$. This can be proven via a hybrid argument:

H_0 the first hybrid runs the extractor for the credential presentation and checks if the extracted values satisfy the predicate. Similarly to the previous argument, if the extracted values are not correct PRF evaluation then it is possible to build a reduction to extractability of the underlying credential system.

$H_1, \dots, H_{q_u}$ replace the i -th query to $\text{NEWUSR}()$ and queries of the form $\text{PRESENTUSR}(i, \cdot)$ to simulated responses. Upon receiving a query $\text{NEWUSR}()$, sample $k_i \leftarrow \$ \mathbb{M}$ and run $st_i \leftarrow \text{Sim.I}(pp, \phi_1)$. Store k_i in Keys and the simulator state st_i in Creds . Upon receiving a query $\text{PRESENTUSR}(i, scp, m)$, run the pseudonym evaluation using k_i and return the simulated proof via $\text{Sim.P}(pp, \phi_{\text{prf}}^{scp, nym} \wedge \phi_{\text{lbl}}^m)$. Evaluation queries are dealt as before using the relative PRF key.

If the adversary’s output is distinguishable with non-negligible probability then it is possible to build a distinguisher for anonymity of KVAC.

$H_{q_u+1}, \dots, H_{2q_u}$ replace all pseudonyms with uniformly-random PRF images. The challenger now holds a table Nyms of evaluations and each $\text{EVAL}(i, scp)$ query is responded as follows: if $\text{Nyms}[i, scp]$ has associated a pseudonym already, then return it; if no such entry exists, then sample a fresh pseudonym $nym \leftarrow \$ \mathbb{I}$, where $\mathbb{I}$ is the image of the PRF.

At this point we have that the returned pseudonyms collides with a random string only if

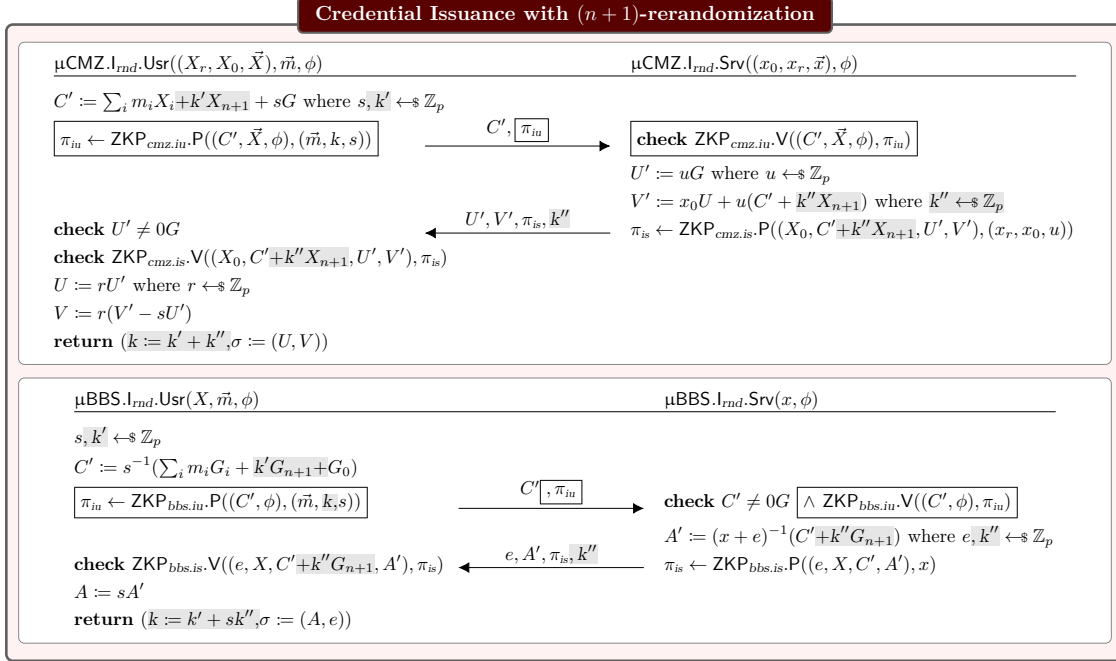


Figure 14: Randomized issuance of μCMZ and μBBS credentials. The relation π_{is} is defined in Equations (10) and (25) for μCMZ and μBBS respectively. Highlighted, the differences with the vanilla protocol; boxed, the parts that may be removed for one-more unforgeability.

there is a collision within PRF. The probability that this bad event occurs is upper bounded by $q^2/|\mathbb{I}|$. □

8.3.4 Instantiation

A suitable PRF that composes well with the studied keyed-verification systems is the PRF from Naor, Pinkas, and Reingold [NPR99]:

$$\text{NPR}(k, \text{scp}) := k \cdot H_{\mathbb{G}}(\text{scp}) \quad (39)$$

The PRF is secure in the random oracle model under the DDH assumption: distinguishing a tuple $(H_{\mathbb{G}}(\text{scp}_0), kH_{\mathbb{G}}(\text{scp}_0), H_{\mathbb{G}}(\text{scp}_1), kH_{\mathbb{G}}(\text{scp}_1))$ from a uniformly random element of $\mathbb{G}^4$ is equivalent to solving the DDH problem: given a DDH challenge (with generator) (P, U, Q, V) , set $H_{\mathbb{G}}(\text{scp}_0) = P$, $H_{\mathbb{G}}(\text{scp}_1) = Q$, and $k = \log_P U$.

8.3.5 One-time linking

It is also possible to upgrade to statistical anonymity in a different model. Instead of providing a *nym*, the user can commit to it and prove it has the same value of a commitment sent in a previous interaction. Given two pseudonyms $C_0 = kH_{\mathbb{G}}(\text{scp}) + rH$ and $C_1 = kH_{\mathbb{G}}(\text{scp}) + r'H$ the user can prove knowledge of the discrete logarithm of $C_0 - C_1$ base H and that C_0, C_1 are correct commitments to *nym* evaluations. The advantage of this last approach is that the user has perfect hiding.

9 Straight-line extraction from Σ -protocols

The notion of soundness required for credentials that we use is strong, as it demands that the simulator is able to extract a witness from an oracle query in order to properly answer the challenge. In the literature, it has sometimes been called online [Fis05] or straight-line [GM06; Sah99] extraction, and is believed not to be satisfied by traditional non-interactive Schnorr proofs relying on rewinding techniques.

The Fiat–Shamir transform of a Σ -protocol is however the most common tool for instantiating the NIZK in credential systems. To complement this heuristic and resolve the theoretical gap, we prove that representation proofs are straight-line extractable in the algebraic group model.

We denote with Σ such a protocol for generic linear relations (m of them) over vectors of size n :

$$R_{\mathbf{F}} := \left\{ (\vec{X}, \vec{x}) \in \mathbb{Z}_p^n \times \mathbb{G}^m : \sum_k x_k F_{1,k} = X_1 \wedge \cdots \wedge \sum_k x_k F_{m,k} = X_m \right\}$$

We will employ the more compact matrix-vector notation $\mathbf{F}\vec{x} = \vec{X}$ where $\mathbf{F} \in \mathbb{G}^{m \times n}$ denotes the matrix whose j -th row vector is $\vec{F}_j$. Denote with $\text{Uniq}(\mathbf{F})$ the list $G_1, \dots, G_\ell$ with $\ell \leq |\mathbf{F}|$ the non-trivial group elements appearing in $\mathbf{F}$, repeated only once.

Definition 9.1. *A relation family $R_{\mathbf{F}}$ is admissible if: for any p.p.t. adversary A , it is computationally hard to find a non-zero vector in $\ker(\vec{F}_j)$ (for $j \in [m]$) or in $\ker(\text{Uniq}(\mathbf{F}))$, where Uniq denotes the set of non-trivial group elements appearing in $\mathbf{F}$.*

We recall here the protocol Σ for the relation $R_{\mathbf{F}}$:

Procedure $\Sigma.P(\vec{X}, \vec{x})$	Procedure $\Sigma.V(\vec{X}, (\vec{R}, \vec{s}))$
$\vec{R} := \mathbf{F}\vec{r}$ where $r \leftarrow \mathbb{Z}_p^n$	$c := H_p(\vec{X}, \vec{R})$
$c := H_p(\vec{X}, \vec{R})$	return $\mathbf{F}\vec{s} = R + c\vec{X}$
$\vec{s} := \vec{r} + c\vec{x}$	
return $(\vec{R}, \vec{s})$	

in the above, the notation $\mathbf{F}\vec{r}$ indicates standard matrix-vector product, i.e., $\vec{R}$ is the vector of m elements indexed in $j \in [m]$ such that $R_j := \sum_k F_{j,k} r_k$. Similarly one proceeds for $\mathbf{F}\vec{s}$. This protocol is a standard Fiat–Shamir transform for a Σ -protocol proving knowledge of a group morphism [Mau09]. The transform as presented here is sometimes also known as “strong Fiat–Shamir transform” [BPW12]. It is a well-known result in the literature that such protocol is zero-knowledge in the random oracle model: the simulator samples $\vec{s}, c$ at random from $\mathbb{Z}_p^n, \mathbb{Z}_p$ respectively and sets $\vec{R} := \mathbf{F}\vec{s} - c\vec{X}$. Finally, it programs the random oracle to respond with c for the query $(\vec{X}, \vec{R})$ finally returns $(\vec{R}, \vec{s})$. The proof distribution is identical to the one generated by the prover: $\vec{R}$ and $\vec{s}$ are uniformly distributed satisfying the verification equation.

Theorem 9.2. *The protocol Σ for an admissible relation $R_{\mathbf{F}}$ is strongly simulation-extractable in the algebraic group model and the random oracle model.*

Proof. Let $\mathbf{A}$ be the algebraic adversary in the game for simulation extractability, and q the number of queries made to the random oracle during its execution. For a statement $(\mathbf{F}, \vec{X})$, let $\vec{F}_j$ denote the j -th row of the matrix $\mathbf{F}$ and X_j the j -th element of X . During its execution, the adversary $\mathbf{A}$ queries the simulation oracle for instance $\vec{X} \in \mathbb{G}^m$ and witness $\vec{x} \in \mathbb{Z}_p^n$. Being algebraic, $\mathbf{A}$ also provides an algebraic representation of $\vec{X}$. Upon receiving the i -th simulation query, the simulator picks $c_i, \vec{s}_i \leftarrow \mathbb{Z}_p$, sets

$$\vec{R}_i := \mathbf{F} \vec{s}_i - c_i \vec{X}_i \quad (40)$$

and programs the random oracle to respond to $H_p(\vec{X}_i, \vec{R}_i)$ with c_i . If such query exists, the simulator aborts and the adversary wins. Since R_i is uniformly distributed over $\mathbb{G}$, no such query has been previously made except with probability at most q/p which is negligible. Finally, the simulator returns $(\vec{R}_i, \vec{s}_i)$. The algebraic representation (χ_i, δ_i) of the i -th queried statement $\vec{X}_i$ is such that:

$$X_{i,j} = \sum_k^\ell \chi_{i,j,k} G_k + \sum_{\substack{\iota < i \\ \kappa \in [m]}} \delta_{i,\iota,\kappa} R_{\iota,\kappa}$$

where the generators $G_1, \dots, G_\ell = \text{Uniq}(\mathbf{F})$ are the group elements appearing in $\mathbf{F}$. The elements $R_{k,j}$ can be simplified thanks to Equation (40). In fact, for the first query, we have that for every element $j \in [m]$:

$$X_{1,j} = \sum_k^\ell \alpha_{1,j,k} G_k \text{ where } \alpha_{1,j,k} = \chi_{1,j,k}$$

since no previous query has been made. For the second query we can consider:

$$X_{2,j} = \sum_k^\ell \alpha_{2,j,k} G_k \text{ where } \alpha_{2,j,k} = \chi_{2,j,k} + \delta_{2,1,k}(s_1 - c_1 \alpha_{1,j,k})$$

and so on. We can partition the indices $k \in [\ell]$ into I_j – the set of indices of $\text{Uniq}(\mathbf{F})$ appearing in $\vec{F}_j$ and $\bar{I}_j$ the set of indices that do not. We can thus see the i -th simulation query ($i \in [q]$) as:

$$X_{i,j} = \sum_{k \in I_j} \alpha_{i,j,k} G_k + \sum_{k \in \bar{I}_j} \alpha_{i,j,k} G_k$$

for all $j \in [m]$, with $\alpha_{i,j,k}$ in $\mathbb{Z}_p$. At the end of the execution the adversary returns a final statement $\vec{X}^*$ and a proof $\pi^* = (\vec{K}^*, \vec{s}^*)$ such that $(\vec{X}^*, \vec{R}^*, \vec{s}^*) \neq (\vec{X}_i, \vec{R}_i, \vec{s}_i)$ for all $i \in [q]$ satisfying $\mathbf{F} \vec{s}^* = \vec{K}^* + c^* \vec{X}^*$ and $c^* = H_p(\vec{X}^*, \vec{R}^*)$.

We claim that $(\vec{X}^*, \vec{R}^*) \neq (\vec{X}_i, \vec{R}_i)$ for all $i \in [q]$. In fact, by contradiction, if $\exists i \in [q]$ such that $\vec{s}_i \neq \vec{s}^*$ and $\vec{X}_i = \vec{X}^*, \vec{R}_i = \vec{R}^*$ then $\mathbf{F}(\vec{s}_i - \vec{s}^*) = \vec{0}$ and non-trivial element of the kernel was found. From the algebraic representation we have that, for each $j \in [m]$

$$\begin{aligned} \vec{R}_j^* &= \sum_{k \in I_j} \beta_{j,k}^* G_k + \sum_{k \in \bar{I}_j} \bar{\beta}_{j,k}^* G_k \\ \vec{X}_j^* &= \sum_{k \in I_j} \alpha_{j,k}^* G_k + \sum_{k \in \bar{I}_j} \bar{\alpha}_{j,k}^* G_k \end{aligned} \quad (41)$$

where the algebraic representation already reduced the terms R_i as linear combinations of the G_k 's in $I_j \cup \bar{I}_j = \text{Uniq}(\mathbf{F})$ using the verification equation as shown above. We distinguish the following possibilities:

1. If $\exists j^* \in [m], k^* \in \bar{I}_{j^*}$ s.t. $\bar{\alpha}_{j^*,k^*}^* \neq 0$, then plugging Equation (41) into the verification equation we have that:

$$\sum_{k \in I_{j^*}} (\beta_{j^*,k}^* + c^* \cdot \alpha_{j^*,k}^* - s_k^*) G_k + \sum_{k \in \bar{I}_{j^*} \setminus \{k^*\}} (\bar{\beta}_{j^*,k}^* + c^* \bar{\alpha}_{j^*,k}^*) G_k + (\bar{\beta}_{j^*,k^*}^* + c^* \bar{\alpha}_{j^*,k^*}^*) G_{k^*} = 0$$

and the coefficient of G_{k^*} is uniformly distributed and non-zero with overwhelming probability since c^* is uniformly distributed and $\bar{\alpha}_{j^*,k^*}^* \neq 0, \bar{\beta}_{j^*,k^*}^*$ are chosen before seeing the challenge. Therefore, we have a non-trivial relation of $\text{Uniq}(\mathbf{F})$.

2. If $\forall i, j, k: \bar{\alpha}_{j,k}^* = 0$ modulo p then we can rearrange the indices of $\alpha_{j,k}^*$ and can consider vectors $\vec{\alpha}_j$ ($j \in [m]$) such that $X_1 = \langle F_1, \vec{\alpha}_1 \rangle, \dots, X_m = \langle F_m, \vec{\alpha}_m \rangle$. We claim that $\vec{\alpha}_1, \dots, \vec{\alpha}_m$ can be used to reconstruct a unique witness vector.

- (a) If $\exists j \neq j'$ such that for some $k \in I_j \cap I_{j'}: \alpha_{j,k} \neq \alpha_{j',k}$ then

$$\vec{F}_j \cdot (\vec{\beta}_j + c^* \vec{\alpha}_j - \vec{s}^*) = 0 \text{ and } \vec{F}_{j'} \cdot (\vec{\beta}_{j'} + c^* \vec{\alpha}_{j'} - \vec{s}^*) = 0$$

and the adversary found another nontrivial relation of the kernel of $\vec{F}_j$ or $\vec{F}_{j'}$: if $(\vec{\beta}_j + c^* \vec{\alpha}_j - \vec{s}^*) \neq 0$ we are done, and if it is instead equal to zero then the term $(\vec{\beta}_{j'} + c^* \vec{\alpha}_{j'} - \vec{s}^*)$ will be also zero only with negligible probability $1/p$ since $(\vec{\beta}_{j'} - \vec{\beta}_j) + c(\vec{\alpha}_{j'} - \vec{\alpha}_j) = 0$ only if the adversary guessed c correctly (note that all the terms are sent to the random oracle before seeing the challenge).

- (b) Otherwise, the extractor defines $\vec{x}^* \in \mathbb{Z}_p^n$ as the vector whose k -th entry is the (unique) element $\alpha_{j,k}$ for all $j \in [m]$ such that $k \in I_j$. The witness is correct since, by construction, $\vec{F}_j \vec{x}^* = \vec{F}_j \vec{\alpha}_j = X_j$ for all $j \in [m]$.

The witness output from Item 2b is correct, and all other cases happen with negligible probability, thus the protocol is strongly simulation-extractable. $\square$

10 Acknowledgements

This work was partially supported by Agence Nationale de la Recherche (ANR) project ANR-24-CE39-6061-01A2C. We are grateful to Trevor Perrin, Dennis Jackson (Firefox). Jacques Traoré (Orange) identified a bug in the initial version of μ CMZ. Geoffroy Couteau (CNRS), Greg Zaverucha (Microsoft Research), Georg Fuchsbauer (TU Wien), Balthazar Bauer (UVSQ), Alexander Koch (IRIF), Jonathan Katz (Google) provided valuable feedback on the manuscript.

References

- [0xP] 0xPARC Foundation. *Zupass: Zuzalu Passport*. URL: <https://github.com/proofcarringdata/zupass>.
- [AFGHO16] M. Abe, G. Fuchsbauer, J. Groth, K. Haralambiev, and M. Ohkubo. “Structure-Preserving Signatures and Commitments to Group Elements”. In: *Journal of Cryptology* 29.2 (2016), pp. 363–421. DOI: 10.1007/s00145-014-9196-7.
- [AHIV17] S. Ames, C. Hazay, Y. Ishai, and M. Venkitasubramaniam. “Ligero: Lightweight Sublinear Arguments Without a Trusted Setup”. In: *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS 2017*. ACM, 2017. DOI: 10.1145/3133956.3134104.
- [Ano] Anon Aadhaar developers. *Anon Aadhaar: zero-knowledge protocol that allows Aadhaar ID owners to prove their identity in a privacy preserving way*. URL: <https://github.com/anon-aadhaar/anon-aadhaar>.
- [App] Apple Inc. *iCloud Private Relay Overview*. URL: https://www.apple.com/icloud/docs/iCloud_Private_Relay_Overview_Dec2021.pdf.
- [ASM06] M. H. Au, W. Susilo, and Y. Mu. “Constant-Size Dynamic k-TAA”. In: *Proceedings of the 5th International Conference on Security and Cryptography for Networks*. SCN ’06. 2006, pp. 111–125. DOI: 10.1007/11832072_8.
- [AYY25] G. Amjad, K. Yeo, and M. Yung. “RSA Blind Signatures with Public Metadata”. In: *Proceedings on Privacy Enhancing Technologies* 2025.1 (2025), pp. 37–57. DOI: 10.56553/POPETS-2025-0004.
- [BBDT17] A. Barki, S. Brunet, N. Desmoulins, and J. Traoré. “Improved Algebraic MACs and Practical Keyed-Verification Anonymous Credentials”. In: *Proceedings of the 23rd International Conference on Selected Areas in Cryptography*. SAC ’16. 2017, pp. 360–380.
- [BBS04] D. Boneh, X. Boyen, and H. Shacham. “Short Group Signatures”. In: *Proceedings of the 24th Annual International Cryptology Conference*. CRYPTO ’04. 2004, pp. 41–55. DOI: 10.1007/978-3-540-28628-8_3.
- [BCKL08] M. Belenkiy, M. Chase, M. Kohlweiss, and A. Lysyanskaya. “P-signatures and Noninteractive Anonymous Credentials”. In: *Proceedings of the 5th Theory of Cryptography Conference*. TCC ’08. 2008, pp. 356–374. DOI: 10.1007/978-3-540-78524-8_22.

- [BCS16] E. Ben-Sasson, A. Chiesa, and N. Spooner. “Interactive Oracle Proofs”. In: *Proceedings of the 14th International Conference on Theory of Cryptography*. TCC ’16. 2016, pp. 31–60. DOI: 10.1007/978-3-662-53644-5_2.
- [BDFG20] D. Boneh, J. Drake, B. Fisch, and A. Gabizon. *Efficient polynomial commitment schemes for multiple points and polynomials*. Cryptology ePrint Archive, Paper 2020/081. <https://eprint.iacr.org/2020/081>. 2020. URL: <https://eprint.iacr.org/2020/081>.
- [BEKRS21] J. Bobolz, F. Eidens, S. Krenn, S. Ramacher, and K. Samelin. “Issuer-Hiding Attribute-Based Credentials”. In: *Proceedings of the 20th International Conference on Cryptology and Network Security*. CANS ’21. 2021, pp. 158–178. DOI: 10.1007/978-3-030-92548-2_9.
- [Ber06] D. J. Bernstein. “Curve25519: New Diffie-Hellman Speed Records”. In: *Proceedings of the 9th International Conference on Practice and Theory in Public Key Cryptography*. PKC ’06. 2006, pp. 207–228. DOI: 10.1007/11745853_14.
- [BLCL91] G. Brassard, S. Laplante, C. Crépeau, and C. Léger. “Computationally convincing proofs of knowledge”. In: *STACS 91*. Ed. by C. Choffrut and M. Jantzen. Berlin, Heidelberg: Springer Berlin Heidelberg, 1991, pp. 251–262. ISBN: 978-3-540-47002-1.
- [BLS04] P. S. L. M. Barreto, B. Lynn, and M. Scott. “On the Selection of Pairing-Friendly Groups”. In: *Proceedings of the 10th International Conference on Selected Areas in Cryptography*. SAC ’03. 2004, pp. 17–25. DOI: 10.1007/978-3-540-24654-1_2.
- [BN06] P. S. L. M. Barreto and M. Naehrig. “Pairing-Friendly Elliptic Curves of Prime Order”. In: *Proceedings of the 12th International Conference on Selected Areas in Cryptography*. SAC ’05. 2006, pp. 319–331. DOI: 10.1007/11693383_22.
- [BPW12] D. Bernhard, O. Pereira, and B. Warinschi. “How Not to Prove Yourself: Pitfalls of the Fiat-Shamir Heuristic and Applications to Helios”. In: *Proceedings of the 18th International Conference on the Theory and Application of Cryptology and Information Security*. ASIACRYPT ’12. 2012, pp. 626–643. DOI: 10.1007/978-3-642-34961-4_38.
- [Bra00] S. Brands. *Rethinking public key infrastructures and digital certificates: building in privacy*. Mit Press, 2000.
- [Bra95] S. Brands. “Off-Line Electronic Cash Based on Secret-Key Certificates”. In: *Proceedings of the 2nd Latin American Symposium on Theoretical Informatics*. LATIN ’95. 1995, pp. 131–166. DOI: 10.1007/3-540-59175-3_86.
- [CBCMRW24] M. Christ, F. Baldimtsi, K. K. Chalkias, D. Maram, A. Roy, and J. Wang. “SoK: Zero-Knowledge Range Proofs”. In: *Proceedings of the 6th Conference on Advances in Financial Technologies*. AFT ’24. 2024, 14:1–14:25. DOI: 10.4230/LIPIcs.AFT.2024.14.
- [CCLMR21] K. Chalkias, S. Cohen, K. Lewi, F. Moezinia, and Y. Romainier. “HashWires: Hyper-efficient Credential-Based Range Proofs”. In: *Proceedings on Privacy Enhancing Technologies* 2021.4 (2021), pp. 76–95. DOI: 10.2478/popets-2021-0061.

- [CDVW24] S. Celi, A. Davidson, S. Valdez, and C. A. Wood. *Privacy Pass Issuance Protocols*. RFC 9578. 2024. DOI: 10.17487/RFC9578. URL: <https://www.rfc-editor.org/info/rfc9578>.
- [CFQ19] M. Campanelli, D. Fiore, and A. Querol. “LegoSNARK: Modular Design and Composition of Succinct Zero-Knowledge Proofs”. In: *Proceedings of the 26th ACM Conference on Computer and Communications Security*. CCS ’19. 2019, pp. 2075–2092. DOI: 10.1145/3319535.3339820.
- [CGKR22] G. Couteau, D. Goudarzi, M. Klooß, and M. Reichle. “Sharp: Short Relaxed Range Proofs”. In: *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*. CCS ’22. 2022, pp. 609–622. DOI: 10.1145/3548606.3560628.
- [Cha82] D. Chaum. “Blind Signatures for Untraceable Payments”. In: *Proceedings of the 2nd Annual International Cryptology Conference*. CRYPTO ’82. 1982, pp. 199–203. DOI: 10.1007/978-1-4757-0602-4_18.
- [Cha85] D. Chaum. “Security without identification: Transaction systems to make big brother obsolete”. In: *Communications of the ACM* 28.10 (1985), pp. 1030–1044.
- [Che06] J. H. Cheon. “Security Analysis of the Strong Diffie-Hellman Problem”. In: *Proceedings of the 25th Annual International Conference on the Theory and Applications of Cryptographic Techniques*. EUROCRYPT ’06. 2006, pp. 1–11. DOI: 10.1007/11761679\1.
- [Chi10] A. Chiesa. “Proof-carrying data”. PhD thesis. Massachusetts Institute of Technology, 2010.
- [CHKLM06] J. Camenisch, S. Hohenberger, M. Kohlweiss, A. Lysyanskaya, and M. Meyerovich. “How to win the clonewars: Efficient periodic n-times anonymous authentication”. In: *Proceedings of the 13th ACM Conference on Computer and Communications Security*. CCS ’06. 2006, pp. 201–210. DOI: 10.1145/1180405.1180431.
- [CHMMVW20] A. Chiesa, Y. Hu, M. Maller, P. Mishra, N. Vesely, and N. Ward. “Marlin: Preprocessing zkSNARKs with Universal and Updatable SRS”. In: *Proceedings of the 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques*. EUROCRYPT ’20. 2020, pp. 738–768. DOI: 10.1007/978-3-030-45721-1_26.
- [CKLMNP16] J. Camenisch, S. Krenn, A. Lehmann, G. L. Mikkelsen, G. Neven, and M. Ø. Pedersen. “Formal Treatment of Privacy-Enhancing Credential Systems”. In: *Proceedings of the 22nd International Conference on Selected Areas in Cryptography*. SAC ’15. 2016, pp. 3–24. DOI: 10.1007/978-3-319-31301-6_1.
- [CL01] J. Camenisch and A. Lysyanskaya. “An Efficient System for Non-transferable Anonymous Credentials with Optional Anonymity Revocation”. In: *Proceedings of the 20th Annual International Conference on Theory and Application of Cryptographic Techniques*. EUROCRYPT ’01. 2001, pp. 93–118. DOI: 10.1007/3-540-44987-6\7.

- [CL03] J. Camenisch and A. Lysyanskaya. “A Signature Scheme with Efficient Protocols”. In: *Proceedings of the 3rd International Conference on Security in Communication Networks*. SCN ’02. 2003, pp. 268–289. DOI: 10.1007/3-540-36413-7_20.
- [CL06] M. Chase and A. Lysyanskaya. “On Signatures of Knowledge”. In: *Proceedings of the 26th Annual International Cryptology Conference*. CRYPTO ’06. 2006, pp. 78–96. DOI: 10.1007/11818175_5.
- [CL19] E. C. Crites and A. Lysyanskaya. “Delegatable Anonymous Credentials from Mercurial Signatures”. In: *Proceedings of the The Cryptographers’ Track at the RSA Conference 2019*. CT-RSA 2019. 2019, pp. 535–555. DOI: 10.1007/978-3-030-12612-4_27.
- [CMZ14] M. Chase, S. Meiklejohn, and G. Zaverucha. “Algebraic MACs and Keyed-Verification Anonymous Credentials”. In: *Proceedings of the 21st ACM Conference on Computer and Communications Security*. CCS ’14. 2014, pp. 1205–1216. DOI: 10.1145/2660267.2660328.
- [CPZ20] M. Chase, T. Perrin, and G. Zaverucha. “The Signal Private Group System and Anonymous Credentials Supporting Efficient Verifiable Encryption”. In: *Proceedings of the 27th ACM Conference on Computer and Communications Security*. CCS ’20. 2020, pp. 1445–1459. DOI: 10.1145/3372297.3417887.
- [CR19] G. Couteau and M. Reichle. “Non-interactive Keyed-Verification Anonymous Credentials”. In: *Proceedings of the 22nd International Conference on Practice and Theory in Public Key Cryptography*. PKC ’19. 2019, pp. 66–96. DOI: 10.1007/978-3-030-17253-4_3.
- [CV02] J. Camenisch and E. Van Herreweghen. “Design and Implementation of The Idemix Anonymous Credential System”. In: *Proceedings of the 9th ACM Conference on Computer and Communications Security*. CCS ’02. 2002, pp. 21–30. DOI: 10.1145/586110.586114.
- [DG23] Q. Dao and P. Grubbs. “Spartan and Bulletproofs are Simulation-Extractable (for Free!)” In: *Proceedings of the 42nd Annual International Conference on the Theory and Applications of Cryptographic Techniques*. EUROCRYPT ’23. 2023, pp. 531–562. DOI: 10.1007/978-3-031-30617-4_18.
- [DGSTV18] A. Davidson, I. Goldberg, N. Sullivan, G. Tankersley, and F. Valsorda. “Privacy Pass: Bypassing Internet Challenges Anonymously”. In: *Proceedings on Privacy Enhancing Technologies* 2018.3 (2018), pp. 164–180. DOI: 10.1515/popets-2018-0026.
- [DMTV24] F. B. Durak, L. Marco, A. Talayhan, and S. Vaudenay. “Non-Transferable Anonymous Tokens by Secret Binding”. In: *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*. CCS ’24. 2024, pp. 2460–2474. DOI: 10.1145/3658644.3670338. URL: <https://doi.org/10.1145/3658644.3670338>.
- [DS] P. by Design Foundation and SIDN. *IRMA server, client, and tooling*. URL: <https://github.com/privacybydesign/irmago>.

- [DVC22] F. B. Durak, S. Vaudenay, and M. Chase. *Anonymous Tokens with Hidden Metadata Bit from Algebraic MACs*. Cryptology ePrint Archive, Report 2022/1622. 2022. URL: <https://eprint.iacr.org/2022/1622>.
- [DY05] Y. Dodis and A. Yampolskiy. “A Verifiable Random Function with Short Proofs and Keys”. In: *Proceedings of the 8th International Workshop on Practice and Theory in Public Key Cryptography*. PKC ’05. 2005, pp. 416–431. DOI: 10.1007/978-3-540-30580-4_28.
- [EKR24] L. Eagen, S. Kanjalkar, T. Ruffing, and J. Nick. “Bulletproofs++: Next Generation Confidential Transactions via Reciprocal Set Membership Arguments”. In: *Proceedings of the 43rd Annual International Conference on the Theory and Applications of Cryptographic Techniques*. EUROCRYPT ’24. 2024, pp. 249–279. DOI: 10.1007/978-3-031-58740-5_9.
- [Ern+24] J. Ernstberger et al. “zk-Bench: A Toolset for Comparative Evaluation and Performance Benchmarking of SNARKs”. In: *Proceedings of the 14th International Conference on Security and Cryptography for Networks*. SCN ’24. 2024, pp. 46–72. DOI: 10.1007/978-3-031-71070-4_3.
- [Eth] Ethereum Foundation. *Semaphore: A zero-knowledge protocol for anonymous interactions*. URL: <https://semaphore.pse.dev/>.
- [FHS19] G. Fuchsbaauer, C. Hanser, and D. Slamanig. “Structure-Preserving Signatures on Equivalence Classes and Constant-Size Anonymous Credentials”. In: *Journal of Cryptology* 32.2 (2019), pp. 498–546. DOI: 10.1007/s00145-018-9281-4.
- [Fis05] M. Fischlin. “Communication-Efficient Non-interactive Proofs of Knowledge with Online Extractors”. In: *Proceedings of the 25th Annual International Cryptology Conference*. CRYPTO ’05. 2005, pp. 152–168. DOI: 10.1007/11535218_10.
- [FKL18] G. Fuchsbaauer, E. Kiltz, and J. Loss. “The Algebraic Group Model and its Applications”. In: *Proceedings of the 38th Annual International Cryptology Conference*. CRYPTO ’18. 2018, pp. 33–62. DOI: 10.1007/978-3-319-96881-0_2.
- [Fs24] M. Frigo and abhi shelat. *Anonymous credentials from ECDSA*. 2024. URL: <https://eprint.iacr.org/2024/2010>.
- [FS86] A. Fiat and A. Shamir. “How to prove yourself: practical solutions to identification and signature problems”. In: *Proceedings of the 6th Annual International Cryptology Conference*. CRYPTO ’86. 1986, pp. 186–194. DOI: 10.1007/3-540-47721-7_12.
- [FW24] G. Fuchsbaauer and M. Wolf. “Concurrently Secure Blind Schnorr Signatures”. In: *Proceedings of the 43rd Annual International Conference on the Theory and Applications of Cryptographic Techniques*. EUROCRYPT ’24. 2024, pp. 124–160. DOI: 10.1007/978-3-031-58723-8_5.
- [GMR89] S. Goldwasser, S. Micali, and C. Rackoff. “The Knowledge Complexity of Interactive Proof Systems”. In: *SIAM Journal on Computing* 18.1 (1989), pp. 186–208.
- [GMY06] J. A. Garay, P. D. MacKenzie, and K. Yang. “Strengthening Zero-Knowledge Protocols Using Signatures”. In: *Journal of Cryptology* 19.2 (2006), pp. 169–209. DOI: 10.1007/s00145-005-0307-3.

- [Goo] Google LLC. *Google Cloud VPN*. URL: <https://one.google.com/about/vpn/howitworks>.
- [Gro15] J. Groth. “Efficient Fully Structure-Preserving Signatures for Large Messages”. In: *Proceedings of the 21st International Conference on the Theory and Application of Cryptology and Information Security*. ASIACRYPT ’15. 2015, pp. 239–259. DOI: 10.1007/978-3-662-48797-6_11.
- [GS08] J. Groth and A. Sahai. “Efficient Non-interactive Proof Systems for Bilinear Groups”. In: *Proceedings of the 27th Annual International Conference on the Theory and Applications of Cryptographic Techniques*. EUROCRYPT ’08. 2008, pp. 415–432. DOI: 10.1007/978-3-540-78967-3_24.
- [Ham15] M. Hamburg. “Decaf: Eliminating Cofactors Through Point Compression”. In: *Proceedings of the 35th Annual International Cryptology Conference*. CRYPTO ’15. 2015, pp. 705–723. DOI: 10.1007/978-3-662-47989-6_34.
- [HIPVW24] S. Hendrickson, J. Iyengar, T. Pauly, S. Valdez, and C. A. Wood. *Rate-Limited Token Issuance Protocol*. Internet-Draft draft-ietf-privacypass-rate-limit-tokens-06. Work in Progress. Internet Engineering Task Force, 2024. 52 pp. URL: <https://datatracker.ietf.org/doc/draft-ietf-privacypass-rate-limit-tokens/06/>.
- [Hypa] Hyperledger Anoncreds Contributors. *Hyperledger Anoncreds CL Signatures*. URL: <https://github.com/hyperledger/anoncreds-clsignatures-rs>.
- [Hypb] Hyperledger Fabric Maintainers. *Hyperledger Fabric’s MSP Implementation with Identity Mixer*. URL: <https://github.com/hyperledger/fabric/blob/main/docs/source/idemix.rst>.
- [JKK14] S. Jarecki, A. Kiayias, and H. Krawczyk. “Round-Optimal Password-Protected Secret Sharing and T-PAKE in the Password-Only Model”. In: *Proceedings of the 20th International Conference on the Theory and Application of Cryptology and Information Security*. ASIACRYPT ’14. 2014, pp. 233–253. DOI: 10.1007/978-3-662-45608-8_13.
- [JY09] D. Jao and K. Yoshida. “Boneh-Boyen Signatures and the Strong Diffie-Hellman Problem”. In: *Pairing-Based Cryptography – Pairing 2009*. Ed. by H. Shacham and B. Waters. Berlin, Heidelberg: Springer Berlin Heidelberg, 2009, pp. 1–16. ISBN: 978-3-642-03298-1.
- [KB] V. Kalos and G. M. Bernstein. *Blind BBS Signatures*. Internet-Draft draft-kalos-bbs-blind-signatures-03. Work in Progress. Internet Engineering Task Force. URL: <https://datatracker.ietf.org/doc/draft-kalos-bbs-blind-signatures/03/>.
- [KB25] V. Kalos and G. M. Bernstein. *BBS per Verifier Linkability*. Internet-Draft draft-irtf-cfrg-bbs-per-verifier-linkability-01. Internet Engineering Task Force, 2025. URL: <https://datatracker.ietf.org/doc/draft-irtf-cfrg-bbs-per-verifier-linkability/01/>.
- [KLOR20] B. Kreuter, T. Lepoint, M. Orrù, and M. Raykova. “Anonymous Tokens with Private Metadata Bit”. In: *Proceedings of the 40th Annual International Cryptology Conference*. CRYPTO ’20. 2020, pp. 308–336. DOI: 10.1007/978-3-030-56784-2_11.

- [KLSS17] S. Krenn, T. Lorünser, A. Salzer, and C. Striecks. “Towards Attribute-Based Credentials in the Cloud”. In: *Proceedings of the 16th International Conference on Cryptology and Network Security*. CANS ’17. 2017, pp. 179–202. DOI: 10.1007/978-3-030-02641-7_9.
- [KT24] T. Kohrita and P. Towa. “Zeromorph: Zero-Knowledge Multilinear-Evaluation Proofs from Homomorphic Univariate Commitments”. In: *Journal of Cryptology* 37.4 (2024), p. 38. DOI: 10.1007/S00145-024-09519-0.
- [KZG10] A. Kate, G. M. Zaverucha, and I. Goldberg. “Constant-Size Commitments to Polynomials and Their Applications”. In: *Proceedings of the 16th International Conference on the Theory and Application of Cryptology and Information Security*. ASIACRYPT ’10. 2010, pp. 177–194. DOI: 10.1007/978-3-642-17373-8_11.
- [Lab] D. Labs. URL: <https://www.dock.io/>.
- [LKWL] T. Looker, V. Kalos, A. Whitehead, and M. Lodder. *The BBS Signature Scheme*. Internet-Draft draft-irtf-cfrg-bbs-signatures-07. Work in Progress. Internet Engineering Task Force. URL: <https://datatracker.ietf.org/doc/draft-irtf-cfrg-bbs-signatures/07/>.
- [Mau09] U. M. Maurer. “Abstraction in Cryptography (Invited Talk)”. In: *Proceedings of the 29th Annual International Cryptology Conference*. CRYPTO ’09. 2009, p. 465. DOI: 10.1007/978-3-642-03356-8_27.
- [NPR99] M. Naor, B. Pinkas, and O. Reingold. “Distributed Pseudo-random Functions and KDCs”. In: *Proceedings of the 17th Annual International Conference on Theory and Application of Cryptographic Techniques*. EUROCRYPT ’99. 1999, pp. 327–346. DOI: 10.1007/3-540-48910-X_23.
- [NS04] L. Nguyen and R. Safavi-Naini. “Efficient and Provably Secure Trapdoor-Free Group Signature Schemes from Bilinear Pairings”. In: *Proceedings of the 10th International Conference on the Theory and Application of Cryptology and Information Security*. ASIACRYPT ’04. 2004, pp. 372–386. DOI: 10.1007/978-3-540-30539-2_26.
- [Nym] Nym Technologies. *What are zk-nyms?* URL: <https://nym.com/docs/network/cryptography/zk-nym>.
- [Ope] OpenWallet Foundation. *bifold-wallet: an extensible open-source React Native project designed to enhance the way we interact with digital identities*. URL: <https://github.com/openwallet-foundation/bifold-wallet>.
- [PS16] D. Pointcheval and O. Sanders. “Short Randomizable Signatures”. In: *Proceedings of the The Cryptographers’ Track at the RSA Conference 2016*. CT-RSA 2016. 2016, pp. 111–126. DOI: 10.1007/978-3-319-29485-8_7.
- [PS18] D. Pointcheval and O. Sanders. “Reassessing Security of Randomizable Signatures”. In: *Proceedings of the The Cryptographers’ Track at the RSA Conference 2018*. CT-RSA 2018. 2018, pp. 319–338. DOI: 10.1007/978-3-319-76953-0_17.
- [PZ13] C. Paquin and G. Zaverucha. *uProve Specification*. 2013. URL: <https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/U-Prove20Cryptographic20Specification20V1.1.pdf>.

- [QRW19] W. Quach, R. D. Rothblum, and D. Wichs. “Reusable Designated-Verifier NIZKs for all NP from CDH”. In: *Proceedings of the 38th Annual International Conference on the Theory and Applications of Cryptographic Techniques*. EUROCRYPT ’19. 2019, pp. 593–621. DOI: 10.1007/978-3-030-17656-3_21.
- [Qu99] M. Qu. *Sec 2: Recommended elliptic curve domain parameters*. 1999.
- [RWM] R. Robert, C. A. Wood, and T. Meunier. *Batched Token Issuance Protocol*. Internet-Draft draft-ietf-privacypass-batched-tokens-04. Work in Progress. Internet Engineering Task Force. URL: <https://datatracker.ietf.org/doc/draft-ietf-privacypass-batched-tokens/04/>.
- [Sah99] A. Sahai. “Non-Malleable Non-Interactive Zero Knowledge and Adaptive Chosen-Ciphertext Security”. In: *Proceedings of the 40th Annual IEEE Symposium on Foundations of Computer Science*. FOCS ’99. 1999, pp. 543–553. DOI: 10.1109/SFFCS.1999.814628.
- [SB] *Data Integrity BBS Cryptosuites v1.0*. Tech. rep. URL: <https://www.w3.org/TR/vc-di-bbs/>.
- [Sch01] C.-P. Schnorr. “Security of Blind Discrete Log Signatures against Interactive Attacks”. In: *Proceedings of the 3rd International Conference on Information and Communications Security*. ICICS ’01. 2001, pp. 1–12. DOI: 10.1007/3-540-45600-7_1.
- [Sho97] V. Shoup. “Lower Bounds for Discrete Logarithms and Related Problems”. In: *Proceedings of the 16th International Conference on the Theory and Application of Cryptographic Techniques*. EUROCRYPT ’97. 1997, pp. 256–266. DOI: 10.1007/3-540-69053-0_18.
- [Sig] Signal Foundation. *Technology Preview: Signal Private Group System*. URL: <https://signal.org/blog/signal-private-group-system/>.
- [SLSRSA] M. Sporny, D. Longley, M. Sabadello, D. Reed, O. Steele, and C. Allen. *Decentralized Identifiers (DIDs) v1.0*. Tech. rep. URL: <https://www.w3.org/TR/did-1.0/>.
- [SS22] T. Silde and M. Strand. “Anonymous Tokens with Public Metadata and Applications to Private Contact Tracing”. In: *Proceedings of the 26th International Conference on Financial Cryptography and Data Security*. FC ’22. 2022, pp. 179–199. DOI: 10.1007/978-3-031-18283-9_9.
- [TFS04] I. Teranishi, J. Furukawa, and K. Sako. “k-Times Anonymous Authentication (Extended Abstract)”. In: *Proceedings of the 10th International Conference on the Theory and Application of Cryptology and Information Security*. ASIACRYPT ’04. 2004, pp. 308–322. DOI: 10.1007/978-3-540-30539-2_22.
- [TG23] L. Tulloch and I. Goldberg. “Lox: Protecting the Social Graph in Bridge Distribution”. In: *Proceedings on Privacy Enhancing Technologies* 2023.1 (2023), pp. 494–509. DOI: 10.56553/popets-2023-0029.
- [TZ23] S. Tessaro and C. Zhu. “Revisiting BBS Signatures”. In: *Proceedings of the 42nd Annual International Conference on the Theory and Applications of Cryptographic Techniques*. EUROCRYPT ’23. 2023, pp. 691–721. DOI: 10.1007/978-3-031-30589-4_24.

- [YW25] C. Yun and C. A. Wood. *Anonymous Rate-Limited Credentials*. Internet-Draft draft-yun-cfrg-arc-00. Internet Engineering Task Force, 2025. URL: <https://datatracker.ietf.org/doc/draft-yun-cfrg-arc/00/>.